

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

Bùi Khắc Trung - Trần Anh Tú

NGHIÊN CỨU PHƯƠNG PHÁP XÂY
DỰNG NEOBERT CHO TIẾNG VIỆT

KHÓA LUẬN TỐT NGHIỆP CỬ NHÂN CNTT
CHƯƠNG TRÌNH CHUẨN

Tp. Hồ Chí Minh, tháng 07/2026

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

Bùi Khắc Trung - Trần Anh Tú
22120396 - 22120400

NGHIÊN CỨU PHƯƠNG PHÁP XÂY
DỰNG NEOBERT CHO TIẾNG VIỆT

KHÓA LUẬN TỐT NGHIỆP CỬ NHÂN CNTT
CHƯƠNG TRÌNH CHUẨN

GIẢNG VIÊN HƯỚNG DẪN

TS. Nguyễn Hồng Bửu Long - TS. Lương An Vinh

Tp. Hồ Chí Minh, tháng 07/2026

Lời cam đoan

Chúng tôi xin cam đoan khóa luận tốt nghiệp với đề tài “*Nghiên cứu phương pháp xây dựng NeoBERT cho tiếng Việt*” là công trình nghiên cứu của chính nhóm chúng tôi, được thực hiện dưới sự hướng dẫn của TS. Nguyễn Hồng Bửu Long và TS. Lương An Vinh.

Các số liệu, kết quả thực nghiệm được trình bày trong khóa luận là trung thực, do chính nhóm thực hiện và chưa từng được công bố trong bất kỳ công trình nào khác trước đây. Tất cả các tài liệu, công trình nghiên cứu liên quan được kế thừa trong khóa luận đều đã được trích dẫn nguồn gốc rõ ràng, đầy đủ trong danh mục tài liệu tham khảo.

Chúng tôi xin chịu hoàn toàn trách nhiệm trước Bộ môn, Khoa và Nhà trường về lời cam đoan này.

Thành phố Hồ Chí Minh, tháng 07 năm 2026

Nhóm sinh viên thực hiện

Bùi Khắc Trung

Trần Anh Tú

Thuyết minh chỉnh sửa đề tài

Bản thuyết minh chỉnh sửa đề tài (nếu có thay đổi tên đề tài, nội dung báo cáo, ... trong cuốn báo cáo sau bảo vệ)

Nhận xét hướng dẫn

Bản nhận xét của giảng viên hướng dẫn (có chữ ký) do giáo vụ cung cấp.

Nhận xét phản biện

Bản nhận xét của giảng viên phản biện (có chữ ký) do giáo vụ cung cấp.

Lời cảm ơn

Lời đầu tiên, chúng tôi xin gửi lời cảm ơn sâu sắc đến thầy TS. Nguyễn Hồng Bửu Long và thầy TS. Lương An Vinh. Hai thầy đã cho phép nhóm thực hiện đề tài khóa luận này và dành nhiều thời gian, tận tâm hướng dẫn nhóm trong suốt quá trình nghiên cứu. Những kiến thức sâu rộng, những định hướng đúng đắn cùng các góp ý quý báu của các thầy đã giúp chúng tôi không chỉ hoàn thành tốt khóa luận mà còn trưởng thành hơn về tư duy nghiên cứu trong lĩnh vực Học sâu nói chung và Xử lý ngôn ngữ tự nhiên nói riêng.

Chúng tôi xin chân thành cảm ơn quý thầy cô Bộ môn Công nghệ tri thức đã tạo điều kiện thuận lợi về môi trường nghiên cứu trong quá trình nhóm thực hiện đề tài.

Đặc biệt, chúng tôi xin bày tỏ lòng biết ơn đến Khoa Công nghệ thông tin, Trường Đại học Khoa học tự nhiên, Đại học Quốc gia Thành phố Hồ Chí Minh vì đã mang lại một môi trường học tập và phát triển tuyệt vời. Chúng tôi cũng xin gửi lời cảm ơn đến tất cả quý thầy cô đã tận tâm truyền đạt những kiến thức nền tảng và những bài học quý giá trong suốt quá trình chúng tôi học tập tại trường.

Chúng tôi cũng xin gửi lời cảm ơn đến gia đình, bạn bè và các anh chị khóa trên đã luôn đồng hành, động viên và giúp đỡ nhóm trong suốt quá trình học tập và nghiên cứu. Chính sự quan tâm và giúp đỡ này là nguồn động lực to lớn để chúng tôi luôn cố gắng phấn đấu hoàn thành thật tốt những mục tiêu đã đặt ra.

Mặc dù đã rất nỗ lực, khóa luận khó tránh khỏi những thiếu sót. Chúng

tôi rất mong nhận được sự thông cảm cùng những ý kiến đóng góp của quý thầy cô và các bạn để khóa luận được hoàn thiện hơn.

Một lần nữa, chúng tôi xin chân thành cảm ơn tất cả những người đã giúp đỡ nhóm trong quá trình thực hiện khóa luận tốt nghiệp này.

Thành phố Hồ Chí Minh, tháng 07 năm 2026

Nhóm sinh viên thực hiện

Bùi Khắc Trung – Trần Anh Tú



fit@hcmus

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

ĐỀ CƯƠNG KHOÁ LUẬN TỐT NGHIỆP

NGHIÊN CỨU PHƯƠNG PHÁP XÂY DỰNG NEOBERT CHO TIẾNG VIỆT

1 THÔNG TIN CHUNG

Người hướng dẫn:

- TS. Nguyễn Hồng Bửu Long (Khoa Công nghệ Thông tin)
- TS. Lương An Vinh (Khoa Công nghệ Thông tin)

Nhóm Sinh viên thực hiện:

1. Bùi Khắc Trung (MSSV: 22120396)
2. Trần Anh Tú (MSSV: 22120400)

Loại đề tài: Nghiên cứu

Thời gian thực hiện: Từ 1/2026 đến 7/2026

2 NỘI DUNG THỰC HIỆN

2.1 Giới thiệu về đề tài

Trong lịch sử phát triển của lĩnh vực xử lý ngôn ngữ tự nhiên (NLP), các mô hình dạng Encoder-only như BERT và RoBERTa từng giữ vai trò nền tảng trong các bài toán hiểu ngôn ngữ. Tuy nhiên, trong khi các dòng mô hình Decoder-only (LLMs) liên tục tiến hóa với quy mô dữ liệu và kiến trúc đột phá, các bộ Encoder gần như không có sự cải thiện đáng kể trong suốt vài năm qua. Phải đến năm 2025, sự ra đời của NeoBERT [1] mới thực sự thu hẹp khoảng cách này. NeoBERT tái định nghĩa khả năng của mô hình bằng cách tích hợp các cải tiến kiến trúc tiên tiến nhất như hàm kích hoạt SwiGLU, vị trí tương đối Rotary Position Embeddings (RoPE), và chuẩn hóa Pre-RMSNorm. Đặc biệt, NeoBERT tối ưu hóa tỷ lệ chiều sâu trên chiều rộng (optimal depth-to-width ratio) với 28 lớp ẩn trên kích thước 768, cho phép mô hình đạt hiệu suất vượt trội trên chuẩn đánh giá MTEB so với các dòng BERT-large dù chỉ sở hữu 250 triệu tham số.

Mặc dù sở hữu sức mạnh suy luận vượt trội và hỗ trợ độ dài ngữ cảnh lên tới 4.096 tokens, việc thích nghi NeoBERT cho ngôn ngữ tiếng Việt vẫn đối mặt với những thách thức lớn. Việc huấn luyện lại từ đầu (from scratch) trên quy mô hàng nghìn tỷ tokens như NeoBERT gốc là điều bất khả thi về mặt tài nguyên. Bên cạnh đó, các phương pháp tiếp tục tiền huấn luyện (Continued Pre-training) thông thường dễ dẫn đến hiện tượng quên lãng thảm họa (catastrophic forgetting), làm phá vỡ các đặc trưng biểu diễn quý giá mà mô hình đã học từ dữ liệu ban đầu.

Xuất phát từ bối cảnh trên, đề tài tập trung nghiên cứu phương pháp thích nghi ngôn ngữ cho NeoBERT bằng kỹ thuật SALT (Semantic Aware Linear Transfer) [2]. Ý tưởng cốt lõi của nghiên cứu là tái sử dụng (recycling) bộ nhúng từ các mô hình tiền huấn luyện tiếng Việt chất lượng cao như ViDeBERTa [3]. Thay vì khởi tạo ngẫu nhiên, phương pháp thực hiện chuyển đổi tuyến tính dựa trên nhận thức ngữ nghĩa: đối với mỗi token không trùng lặp, thuật toán sẽ thiết lập các đường

hồi quy tuyến tính riêng biệt dựa trên tập từ vựng chung. Quá trình này giúp ánh xạ chính xác các thành phần ngữ nghĩa từ không gian của ViDeBERTa sang không gian của NeoBERT.

Thông qua đó, nghiên cứu hướng tới việc xây dựng mô hình Vietnamese NeoBERT có khả năng xử lý ngữ cảnh dài, tốc độ suy luận nhanh và độ chính xác cao. Kết quả của đề tài không chỉ mang ý nghĩa về mặt hiệu suất mà còn khẳng định tính khả thi của việc tối ưu hóa tài nguyên huấn luyện. Cụ thể, việc linh hoạt tái sử dụng không gian biểu diễn (embedding) từ các mô hình ngôn ngữ mạnh mẽ sẵn có mở ra một hướng tiếp cận tiết kiệm và hiệu quả để chuyển đổi ngôn ngữ cho các kiến trúc LLM thế hệ mới, thay vì phải đầu tư chi phí khổng lồ cho việc huấn luyện lại từ đầu.

2.2 Mục tiêu đề tài

Trong bối cảnh các mô hình ngôn ngữ lớn đang đạt được những bước tiến vượt bậc về khả năng suy luận, các kiến trúc Encoder-only vẫn đóng vai trò nền tảng không thể thay thế trong các tác vụ trích xuất đặc trưng và biểu diễn ngữ nghĩa văn bản. Tuy nhiên, các mô hình Encoder dành cho tiếng Việt hiện nay chủ yếu dựa trên những kiến trúc cũ, thiếu đi sự tối ưu từ các kỹ thuật hiện đại và bị giới hạn ở độ dài ngữ cảnh ngắn. Việc huấn luyện lại các kiến trúc thế hệ mới như NeoBERT từ đầu cho tiếng Việt đòi hỏi nguồn lực tính toán khổng lồ. Do đó, việc nghiên cứu một phương pháp thích nghi ngôn ngữ hiệu quả, ít tốn kém nhưng vẫn bảo toàn được sức mạnh của mô hình gốc là một nhu cầu cấp thiết.

Đề tài tập trung nghiên cứu việc khai thác và chuyển đổi tri thức từ các mô hình ngôn ngữ tiền huấn luyện tiếng Việt sẵn có vào kiến trúc NeoBERT thông qua kỹ thuật SALT. Mục tiêu cụ thể là sử dụng các tập neo (anchor tokens) để tính toán các đường hồi quy tuyến tính riêng biệt, từ đó ánh xạ chính xác không gian biểu diễn (embedding) của mô hình nguồn (tiêu biểu như ViDeBERTa) sang không gian vector của NeoBERT. Thông qua cách tiếp cận này, nghiên cứu hướng

đến việc xây dựng một mô hình Vietnamese NeoBERT có khả năng xử lý ngữ cảnh dài (lên tới 4.096 tokens) với tốc độ hội tụ nhanh và chi phí huấn luyện thấp hơn đáng kể so với các phương pháp truyền thống.

Kết quả của đề tài sẽ góp phần chứng minh hiệu quả của việc linh hoạt tái sử dụng không gian biểu diễn từ các mô hình ngôn ngữ mạnh mẽ sẵn có để chuyển giao tri thức, giúp rút ngắn thời gian hội tụ và tránh hiện tượng quên lãng thảm họa (catastrophic forgetting). Nghiên cứu không chỉ cung cấp một bộ Encoder tiên tiến cho các tác vụ xử lý ngôn ngữ thực tế tại Việt Nam, mà còn đề xuất một quy trình thích nghi ngôn ngữ tối ưu, tạo tiền đề cho việc cập nhật và ứng dụng các kiến trúc mô hình ngôn ngữ thế hệ mới một cách linh hoạt, tiết kiệm và bền vững.

2.3 Phạm vi của đề tài

Đề tài tập trung vào việc nghiên cứu thực nghiệm phương pháp thích nghi mô hình ngôn ngữ thế hệ mới NeoBERT cho ngôn ngữ tiếng Việt bằng cách tái sử dụng không gian biểu diễn từ các mô hình tiền huấn luyện sẵn có. Phạm vi cụ thể bao gồm các thành phần sau:

Đối tượng nghiên cứu chính:

- *Kiến trúc mô hình NeoBERT*: Tập trung vào các cải tiến hiện đại như SwiGLU activation, RoPE, và cấu trúc Pre-RMSNorm với khả năng hỗ trợ ngữ cảnh lên đến 4.096 tokens.
- *Thuật toán SALT*: Nghiên cứu cơ chế trích xuất tập neo (anchor tokens) dựa trên độ tương đồng ngữ nghĩa và phương pháp hồi quy tuyến tính (Linear Least Squares Transform) để ánh xạ không gian vector.
- *Mô hình nguồn (Source Model)*: Sử dụng bộ nhúng đã được tối ưu cho tiếng Việt từ mô hình ViDeBERTa để làm cơ sở chuyển đổi tri thức.

Tập dữ liệu thực nghiệm:

- *Ngữ liệu tiền huấn luyện*: Sử dụng các tập dữ liệu văn bản tiếng Việt quy mô lớn, được trích xuất và làm sạch từ Wikipedia và các trang tin tức tổng hợp nhằm phục vụ giai đoạn huấn luyện thích nghi (Language Adaptive Continual Pre-training).
- *Dữ liệu đánh giá*: Sử dụng các bộ benchmark tiêu chuẩn cho tiếng Việt như VLSP (phân loại văn bản, nhận dạng thực thể) và các tập dữ liệu con trong MTEB (Massive Text Embedding Benchmark) để đo lường khả năng biểu diễn ngữ nghĩa.

Các giới hạn và ràng buộc:

- *Về phương pháp*: Nghiên cứu không thực hiện huấn luyện mô hình NeoBERT từ đầu do hạn chế về tài nguyên tính toán; thay vào đó, tập trung tối ưu hóa quá trình chuyển đổi không gian embedding.
- *Về tác vụ*: Đề tài tập trung vào các bài toán thuộc nhóm Hiểu ngôn ngữ tự nhiên (NLU) và Trích xuất đặc trưng (Feature Extraction), không bao gồm các bài toán sinh văn bản tự do (NLG) của các mô hình Decoder-only.

2.4 Cách tiếp cận dự kiến

Các nghiên cứu liên quan:

Trong những năm qua, hướng nghiên cứu về các mô hình Encoder-only đã đạt được nhiều thành tựu quan trọng. Điển hình là kiến trúc BERT [4] và các biến thể cải tiến như RoBERTa [5] đã thiết lập nền tảng vững chắc cho việc biểu diễn ngữ nghĩa. Tại Việt Nam, các mô hình như PhoBERT hay ViDeBERTa [3] đã tận dụng phương pháp huấn luyện tiếp tục (Continued Pre-training) trên quy mô dữ liệu lớn để tối ưu hóa khả năng xử lý tiếng Việt. Tuy nhiên, các mô hình này chủ yếu dựa trên kiến trúc Transformer cũ, gặp hạn chế về tốc độ huấn luyện và độ dài ngữ cảnh.

Gần đây, nghiên cứu về NeoBERT [1] đã tái định nghĩa sức mạnh của Encoder

bằng cách tích hợp các thành phần hiện đại. NeoBERT chứng minh rằng một mô hình quy mô 250 triệu tham số, nếu được tối ưu về tỷ lệ chiều sâu-chiều rộng và huấn luyện trên 2,1 nghìn tỷ tokens, có thể vượt qua hiệu suất của các mô hình lớn hơn gấp nhiều lần trên chuẩn đánh giá MTEB.

Bên cạnh đó, bài toán thích nghi ngôn ngữ (Cross-lingual Transfer) đã có bước đột phá với kỹ thuật SALT [2]. Thay vì thay thế tập từ vựng một cách máy móc, SALT đề xuất cơ chế tái sử dụng bộ nhúng từ các mô hình tiền huấn luyện của ngôn ngữ mục tiêu. Kết quả thực nghiệm cho thấy SALT giúp tăng tốc độ hội tụ nhanh hơn và đạt mức tổn thất (loss) thấp hơn so với các phương pháp khởi tạo truyền thống, đồng thời duy trì khả năng liên kết tri thức xuyên ngôn ngữ.

Phương pháp thực hiện dự kiến:

Dựa trên việc kế thừa sức mạnh kiến trúc của NeoBERT và phương pháp chuyển đổi hiệu quả của SALT, đề tài đề xuất quy trình xây dựng mô hình Vietnamese NeoBERT thông qua các bước sau:

- 1. Căn chỉnh không gian (Alignment Stage):** Sử dụng bộ nhúng tiếng Việt từ ViDeBERTa làm nguồn. Thông qua các bộ nhúng tĩnh (như fastText), thuật toán sẽ trích xuất một tập neo (Anchor Extraction) gồm các token có sự tương đồng cao về ngữ nghĩa giữa từ điển của NeoBERT và ViDeBERTa.
- 2. Chuyển đổi tuyến tính nhận thức ngữ nghĩa:** Đối với mỗi token không trùng lặp trong từ điển tiếng Việt, hệ thống thiết lập các đường hồi quy tuyến tính riêng biệt (unique regression lines) thông qua phương pháp Bình phương tối thiểu (Linear Least Squares). Bước này giúp chuyển đổi trực tiếp các biểu diễn ngữ nghĩa từ ViDeBERTa vào đúng hệ tọa độ không gian mà NeoBERT đã được học.
- 3. Huấn luyện thích nghi có chọn lọc:** Tiến hành huấn luyện tiếp tục trên ngữ liệu tiếng Việt. Để ngăn chặn hiện tượng quên lãng thảm họa, mô hình áp dụng chiến lược đóng băng (freezing) một phần các lớp Transformer và sử

dụng bộ lập lịch tốc độ học (learning rate scheduler) tối ưu nhằm tinh chỉnh lớp nhúng mới sao cho tương thích hoàn toàn với các lớp ẩn phía trên.

Sự khác biệt so với các nghiên cứu trước đây:

So với các phương pháp xây dựng mô hình tiếng Việt truyền thống, cách tiếp cận này không yêu cầu huấn luyện lại từ đầu mà thực hiện ánh xạ toán học để tái sử dụng tri thức. Điểm đột phá của đề tài là việc áp dụng SALT lên kiến trúc NeoBERT - một kiến trúc hiện đại hỗ trợ ngữ cảnh dài 4.096 tokens và sử dụng FlashAttention-2, thay vì áp dụng trên các kiến trúc BERT cũ. Điều này kỳ vọng sẽ tạo ra một bộ Encoder mạnh mẽ, tối ưu tài nguyên và có tốc độ suy luận nhanh nhất cho tiếng Việt.

2.5 Kết quả dự kiến của đề tài

Đề tài dự kiến đạt được các kết quả cụ thể về mặt định lượng, sản phẩm và đóng góp khoa học như sau:

Số liệu định lượng:

- *Hiệu suất mô hình:* Vietnamese NeoBERT dự kiến đạt kết quả tương đương hoặc vượt trội so với các mô hình baseline hiện tại (như PhoBERT, ViDeBERTa) trên các bài toán NLU thuộc bộ benchmark VLSP và tập dữ liệu MTEB tiếng Việt.
- *Tốc độ hội tụ và chi phí:* Nhờ việc khởi tạo lớp nhúng bằng SALT, quá trình huấn luyện dự kiến có tốc độ hội tụ nhanh hơn, giảm từ 40% đến 50% số lượng token cần thiết so với việc huấn luyện lại từ đầu hoặc tiếp tục tiền huấn luyện không có định hướng.
- *Tốc độ thực thi (Throughput):* Nhờ kiến trúc tối ưu (optimal depth-to-width), mô hình dự kiến đạt tốc độ suy luận nhanh hơn khoảng 30% - 45% so với các mô hình Encoder-large truyền thống trên cùng một cấu hình phần cứng.

- *Khả năng xử lý ngữ cảnh:* Mô hình xử lý ổn định các văn bản có độ dài lên tới 4.096 tokens, khắc phục được rào cản 512 tokens của các kiến trúc tiền nhiệm.

Sản phẩm đầu ra:

- Trọng số mô hình (Model Weights) Vietnamese NeoBERT hoàn chỉnh (250M tham số), tương thích với thư viện Hugging Face Transformers.
- Bộ mã nguồn (Source Code) bao gồm thuật toán trích xuất tập neo, hàm chuyển đổi tuyến tính SALT và kịch bản huấn luyện.
- Báo cáo thực nghiệm chi tiết so sánh độ chính xác, tốc độ và biểu đồ tổn thất (loss curve) giữa các phương pháp.

Đóng góp khoa học:

- Khẳng định khả năng ứng dụng của kiến trúc NeoBERT kết hợp với kỹ thuật SALT trong việc giải quyết bài toán thiếu hụt tài nguyên tính toán cho các ngôn ngữ ít nguồn lực.
- Kết quả nghiên cứu có triển vọng phát triển thành bài báo khoa học công bố tại các hội nghị chuyên ngành uy tín trong nước và quốc tế.

2.6 Kế hoạch thực hiện

Thời gian	Nội dung công việc	Người thực hiện
03/01/2026 – 20/01/2026	Khảo sát các tài liệu nghiên cứu về kiến trúc NeoBERT, thuật toán SALT và các phương pháp Cross-lingual Transfer.	Trung, Tú
21/01/2026 – 16/02/2026	Thu thập và tiền xử lý ngữ liệu tiếng Việt; xây dựng pipeline; chuẩn bị môi trường huấn luyện.	Trung, Tú
16/02/2026 – 10/03/2026	Cài đặt thuật toán trích xuất tập neo; lập trình mô-đun hồi quy tuyến tính (Least Squares) cho các non-shared tokens.	Trung
	Huấn luyện mô hình NeoBert baseline trên dữ liệu tiếng Việt mà không dùng SALT.	Tú
11/03/2026 – 31/04/2026	Thực hiện giai đoạn ánh xạ không gian vector và tiến hành huấn luyện thích nghi (Language Adaptive Continual Pre-training). Bên cạnh đó thực nghiệm đánh giá mô hình NeoBert baseline về cả hiệu suất và chi phí huấn luyện.	Trung, Tú
01/04/2026 – 15/04/2026	Thực nghiệm đánh giá mô hình NeoBert có sử dụng SALT và so sánh kết quả với baseline. Đánh giá cả hai mô hình trên cả quá trình huấn luyện và hiệu suất của mô hình.	Trung, Tú
16/04/2026 – 15/05/2026	Thực hiện các thí nghiệm để so sánh mô hình NeoBert tiếng Việt với các mô hình khác. Đánh giá và phân tích kết quả của thí nghiệm.	Trung, Tú
16/05/2026 – 28/07/2026	Viết báo cáo khóa luận, trực quan hóa kết quả bằng biểu đồ và chuẩn bị slide bảo vệ.	Trung, Tú

Tài liệu

- [1] L. L. Breton, Q. Fournier, J. X. Morris, M. E. Mezouar, and S. Chandar, "NeoBERT: A Next-Generation BERT," *arXiv preprint arXiv:2502.19587v2*, 2025.
- [2] S. Lee, S. Hong, H. Moon, and H. Lim, "Semantic Aware Linear Transfer by Recycling Pre-trained Language Models for Cross-lingual Transfer," *arXiv preprint arXiv:2505.10945v2*, 2025.
- [3] V. D. Lai, C. V. Nguyen, N. T. Ngo, T. Nguyen, F. Dernoncourt, R. A. Rossi, and T. H. Nguyen, "ViDeBERTa: A Powerful Pre-trained Language Model

for Vietnamese," in *Findings of the Association for Computational Linguistics: EACL 2023*, pp. 1032–1043, 2023.

- [4] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics*, pp. 4171–4186, 2019.
- [5] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "RoBERTa: A Robustly Optimized BERT Pretraining Approach," *arXiv preprint arXiv:1907.11692*, 2019.

XÁC NHẬN
CỦA NGƯỜI HƯỚNG DẪN
(Ký và ghi rõ họ tên)

TP. Hồ Chí Minh, ngày... tháng... năm...
NHÓM SINH VIÊN THỰC HIỆN
(Ký và ghi rõ họ tên)

Danh sách các từ viết tắt

KÝ HIỆU	TÊN TIẾNG ANH
CPT	Continued Pre-training
EM	Exact Match
FFN	Feed-Forward Network
GELU	Gaussian Error Linear Unit
GLU	Gated Linear Unit
LLM	Large Language Model
MCC	Matthews Correlation Coefficient
MLM	Masked Language Modeling
MRC	Machine Reading Comprehension
MRR	Mean Reciprocal Rank
MTEB	Massive Text Embedding Benchmark
nDCG	Normalized Discounted Cumulative Gain
NLP	Natural Language Processing
NLU	Natural Language Understanding
NSP	Next Sentence Prediction
PLM	Pre-trained Language Model
PPL	Perplexity
PPPL	Pseudo-Perplexity
RMSNorm	Root Mean Square Normalization
RoPE	Rotary Position Embeddings
SALT	Semantic Aware Linear Transfer
SiLU	Sigmoid Linear Unit
WSD	Warmup-Stable-Decay

Mục lục

Lời cam đoan	i
Thuyết minh chỉnh sửa đề tài	ii
Nhận xét của GV hướng dẫn	iii
Nhận xét của GV phản biện	iv
Lời cảm ơn	v
Đề cương	vii
Danh sách các từ viết tắt	xvii
Mục lục	xvii
Danh sách hình	xxii
Danh sách bảng	xxiii
Tóm tắt	xxiii
1 Tổng quan	1
1.1 Bối cảnh và sự cần thiết của đề tài	1
1.2 Mục tiêu nghiên cứu	3
1.3 Phạm vi nghiên cứu	4

1.4	Đóng góp của đề tài	5
1.5	Cấu trúc đề tài	6
2	Cơ sở lý thuyết và các công trình liên quan	7
2.1	Kiến trúc Transformer và mô hình Encoder-only	8
2.1.1	Cơ chế tự chú ý trong kiến trúc Transformer	8
2.1.2	BERT và mô hình hóa ngôn ngữ có mặt nạ	9
2.1.3	Giới hạn của các mô hình Encoder thế hệ cũ	10
2.2	Kiến trúc NeoBERT	11
2.2.1	Tổng quan	11
2.2.2	Tỷ lệ chiều sâu trên chiều rộng tối ưu	13
2.2.3	Mã hóa vị trí quay (Rotary Position Embeddings)	14
2.2.4	Hàm kích hoạt SwiGLU	18
2.2.5	Chuẩn hóa Pre-RMSNorm	19
2.2.6	Dữ liệu và chiến lược tiền huấn luyện	21
2.3	Chuyển giao không gian nhúng và thích nghi ngôn ngữ	22
2.3.1	Hình học của không gian biểu diễn	23
2.3.2	Thích nghi ngôn ngữ và rủi ro quên lãng	23
2.3.3	Khởi tạo nhúng có thông tin	24
2.3.4	Semantic Aware Linear Transfer (SALT)	24
2.4	Nhận xét định hướng	25
3	Phương pháp xây dựng Vietnamese NeoBERT	27
3.1	Tổng quan phương pháp	27
3.1.1	Đặt vấn đề	27
3.1.2	Mô hình nguồn và mô hình hiển tặng	28
3.1.3	Quy trình ba giai đoạn	28
3.2	Lựa chọn mô hình hiển tặng và bộ tách từ	29
3.2.1	Vai trò của bộ tách từ trong mô hình ngôn ngữ	29
3.2.2	So sánh PhoBERT và ViDeBERTa làm mô hình hiển tặng	30
3.3	Cắt tỉa và đồng bộ từ vựng	31

3.4	Trích xuất tập neo (anchor set)	32
3.4.1	Vai trò của tập neo	32
3.4.2	Neo trùng lặp bề mặt và neo số	33
3.4.3	Mở rộng tập neo bằng cặp dịch ba tầng	34
3.5	Phép chiếu SALT cho lớp nhúng và lớp đầu ra	35
3.5.1	Lựa chọn neo bằng Sparsemax	36
3.5.2	Ánh xạ tuyến tính cục bộ theo từng token	37
3.5.3	Áp dụng cho cả lớp nhúng đầu vào và lớp đầu ra	38
3.5.4	Hiệu chỉnh chuẩn của lớp nhúng	39
3.5.5	Khởi tạo độ chệch đầu ra theo tần suất	39
3.5.6	Co trọng số đầu ra để tiên nghiệm dẫn dắt giai đoạn đầu	41
3.6	Giai đoạn căn chỉnh đóng băng	43
3.7	Tiếp tục tiền huấn luyện thích nghi	43
3.8	Tổng kết phương pháp	45
4	Thực nghiệm và Đánh giá	46
4.1	Môi trường và dữ liệu thực nghiệm	46
4.1.1	Môi trường thực nghiệm	46
4.1.2	Dữ liệu tiền huấn luyện	47
4.1.3	Dữ liệu đánh giá hạ nguồn	47
4.2	Cấu hình huấn luyện	48
4.3	Độ đo đánh giá	50
4.4	So sánh các phương án khởi tạo	51
4.4.1	Kiểm chứng hình học của phép chiếu SALT	51
4.4.2	Lựa chọn hệ số co trọng số đầu ra	53
4.4.3	Lựa chọn chiến lược khởi tạo hai ma trận từ vựng	54
4.5	Kết quả của mô hình cuối	56
4.5.1	Diễn tiến theo ngân sách huấn luyện	56
4.5.2	Đánh giá đa nhiệm tại mốc 5 tỷ token	59
4.5.3	Mở rộng ngữ cảnh lên 4.096 token	61

4.6	Thảo luận	62
5	Kết luận và Hướng phát triển	64
5.1	Kết luận	64
5.2	Hạn chế của đề tài	66
5.3	Hướng phát triển	66
	Tài liệu tham khảo	68
	Phụ lục	75
A	Bộ lọc chính tả tiếng Việt cho khai thác neo	76
B	Định nghĩa hình thức các độ đo đánh giá	78

Danh sách hình

2.1	Luồng xử lý của mô hình Encoder-only.	9
2.2	Kiến trúc NeoBERT [5].	12
2.3	Cơ chế mã hóa vị trí quay (RoPE)	16
2.4	Cấu trúc cổng SwiGLU [16].	19
2.5	Sơ đồ tổng quan SALT	25
3.1	Quy trình ba giai đoạn xây dựng ViNeoBERT.	29
3.2	Lịch tốc độ học Warmup–Stable–Decay (WSD).	44
4.1	Lớp nhúng tiếng Việt trước và sau phép chiếu SALT.	52
4.2	Mất mát MLM theo hệ số co γ	54
4.3	So sánh ba chiến lược khởi tạo.	55
4.4	Đường học giai đoạn CPT (WSD).	57
4.5	Điểm F1 trên UIT-ViQuAD theo số token CPT.	58
4.6	Pseudo-perplexity theo độ dài chuỗi, trước và sau pha mở rộng 4.096 token.	62

Danh sách bảng

2.1	Cấu hình NeoBERT [5].	21
3.1	So sánh bộ tách từ của PhoBERT và ViDeBERTa.	31
4.1	Thống kê dữ liệu thực nghiệm.	47
4.2	Các tác vụ đánh giá hạ nguồn.	49
4.3	Các mô hình Encoder được đánh giá.	49
4.4	Siêu tham số huấn luyện.	50
4.5	Lượng dữ liệu tiếng Việt đã duyệt.	59
4.6	Kết quả đánh giá đa nhiệm tại mốc 5 tỷ token (giá trị tốt nhất in đậm).	59

Tóm tắt

Các mô hình Encoder-only như BERT giữ vai trò nền tảng cho các bài toán hiểu ngôn ngữ tự nhiên, nhưng trong nhiều năm gần như không được cập nhật về kiến trúc. NeoBERT (2025) thu hẹp khoảng cách này bằng cách tích hợp các cải tiến từ thế hệ mô hình ngôn ngữ lớn – RoPE, SwiGLU, Pre-RMSNorm, tỷ lệ sâu–hẹp tối ưu và ngữ cảnh 4.096 token – song chỉ được tiền huấn luyện trên tiếng Anh, trong khi các Encoder tiếng Việt hiện có vẫn dựa trên kiến trúc thế hệ cũ.

Khóa luận xây dựng **ViNeoBERT** – mô hình NeoBERT cho tiếng Việt – bằng quy trình thích nghi ngôn ngữ ba giai đoạn thay vì huấn luyện lại từ đầu: (1) khởi tạo lớp nhúng và lớp đầu ra bằng kỹ thuật Semantic Aware Linear Transfer (SALT), tái sử dụng không gian nhúng của mô hình tiếng Việt ViDeBERTa thông qua các ánh xạ tuyến tính cục bộ theo từng token; (2) căn chỉnh đóng băng: chỉ huấn luyện hai ma trận vừa khởi tạo trên phần thân cố định; và (3) tiếp tục tiền huấn luyện trên ngữ liệu CulturaX tiếng Việt theo lịch tốc độ học Warmup–Stable–Decay. So với SALT gốc, đề tài đưa ra ba cải tiến: mở rộng tập neo bằng cặp dịch Anh–Việt khai thác tự động (kèm cơ chế lọc từ đồng tự khác nghĩa), khởi tạo riêng lớp đầu ra tách rời với độ chệch theo tần suất từ đơn và hệ số co trọng số, cùng giai đoạn căn chỉnh đóng băng.

Thực nghiệm so sánh có kiểm soát ở cùng ngân sách huấn luyện xác nhận từng lựa chọn thiết kế, từ hệ số co trọng số đầu ra tới khởi tạo riêng hai ma trận và giai đoạn căn chỉnh đóng băng. Trên bộ đánh giá gồm mười hai tác vụ hiểu tiếng Việt (phân loại, suy luận, gán nhãn chuỗi, tương đồng

ngữ nghĩa và biểu diễn câu), ViNeoBERT sau 5 tỷ token ($\sim 20\text{GB}$) đạt kết quả ngang ngửa PhoBERT-base-v2 ở đa số tác vụ và thắng ở gán nhãn từ loại – dù lượng tiếng Việt tiếp xúc chỉ bằng $\sim 3,5\%$ so với PhoBERT và ít hơn ~ 27 lần so với phần tiếng Việt trong XLM-R. Trên đọc hiểu UIT-ViQuAD, mô hình đạt 76,6 điểm F1 – hơn XLM-R-base 3,3 điểm và cạnh tranh với PhoBERT-large lớn gấp 1,5 lần. Ngoài ra, một pha mở rộng ngắn ~ 200 triệu token ở độ dài 4.096 token giữ pseudo-perplexity ổn định trên toàn dải ngữ cảnh, biến trần 4.096 token của kiến trúc thành năng lực sử dụng được. Kết quả cho thấy tái sử dụng không gian biểu diễn sẵn có là con đường tiết kiệm và hiệu quả để đưa các kiến trúc Encoder thế hệ mới đến với tiếng Việt.

Từ khóa: NeoBERT, tiếng Việt, thích nghi ngôn ngữ, khởi tạo nhúng xuyên ngôn ngữ, SALT, tiếp tục tiền huấn luyện.

Chương 1

Tổng quan

1.1 Bối cảnh và sự cần thiết của đề tài

Trong lịch sử phát triển của lĩnh vực Xử lý Ngôn ngữ Tự nhiên (Natural Language Processing – NLP), các mô hình dạng Encoder-only như BERT [1] và RoBERTa [2] từng đóng vai trò nền tảng không thể thiếu cho các bài toán hiểu ngôn ngữ (Natural Language Understanding – NLU) như phân loại văn bản, nhận dạng thực thể có tên, suy luận ngôn ngữ tự nhiên và trả lời câu hỏi. Nhờ cơ chế mã hóa hai chiều (bidirectional encoding), các mô hình này có khả năng nắm bắt ngữ cảnh từ cả hai phía của một chuỗi đầu vào, tạo ra biểu diễn ngữ nghĩa phong phú và được ứng dụng rộng rãi trong thực tế.

Tuy nhiên, trong khi các dòng mô hình Decoder-only – hay còn gọi là các Mô hình Ngôn ngữ Lớn (Large Language Models – LLMs) như LLaMA [3] và DeepSeek [4] – liên tục tiến hóa với quy mô dữ liệu và những đột phá về kiến trúc, các bộ Encoder-only lại gần như không có sự cải thiện đáng kể trong suốt nhiều năm qua. BERT và RoBERTa vẫn tiếp tục được sử dụng rộng rãi dù kiến thức của chúng ngày càng trở nên lỗi thời, đặc biệt trong các ứng dụng đòi hỏi hiểu ngữ cảnh dài hoặc tốc độ suy luận cao.

Sự ra đời của NeoBERT [5] vào năm 2025 đã thu hẹp đáng kể khoảng cách này. NeoBERT tái định nghĩa khả năng của các mô hình Encoder

bằng cách tích hợp các cải tiến kiến trúc tiên tiến nhất từ thế giới LLM: hàm kích hoạt SwiGLU, cơ chế mã hóa vị trí tương đối Rotary Position Embeddings (RoPE), chuẩn hóa Pre-RMSNorm, và tỷ lệ chiều sâu trên chiều rộng tối ưu (28 lớp ẩn với kích thước ẩn 768). Với khoảng 245 triệu tham số và được huấn luyện trên 2,1 nghìn tỷ token, NeoBERT vượt trội BERT_{large} và RoBERTa_{large} trên chuẩn đánh giá MTEB (Massive Text Embedding Benchmark), đồng thời hỗ trợ ngữ cảnh lên tới 4.096 token – gấp 8 lần so với BERT và RoBERTa (512 token).

Tuy nhiên, NeoBERT chỉ được huấn luyện trên ngữ liệu tiếng Anh. Đối với tiếng Việt, nhu cầu về một bộ Encoder mạnh mẽ và hiện đại ngày càng cấp thiết khi các ứng dụng NLP trong nước – từ hệ thống tìm kiếm ngữ nghĩa, phân tích cảm xúc, đến các hệ thống hỏi-đáp tự động – đòi hỏi chất lượng biểu diễn ngữ nghĩa ngày càng cao. Các mô hình Encoder tiếng Việt hiện có như PhoBERT [6] và ViDeBERTa [7] tuy hiệu quả nhưng vẫn dựa trên kiến trúc Transformer cũ, bị giới hạn ở độ dài ngữ cảnh 512 token và thiếu đi các tối ưu hóa kiến trúc hiện đại.

Việc huấn luyện lại NeoBERT từ đầu (from scratch) cho tiếng Việt trên quy mô hàng nghìn tỷ token là điều bất khả thi về mặt tài nguyên tính toán đối với hầu hết các nhóm nghiên cứu. Bên cạnh đó, phương pháp tiếp tục tiền huấn luyện (Continued Pre-training) thông thường – tức là khởi tạo ngẫu nhiên lớp nhúng mới rồi huấn luyện trực tiếp – thường dẫn đến hiện tượng *quên lãng thảm họa* (catastrophic forgetting), làm phá vỡ các đặc trưng biểu diễn quý giá mà mô hình đã học từ dữ liệu tiếng Anh.

Xuất phát từ những thách thức trên, đề tài nghiên cứu phương pháp thích nghi ngôn ngữ hiệu quả cho NeoBERT thông qua kỹ thuật Semantic Aware Linear Transfer (SALT) [8]. Thay vì huấn luyện từ đầu, SALT tái sử dụng (recycling) bộ nhúng từ mô hình tiền huấn luyện tiếng Việt chất lượng cao ViDeBERTa, chuyển đổi không gian biểu diễn sang không gian của NeoBERT thông qua các phép hồi quy tuyến tính có nhận thức ngữ nghĩa. Đây là một hướng tiếp cận tiết kiệm tài nguyên, vừa kế thừa tri thức ngôn ngữ học tiếng Việt sẵn có, vừa bảo toàn được sức mạnh kiến

trúc của NeoBERT.

1.2 Mục tiêu nghiên cứu

Đề tài đặt ra các mục tiêu nghiên cứu cụ thể như sau:

1. **Nghiên cứu và nắm vững kiến trúc NeoBERT:** Phân tích chi tiết các cải tiến kiến trúc của NeoBERT so với BERT/RoBERTa truyền thống, bao gồm SwiGLU, RoPE, Pre-RMSNorm và tỷ lệ chiều sâu trên chiều rộng tối ưu; từ đó xác định những điểm cần điều chỉnh khi thích nghi sang tiếng Việt.
2. **Áp dụng và cải tiến kỹ thuật SALT để chuyển giao không gian biểu diễn:** Nghiên cứu và triển khai thuật toán SALT – bao gồm trích xuất tập neo (anchor extraction) dựa trên độ tương đồng ngữ nghĩa và ánh xạ tuyến tính cục bộ theo phương pháp bình phương tối thiểu – nhằm ánh xạ không gian nhúng từ ViDeBERTa sang không gian véc-tơ của NeoBERT; đồng thời cải tiến cho cặp ngôn ngữ Anh–Việt: mở rộng tập neo bằng cặp dịch và khởi tạo riêng lớp đầu ra tách rời.
3. **Xây dựng mô hình Vietnamese NeoBERT (ViNeoBERT):** Kết hợp lớp nhúng và lớp đầu ra đã được khởi tạo bằng SALT với phần thân NeoBERT gốc, sau đó huấn luyện qua giai đoạn căn chỉnh đóng băng và tiếp tục tiền huấn luyện thích nghi (Continued Pre-training) trên ngữ liệu tiếng Việt quy mô lớn theo lịch tốc độ học Warmup–Stable–Decay.
4. **Đánh giá hiệu quả của phương pháp:** Đánh giá ViNeoBERT trên một bộ tác vụ hiểu ngôn ngữ tiếng Việt đa dạng – gồm hỏi đáp đọc hiểu (UIT-ViQuAD), suy luận, phân loại văn bản, gán nhãn chuỗi, tương đồng ngữ nghĩa và biểu diễn câu (Retrieval, Reranking)

– so sánh với các mô hình tiếng Việt và đa ngữ hiện có (PhoBERT, XLM-R); đồng thời so sánh các phương án khởi tạo khác nhau ở cùng ngân sách huấn luyện để định lượng đóng góp của từng thành phần.

5. **Kiểm chứng tính hiệu quả của việc tái sử dụng tri thức:** Thông qua kết quả thực nghiệm, xác minh rằng SALT giúp giảm đáng kể số lượng token huấn luyện cần thiết và ngăn chặn hiện tượng quên lãng thảm họa so với phương pháp khởi tạo ngẫu nhiên.

1.3 Phạm vi nghiên cứu

Đề tài tập trung nghiên cứu thực nghiệm trong các giới hạn sau:

Đối tượng nghiên cứu chính bao gồm: (1) kiến trúc NeoBERT với trọng số gốc được công bố công khai; (2) thuật toán SALT với cơ chế trích xuất tập neo và ánh xạ tuyến tính cục bộ; (3) mô hình hiến tặng (donor) ViDeBERTa cung cấp không gian nhúng tiếng Việt cho bước chuyển giao.

Dữ liệu thực nghiệm gồm phần tiếng Việt của kho ngữ liệu web đa ngữ CulturaX cho giai đoạn huấn luyện thích nghi, cùng một bộ tác vụ đánh giá tiếng Việt đa dạng cho bước tinh chỉnh hạ nguồn – từ hỏi đáp đọc hiểu (UIT-ViQuAD), suy luận ngôn ngữ tự nhiên (XNLI, ViANLI, QNLI), phân loại văn bản (UIT-VSFC, UIT-VSMEC, UIT-ViCTSD), tính chấp nhận về ngữ pháp (ViCoLA-syn), gán nhãn từ loại (UD-VTB), tương đồng ngữ nghĩa (STS, Paraphrase), tới biểu diễn câu (Retrieval, Reranking).

Các ràng buộc và giới hạn: Nghiên cứu *không* thực hiện huấn luyện NeoBERT từ đầu do hạn chế tài nguyên tính toán. Phạm vi bài toán chỉ bao gồm các tác vụ NLU và trích xuất đặc trưng (Feature Extraction), không bao gồm các bài toán sinh văn bản tự do của các mô hình Decoder-only. Mô hình kế thừa kiến trúc hỗ trợ ngữ cảnh tới 4.096 token của NeoBERT và được huấn luyện bổ sung một pha ngắn ở độ dài này; việc kiểm chứng ngữ cảnh dài giới hạn ở độ đo nội tại (pseudo-perplexity theo độ dài chuỗi), còn đánh giá hạ nguồn chuyên biệt cho ngữ cảnh dài được

để lại như hướng phát triển.

1.4 Đóng góp của đề tài

Đề tài đưa ra các đóng góp cụ thể sau:

- **Mô hình Vietnamese NeoBERT (ViNeoBERT):** Cung cấp một bộ Encoder hiện đại khoảng 245 triệu tham số cho tiếng Việt, kế thừa các cải tiến kiến trúc của NeoBERT (RoPE, SwiGLU, Pre-RMSNorm, FlashAttention) cùng trần ngữ cảnh 4.096 token, tương thích với thư viện Hugging Face Transformers và sẵn sàng sử dụng cho các bài toán NLU thực tế.
- **Quy trình thích nghi ngôn ngữ cải tiến:** Đề xuất và kiểm chứng một quy trình chuyển giao ba giai đoạn áp dụng SALT lên kiến trúc NeoBERT – lần đầu tiên cho một kiến trúc Encoder thế hệ mới – với ba cải tiến so với nghiên cứu gốc: (1) mở rộng tập neo bằng cặp dịch Anh–Việt khai thác tự động; (2) khởi tạo riêng lớp đầu ra tách rời, kèm độ chệch theo tần suất từ đơn và hệ số co trọng số để tiên nghiệm dẫn dắt giai đoạn đầu; (3) bổ sung giai đoạn căn chỉnh đóng băng trước khi tiếp tục tiền huấn luyện.
- **Bộ mã nguồn công khai:** Bao gồm thuật toán trích xuất tập neo, các mô-đun chiếu SALT cho lớp nhúng và lớp đầu ra, cùng kịch bản huấn luyện thích nghi, phục vụ cho cộng đồng nghiên cứu NLP tiếng Việt.
- **Đóng góp khoa học:** Kết quả thực nghiệm khẳng định tính khả thi của việc tái sử dụng không gian biểu diễn từ các mô hình sẵn có để chuyển giao tri thức cho các ngôn ngữ ít nguồn lực, mở ra một hướng tiếp cận tiết kiệm chi phí cho việc cập nhật và áp dụng các kiến trúc thế hệ mới.

1.5 Cấu trúc đề tài

Phần còn lại của khóa luận được tổ chức như sau:

- **Chương 1 – Tổng quan:** Giới thiệu bối cảnh và sự cần thiết của đề tài, mục tiêu nghiên cứu, phạm vi nghiên cứu và các đóng góp chính.
- **Chương 2 – Cơ sở lý thuyết và các công trình liên quan:** Trình bày kiến trúc Transformer và dòng mô hình Encoder-only, các đặc điểm chính của NeoBERT, cơ sở hình học của không gian biểu diễn, cùng các phương pháp khởi tạo nhúng xuyên ngôn ngữ dẫn tới kỹ thuật SALT.
- **Chương 3 – Phương pháp xây dựng Vietnamese NeoBERT:** Trình bày chi tiết quy trình ba giai đoạn của đề tài: cắt tỉa từ vựng và trích xuất tập neo, phép chiếu SALT cho cả lớp nhúng lẫn lớp đầu ra, giai đoạn căn chỉnh đóng băng và tiếp tục tiền huấn luyện theo lịch WSD.
- **Chương 4 – Thực nghiệm và Đánh giá:** Mô tả dữ liệu, cấu hình huấn luyện, các độ đo và phân tích kết quả thực nghiệm, bao gồm so sánh các phương án khởi tạo và đánh giá trên tác vụ hạ nguồn.
- **Chương 5 – Kết luận và Hướng phát triển:** Tổng kết các kết quả đạt được so với mục tiêu đề ra và đề xuất các hướng phát triển tiếp theo.

Chương 2

Cơ sở lý thuyết và các công trình liên quan

Chương này trình bày nền tảng lý thuyết và các công trình liên quan làm cơ sở cho phương pháp được đề xuất. Trước hết, chúng tôi khảo sát kiến trúc Transformer cùng dòng mô hình Encoder-only đã trở thành chuẩn mực cho các bài toán hiểu ngôn ngữ (Mục 2.1). Tiếp theo, chúng tôi trình bày các đặc điểm chính của NeoBERT – mô hình nền mà đề tài kế thừa – cùng những cải tiến có ảnh hưởng trực tiếp tới hướng chuyển giao tiếng Việt (Mục 2.2). Sau đó, chúng tôi trình bày cơ sở hình học của không gian biểu diễn, bài toán thích nghi ngôn ngữ và các phương pháp khởi tạo nhúng xuyên ngôn ngữ, dẫn tới kỹ thuật Semantic Aware Linear Transfer (SALT) mà đề tài áp dụng (Mục 2.3). Cuối cùng, chương đưa ra một số nhận xét định hướng cho phương pháp được trình bày ở chương tiếp theo.

2.1 Kiến trúc Transformer và mô hình Encoder-only

2.1.1 Cơ chế tự chú ý trong kiến trúc Transformer

Transformer được Vaswani và cộng sự [9] giới thiệu như một kiến trúc xử lý chuỗi không dùng hồi tiếp. Thay vì đọc token tuần tự như RNN/LSTM, Transformer cho phép mỗi vị trí quan sát trực tiếp các vị trí còn lại thông qua cơ chế tự chú ý (self-attention). Đây là lý do kiến trúc này vừa song song hóa tốt trên GPU, vừa học được các phụ thuộc xa trong câu.

Với ma trận trạng thái ẩn X , mô hình tạo ba phép chiếu tuyến tính gồm truy vấn Q , khóa K và giá trị V . Trọng số chú ý được tính bằng tích vô hướng có tỷ lệ:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \quad (2.1)$$

Trong đó, ma trận $QK^\top$ biểu diễn mức liên hệ giữa mọi cặp token. Vì ma trận này có kích thước $n \times n$, chi phí chú ý tăng theo $O(n^2)$ khi độ dài chuỗi tăng. Đây là một lý do các Encoder hiện đại cần kết hợp thêm kỹ thuật tối ưu bộ nhớ như FlashAttention khi mở rộng ngữ cảnh lên hàng nghìn token.

Transformer thường dùng tự chú ý đa đầu (Multi-Head Attention) để học nhiều kiểu quan hệ song song. Công thức tổng quát được viết gọn như sau:

$$\begin{aligned} \text{MultiHead}(Q, K, V) &= \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O, \\ \text{head}_i &= \text{Attention}(QW_i^Q, KW_i^K, VW_i^V). \end{aligned} \quad (2.2)$$

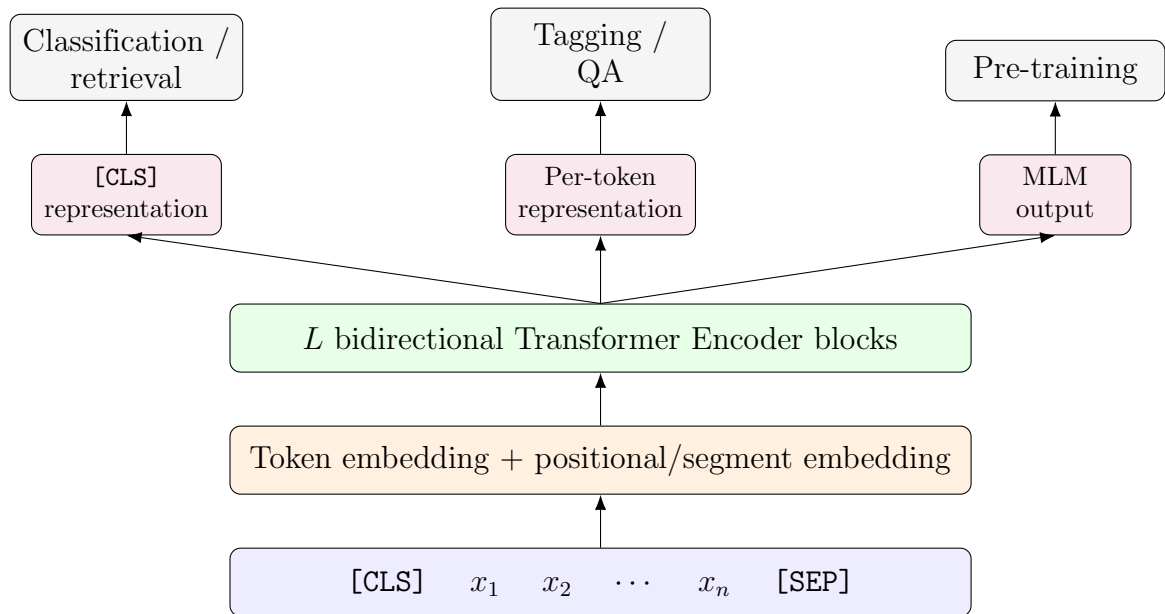
Sau attention, mỗi khối Transformer còn có mạng truyền thẳng theo vị trí, kết nối tắt và lớp chuẩn hóa. Các biến thể hiện đại chủ yếu khác nhau ở cách mã hóa vị trí, loại chuẩn hóa và hàm kích hoạt. Những điểm này sẽ

được dùng để giải thích NeoBERT ở Mục 2.2.

2.1.2 BERT và mô hình hóa ngôn ngữ có mặt nạ

Từ phần encoder của Transformer, Devlin và cộng sự [1] đề xuất BERT (Bidirectional Encoder Representations from Transformers). BERT là mô hình Encoder-only: nó không có bộ giải mã tự hồi quy và không dùng mặt nạ nhân quả, nên mỗi token có thể khai thác đồng thời ngữ cảnh bên trái lẫn bên phải. Đầu ra của BERT là một chuỗi biểu diễn theo ngữ cảnh, phù hợp với các tác vụ hiểu ngôn ngữ như phân loại văn bản, gán nhãn chuỗi, hỏi đáp trích xuất và truy hồi.

Hình 2.1 minh họa luồng xử lý điển hình của một mô hình Encoder-only kiểu BERT.



Hình 2.1: Luồng xử lý của mô hình Encoder-only.

Khác với các mô hình ngôn ngữ tự hồi quy chỉ mô hình hóa xác suất từ trái sang phải, BERT được tiền huấn luyện đồng thời trên hai tác vụ tự giám sát. Tác vụ thứ nhất và chủ yếu là mô hình hóa ngôn ngữ có mặt nạ (Masked Language Modeling – MLM): 15% token trong chuỗi được chọn ngẫu nhiên, trong đó 80% thay bằng token đặc biệt [MASK], 10% thay bằng

token ngẫu nhiên, và 10% giữ nguyên – cơ chế này giúp giảm sự bất nhất giữa huấn luyện (có [MASK]) và tinh chỉnh (không có [MASK]). Mô hình dự đoán lại token gốc dựa trên ngữ cảnh còn lại:

$$\mathcal{L}_{\text{MLM}} = - \sum_{i \in \mathcal{M}} \log P(x_i \mid x_{\setminus \mathcal{M}}) \quad (2.3)$$

Trong đó $\mathcal{M}$ là tập vị trí bị che. Cách huấn luyện này buộc mô hình học biểu diễn hai chiều thay vì chỉ dự đoán token kế tiếp. Tác vụ thứ hai là dự đoán câu tiếp theo (Next Sentence Prediction – NSP): phân loại nhị phân xem câu B có thực sự theo sau câu A trong văn bản gốc hay không. Tuy nhiên, NSP về sau được Liu và cộng sự [2] chứng minh là không cần thiết và có thể gây hại, nên RoBERTa và các mô hình hiện đại kế tiếp đã loại bỏ hoàn toàn.

Tiếp nối BERT, Liu và cộng sự [2] đề xuất RoBERTa với các cải tiến về quy trình tiền huấn luyện, nổi bật là loại bỏ NSP, dùng mặt nạ động và tăng quy mô dữ liệu. Các mô hình tiếng Việt như PhoBERT [6] và ViDeBERTa [7] cũng kế thừa hướng Encoder-only này. Do đó, khi xây dựng NeoBERT cho tiếng Việt, bài toán trọng tâm không phải thay đổi bản chất Encoder-only, mà là chuyển giao một phần thân Encoder hiện đại sang không gian từ vựng tiếng Việt.

2.1.3 Giới hạn của các mô hình Encoder thế hệ cũ

BERT và RoBERTa vẫn rất quan trọng, nhưng chúng mang nhiều giả định của giai đoạn đầu: ngữ cảnh thường giới hạn ở 512 token, mã hóa vị trí tuyệt đối khó mở rộng, và kiến trúc chưa có các thành phần hiện đại như RoPE, SwiGLU, RMSNorm hay FlashAttention. Khi độ dài chuỗi tăng từ 512 lên 4.096 token, riêng ma trận chú ý đã tăng 64 lần về số phần tử, nên việc mở rộng ngữ cảnh không thể chỉ dựa vào tăng độ dài đầu vào.

Ngoài kiến trúc, quy mô dữ liệu cũng thay đổi mạnh. BERT được huấn luyện trên BooksCorpus và Wikipedia với khoảng 13GB dữ liệu,

RoBERTa tăng lên khoảng 160GB, trong khi các Encoder gần đây như NomicBERT [10], ModernBERT [11] và NeoBERT [5] chuyển sang dữ liệu web lớn hơn, ngữ cảnh dài hơn và quy trình huấn luyện tối ưu hơn. Vì vậy, NeoBERT được chọn trong đề tài như một phần thân Encoder hiện đại để thích nghi sang tiếng Việt thay vì tiếp tục dựa trên thể hệ BERT cũ.

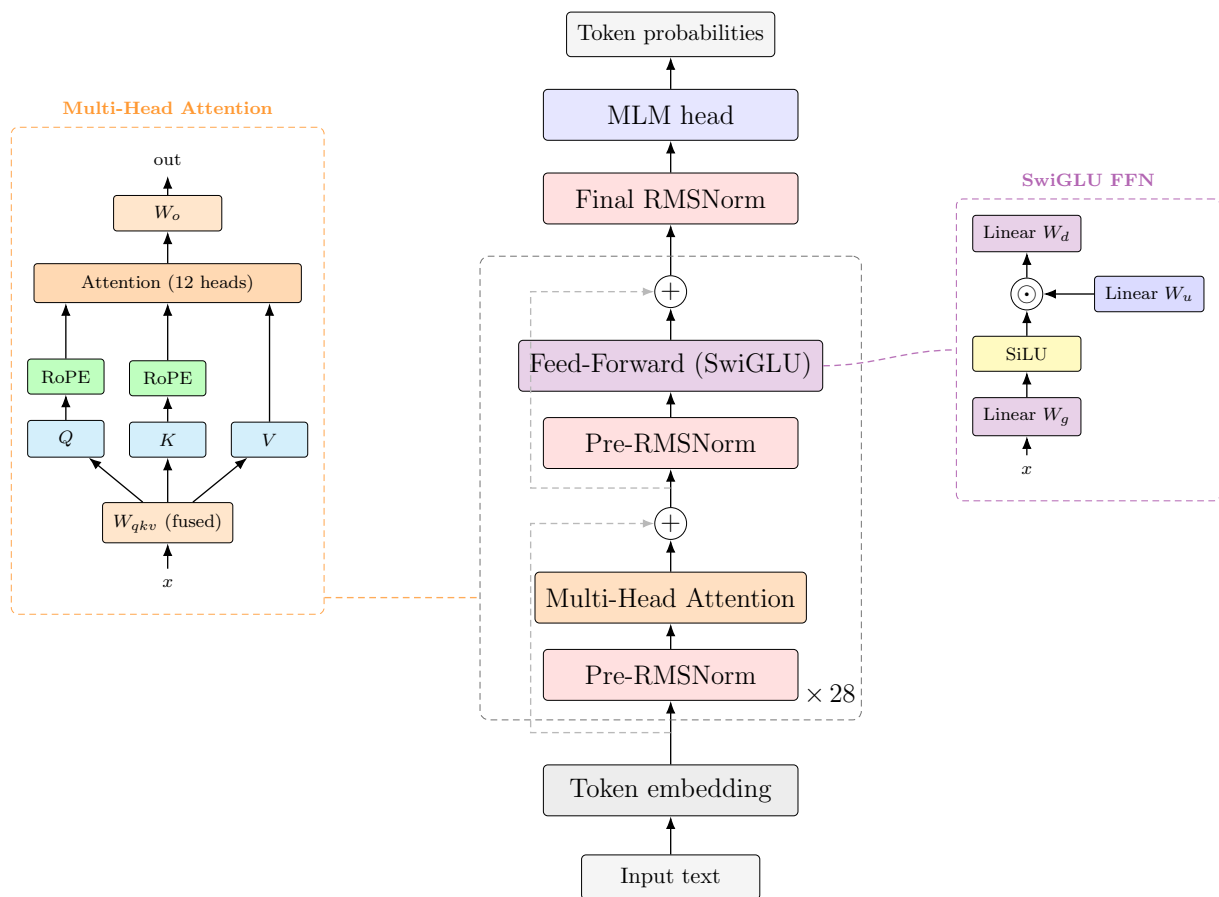
2.2 Kiến trúc NeoBERT

2.2.1 Tổng quan

Về bản chất, NeoBERT [5] vẫn là một mô hình thuộc dòng BERT: mã hóa hai chiều toàn chuỗi và được tiền huấn luyện bằng tác vụ MLM. Điều làm nó khác biệt là toàn bộ “phần cứng” bên trong đã được thay mới theo những gì thể hệ mô hình ngôn ngữ lớn gần đây chứng minh là hiệu quả. Mô hình có khoảng 245 triệu tham số, giữ kích thước ẩn 768 như BERT-base nhưng sâu 28 lớp, dùng 12 đầu chú ý (mỗi đầu 64 chiều), hỗ trợ ngữ cảnh 4.096 token, và được tiền huấn luyện trên khoảng 2,1 nghìn tỷ token văn bản web tiếng Anh (RefinedWeb); mục tiêu tiền huấn luyện là MLM thuần túy (bỏ NSP theo RoBERTa), tỷ lệ che 20%, và luôn dùng token [MASK] để che (bỏ quy tắc 80/10/10 của BERT).

Kiến trúc của mô hình được minh họa trong Hình 2.2. So với BERT, có bốn thay đổi cốt lõi, cũng chính là bốn nội dung sẽ được phân tích trong các mục con tiếp theo: (1) tỷ lệ chiều sâu trên chiều rộng được chọn lại theo phân tích lý thuyết – sâu hơn nhưng không rộng hơn; (2) nhúng vị trí tuyệt đối bị loại bỏ hoàn toàn, thay bằng phép quay RoPE áp dụng trên truy vấn và khóa bên trong từng khối; (3) mạng truyền thẳng dùng cơ chế cổng SwiGLU thay cho GELU; và (4) chuẩn hóa RMSNorm đặt *trước* mỗi tầng con thay vì LayerNorm đặt sau. Ngoài ra, đầu MLM của NeoBERT là một lớp tuyến tính *tách rời* khỏi ma trận nhúng đầu vào – chi tiết tưởng nhỏ nhưng ảnh hưởng trực tiếp tới thiết kế phương pháp ở Chương 3.

Điểm cần chú ý khi đọc Hình 2.2: khác với sơ đồ BERT quen thuộc,



Hình 2.2: Kiến trúc NeoBERT [5].

không có khối nhúng vị trí cộng vào sau nhúng token – thông tin vị trí được đưa vào bên trong từng khối qua RoPE; chuẩn hóa nằm *trước* tầng tự chú ý và tầng truyền thẳng, còn hai kết nối tắt cộng thẳng vào luồng chính; và sau khối cuối cùng có một lớp RMSNorm khép lại trước khi tín hiệu đi vào các đầu tác vụ. Trong phạm vi đề tài, NeoBERT được xem là *kiến trúc đích*: phần thân Transformer đã được tiền huấn luyện mạnh trên tiếng Anh và sẽ được giữ nguyên, còn lớp nhúng và đầu MLM gắn với từ vựng tiếng Anh là phần phải thay thế. Mỗi cải tiến dưới đây đều để lại một ràng buộc hoặc một thuận lợi cụ thể cho việc thay thế đó.

2.2.2 Tỷ lệ chiều sâu trên chiều rộng tối ưu

Với một ngân sách tham số cố định, câu hỏi đầu tiên của thiết kế kiến trúc là phân bổ tham số vào *chiều sâu* (số lớp L) hay *chiều rộng* (kích thước ẩn d). Levine và cộng sự [12] phân tích lý thuyết khả năng biểu diễn của self-attention và chỉ ra rằng hai đại lượng này phải tăng *đồng bộ*: một mạng quá nông–rộng lãng phí tham số vì các phụ thuộc bậc cao giữa token không thể được hợp thành qua ít bước chú ý, trong khi một mạng quá sâu–hẹp lại bị chiều rộng chặn khả năng lưu trữ thông tin ở mỗi tầng. Từ đó tồn tại một *vùng sâu–rộng tối ưu* cho từng cỡ mô hình.

Theo phân tích này, các cấu hình BERT truyền thống ($12 \text{ lớp} \times 768$ hoặc $24 \text{ lớp} \times 1.024$) nằm lệch về phía *quá rộng so với độ sâu*. NeoBERT do đó giữ kích thước ẩn 768 như BERT-base nhưng tăng số lớp lên 28 – trở thành một mô hình “sâu mà hẹp” so với BERT-large dù có ít tham số hơn. Đối với đề tài, thiết kế này mang hai ý nghĩa: (1) phần thân đủ sâu để học các biến đổi phi tuyến mạnh – chính là tài sản mà quá trình chuyển giao muốn giữ nguyên; và (2) chiều ẩn 768 trùng với chiều nhúng của mô hình hiến tặng ViDeBERTa-base, giúp phép chiếu không gian nhúng ở Chương 3 không phải xử lý thêm bước đổi chiều.

2.2.3 Mã hóa vị trí quay (Rotary Position Embeddings)

BERT mã hóa vị trí bằng một bảng nhúng vị trí *tuyệt đối* học được, cộng trực tiếp vào nhúng token trước khối Transformer đầu tiên: $h_i = e_i + p_i$. Cách tiếp cận này mang hai hạn chế căn bản: (1) bảng p bị giới hạn cứng bởi độ dài huấn luyện (512 với BERT), không thể mở rộng sang chuỗi dài hơn mà không huấn luyện lại; (2) điểm chú ý giữa hai token phụ thuộc vào vị trí tuyệt đối thay vì khoảng cách tương đối – vốn mới là đại lượng mang ý nghĩa cú pháp, vì “token A đứng trước token B hai bước” quan trọng hơn “token A ở vị trí 37”. Ngoài ra, thông tin vị trí phải lan truyền qua từng lớp chuẩn hóa và phi tuyến, dễ bị suy giảm ở mạng sâu.

Mục tiêu hình thức của mã hóa vị trí tương đối là tìm hàm biến đổi f_q, f_k sao cho tích vô hướng giữa truy vấn tại vị trí m và khóa tại vị trí n chỉ phụ thuộc vào nội dung token và khoảng cách tương đối:

$$\langle f_q(x_m, m), f_k(x_n, n) \rangle = g(x_m, x_n, m - n). \quad (2.4)$$

Dẫn xuất từ trường hợp 2D. Su và cộng sự [13] bắt đầu với trường hợp $d = 2$, khai thác hình học mặt phẳng phức. Véc-tơ $q = W_q x_m \in \mathbb{R}^2$ được xem như số phức $q = \|q\|e^{i\theta_q}$, và tương tự cho k . Điều kiện ban đầu yêu cầu $f_q(x_m, 0) = W_q x_m$ và $f_k(x_n, 0) = W_k x_n$ (véc-tơ không mang thông tin vị trí khi ở vị trí 0). Từ đó, Su và cộng sự chỉ ra rằng một nghiệm thỏa mãn Công thức (2.4) là:

$$f_q(x_m, m) = (W_q x_m) e^{im\theta}, \quad f_k(x_n, n) = (W_k x_n) e^{in\theta}, \quad (2.5)$$

với $\theta \in \mathbb{R}$ là hằng số tần số quay. Ý nghĩa hình học rất trực quan: mỗi token được *quay* thêm một góc tỷ lệ với vị trí của nó trong mặt phẳng phức. Tích vô hướng trở thành:

$$\langle f_q(x_m, m), f_k(x_n, n) \rangle = \text{Re} \left[(W_q x_m)(W_k x_n)^* e^{i(m-n)\theta} \right],$$

chỉ phụ thuộc vào $m - n$ – thỏa mãn ràng buộc trong Công thức (2.4). Đây là trực giác cốt lõi của RoPE: *mã hóa vị trí bằng phép quay, không phải phép cộng*.

Dạng tổng quát cho véc-tơ d_h chiều. Để mở rộng sang $x \in \mathbb{R}^{d_h}$ (d_h chẵn), không gian được chia thành $d_h/2$ mặt phẳng con hai chiều độc lập, mỗi mặt phẳng quay với tần số riêng θ_i . Ma trận quay khối-chéo $R_{\Theta,m}$ kết hợp tất cả mặt phẳng con:

$$R_{\Theta,m} = \text{diag}(R_{m,1}, \dots, R_{m,d_h/2}),$$

$$R_{m,i} = \begin{pmatrix} \cos m\theta_i & -\sin m\theta_i \\ \sin m\theta_i & \cos m\theta_i \end{pmatrix}, \quad \theta_i = 10000^{-2(i-1)/d_h}. \quad (2.6)$$

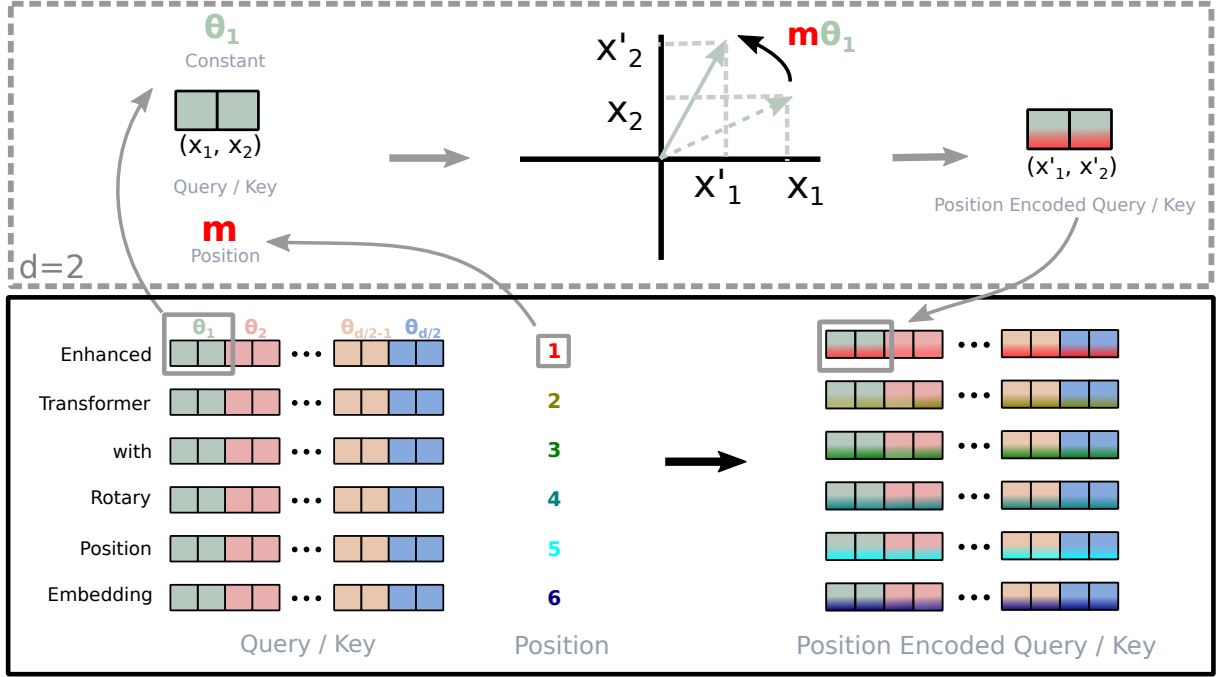
Trong đó:

- $\theta_i = 10000^{-2(i-1)/d_h}$ tạo dải tần số hình học: các cặp chiều thấp (nhỏ i) có θ_i lớn, *quay nhanh*, nắm bắt quan hệ cục bộ giữa các token gần nhau; các cặp chiều cao (lớn i) có θ_i nhỏ, *quay chậm*, mã hóa quan hệ toàn cục. Thiết kế này phản chiếu mã hóa sinusoidal của Transformer gốc nhưng đưa thông tin vị trí vào qua phép quay thay vì phép cộng;
- $R_{\Theta,m}$ là ma trận trực giao ($R_{\Theta,m}^\top R_{\Theta,m} = I$), bảo toàn hoàn toàn độ lớn của véc-tơ – không có bước nào thêm hay bớt năng lượng tín hiệu;
- $d_h = 64$ là số chiều mỗi đầu chú ý trong NeoBERT.

Hình 2.3 minh họa cơ chế quay. Phép biến đổi và tính chất tương đối được viết gọn như Công thức (2.7):

$$q_m = R_{\Theta,m} W_q x_m, \quad k_n = R_{\Theta,n} W_k x_n, \quad q_m^\top k_n = x_m^\top W_q^\top R_{\Theta,n-m} W_k x_n, \quad (2.7)$$

trong đó $R_{\Theta,n-m} = R_{\Theta,m}^\top R_{\Theta,n}$. Tích của hai ma trận trực giao vẫn là ma



Hình 2.3: Minh họa cơ chế RoPE (Nguồn: Su và cộng sự [13]).

trận trực giao, và tích vô hướng $q_m^\top k_n$ chỉ phụ thuộc vào $n - m$ – xác nhận RoPE thỏa mãn ràng buộc trong Công thức (2.4).

Triển khai hiệu quả. Do $R_{\Theta, m}$ thưa (chỉ $2d_h$ phần tử khác 0 trên ma trận $d_h \times d_h$), phép nhân ma trận đầy đủ tốn $O(d_h^2)$ là lãng phí. Su và cộng sự [13] cung cấp cách tính tương đương bằng phép nhân theo từng phần tử (Hadamard), hoàn toàn tránh phép nhân ma trận:

$$R_{\Theta, m} x = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ \vdots \\ x_{d_h-1} \\ x_{d_h} \end{pmatrix} \odot \begin{pmatrix} \cos m\theta_1 \\ \cos m\theta_1 \\ \cos m\theta_2 \\ \cos m\theta_2 \\ \vdots \\ \cos m\theta_{d_h/2} \\ \cos m\theta_{d_h/2} \end{pmatrix} + \begin{pmatrix} -x_2 \\ x_1 \\ -x_4 \\ x_3 \\ \vdots \\ -x_{d_h} \\ x_{d_h-1} \end{pmatrix} \odot \begin{pmatrix} \sin m\theta_1 \\ \sin m\theta_1 \\ \sin m\theta_2 \\ \sin m\theta_2 \\ \vdots \\ \sin m\theta_{d_h/2} \\ \sin m\theta_{d_h/2} \end{pmatrix}. \quad (2.8)$$

Mỗi chiều chỉ tham gia đúng hai phép nhân và một phép cộng – tổng chi

phí $O(d_h)$, dễ vectorize hoàn toàn trên GPU mà không cần xây dựng ma trận quay tường minh.

Tính chất suy giảm đường dài (long-term decay). Một tính chất quan trọng của RoPE là điểm chú ý *suy giảm tự nhiên* khi khoảng cách $|n - m|$ tăng. Viết tích vô hướng dưới dạng tổng số phức:

$$q_m^\top k_n = \text{Re} \left[\sum_{i=1}^{d_h/2} q_{[2i-1:2i]} k_{[2i-1:2i]}^* e^{i(m-n)\theta_i} \right], \quad (2.9)$$

trong đó $q_{[2i-1:2i]}$ là cặp chiều thứ i của q biểu diễn như số phức. Áp dụng biến đổi Abel (tổng từng phần), biểu thức này bị chặn trên bởi một đại lượng tỉ lệ với $\frac{1}{d_h/2} \sum_j |S_j|$, với $S_j = \sum_{i=1}^j e^{i(m-n)\theta_i}$. Khi $|n - m|$ lớn, các số hạng $e^{i(m-n)\theta_i}$ trải đều trên vòng tròn đơn vị và triệt tiêu lẫn nhau, khiến $|S_j|$ – và theo đó điểm chú ý – giảm dần theo khoảng cách. Tính chất này hoàn toàn tự nhiên từ bản chất quay và phù hợp với trực giác ngôn ngữ học rằng hai token xa nhau nên ít liên quan hơn, mà không cần thêm bất kỳ ràng buộc kỹ thuật nào.

Mở rộng ngữ cảnh. Vì không có bảng vị trí cứng, NeoBERT nâng ngữ cảnh từ 1.024 lên 4.096 token chỉ bằng một pha huấn luyện bổ sung [5]. Xa hơn nữa, kỹ thuật YaRN [14] nội suy tần số θ_i để kéo dài ngữ cảnh mà không cần huấn luyện lại toàn bộ mô hình – một lợi thế trực tiếp từ kiến trúc không dùng bảng vị trí cứng.

Tổng kết lại, RoPE mang bốn thuận lợi đồng thời: (1) *tính tương đối* của điểm chú ý (Công thức (2.7)); (2) *không thêm tham số* – không có bảng nhúng vị trí; (3) *bảo toàn chuẩn* – phép quay trực giao không thay đổi độ lớn véc-tơ, nhất quán với triết lý chuẩn hóa theo độ lớn của RMSNorm; (4) thông tin vị trí *không nằm trong ma trận nhúng*, nên việc thay thế toàn bộ lớp nhúng tiếng Việt không đụng chạm tới cơ chế vị trí của mô hình.

2.2.4 Hàm kích hoạt SwiGLU

Trong BERT, mạng truyền thẳng của mỗi khối là hai phép chiếu với hàm kích hoạt GELU [15] ở giữa, như Công thức (2.10):

$$\text{FFN}_{\text{GELU}}(x) = \text{GELU}(xW_1 + b_1) W_2 + b_2 \quad (2.10)$$

Trong đó $W_1 \in \mathbb{R}^{d \times 4d}$ và $W_2 \in \mathbb{R}^{4d \times d}$ lần lượt mở rộng rồi thu hẹp chiều biểu diễn. Mọi chiều của biểu diễn trung gian đều đi qua cùng một phi tuyến – không có cơ chế nào cho phép mô hình *chọn lọc* thông tin truyền qua.

NeoBERT thay bằng SwiGLU [16], một biến thể thuộc họ *đơn vị tuyến tính có cổng* (Gated Linear Unit – GLU), như Công thức (2.11):

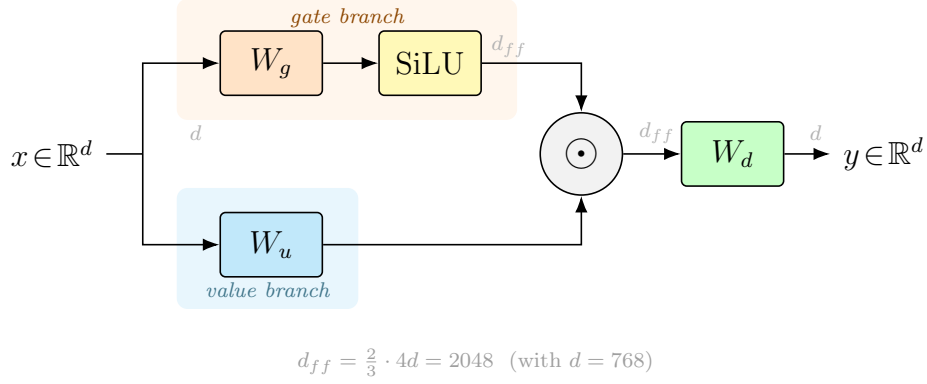
$$\text{FFN}_{\text{SwiGLU}}(x) = (\text{SiLU}(xW_g) \odot (xW_u)) W_d, \quad \text{SiLU}(z) = z \cdot \sigma(z) \quad (2.11)$$

Trong đó:

- $W_g, W_u \in \mathbb{R}^{d \times d_{ff}}$ là hai phép chiếu song song: nhánh *cổng* (qua phi tuyến SiLU) và nhánh *giá trị* (tuyến tính);
- $\odot$ là phép nhân theo từng phần tử: nhánh cổng quyết định *mỗi chiều* của nhánh giá trị được truyền qua bao nhiêu;
- $W_d \in \mathbb{R}^{d_{ff} \times d}$ chiếu về chiều gốc; cả ba ma trận đều không dùng bias;
- σ là hàm sigmoid.

Hình 2.4 thể hiện luồng tính toán của FFN SwiGLU. Vì SwiGLU dùng *ba* ma trận thay vì hai, chiều trung gian được thu lại theo quy tắc $d_{ff} = \frac{2}{3} \cdot 4d$ để giữ nguyên ngân sách tham số [16]. Với NeoBERT: $d = 768$, chiều trung gian danh nghĩa $4d = 3.072$, sau điều chỉnh $\frac{2}{3}$ còn $d_{ff} = 2.048$. Thực nghiệm của Shazeer cho thấy các biến thể GLU cải thiện chất lượng tiền huấn luyện và tinh chỉnh so với GELU/ReLU ở cùng chi phí; cơ chế cổng

đặc biệt hữu ích khi kết hợp với thiết kế sâu, nơi khả năng điều tiết luồng thông tin qua từng tầng trở nên quan trọng.



Hình 2.4: Cấu trúc cổng SwiGLU [16].

2.2.5 Chuẩn hóa Pre-RMSNorm

Cải tiến thứ tư gồm hai quyết định độc lập nhưng bổ trợ nhau: *loại chuẩn hóa* (RMSNorm thay cho LayerNorm) và *vị trí chuẩn hóa* (trước thay vì sau tầng con).

RMSNorm. LayerNorm truyền thống chuẩn hóa véc-tơ đầu vào $a \in \mathbb{R}^d$ bằng cách trừ trung bình rồi chia độ lệch chuẩn, như Công thức (2.12):

$$\bar{a}_i = \frac{a_i - \mu}{\sigma} g_i + \beta_i, \quad \mu = \frac{1}{d} \sum_{i=1}^d a_i, \quad \sigma = \sqrt{\frac{1}{d} \sum_{i=1}^d (a_i - \mu)^2 + \epsilon} \quad (2.12)$$

Trong đó $g, \beta \in \mathbb{R}^d$ lần lượt là véc-tơ khuếch đại và dịch học được. Quá trình này yêu cầu *hai lượt duyệt* qua d phần tử: lượt đầu tính μ , lượt sau tính σ (vì σ phụ thuộc μ đã biết). Ngoài chi phí tính toán, LayerNorm còn mang d tham số bổ sung β chỉ để tái định tâm đầu ra.

Zhang và Sennrich [17] lập luận rằng đặc tính *tái định tâm* (phép trừ μ) không phải nguồn gốc thực sự của hiệu quả LayerNorm – mà chính là đặc tính *tái tỷ lệ* (chia theo độ lớn). Từ đó họ đề xuất RMSNorm, chỉ chuẩn hóa theo căn trung bình bình phương (Root Mean Square), như

Công thức (2.13):

$$\bar{a}_i = \frac{a_i}{\text{RMS}(a)} g_i, \quad \text{RMS}(a) = \sqrt{\frac{1}{d} \sum_{i=1}^d a_i^2 + \epsilon} \quad (2.13)$$

Trong đó $g \in \mathbb{R}^d$ là véc-tơ khuếch đại học được. RMSNorm bỏ hoàn toàn μ và β : chỉ cần *một lượt* duyệt qua d phần tử để tính tổng bình phương, không có phép trừ trung bình, và giảm đi d tham số. Thực nghiệm cho thấy RMSNorm đạt chất lượng tương đương hoặc tốt hơn LayerNorm ở cùng chi phí – do đó trở thành lựa chọn mặc định của các kiến trúc hiện đại như LLaMA, Gemma và NeoBERT.

Pre-norm. Về vị trí, BERT dùng Post-LN (chuẩn hóa *sau* khi cộng kết nối tắt), còn NeoBERT dùng Pre-norm (chuẩn hóa *trước* mỗi tầng con), như Công thức (2.14):

$$\underbrace{x_{l+1} = \text{LN}(x_l + F_l(x_l))}_{\text{Post-LN (BERT)}} \quad \text{so với} \quad \underbrace{x_{l+1} = x_l + F_l(\text{LN}(x_l))}_{\text{Pre-norm (NeoBERT)}} \quad (2.14)$$

Trong đó F_l là tầng con thứ l (chú ý hoặc FFN). Xiong và cộng sự [18] chứng minh rằng với Post-LN, độ lớn gradient tại thời điểm khởi tạo tăng theo độ sâu – buộc phải khởi động tốc độ học rất thận trọng; với Pre-norm, đường kết nối tắt là một “xa lộ” đồng nhất không bị chuẩn hóa cắt ngang, gradient chảy ổn định qua mọi tầng. Với 28 lớp – sâu hơn hẳn thể hệ BERT – lựa chọn Pre-RMSNorm là điều kiện để NeoBERT huấn luyện ổn định.

Đối với đề tài, đặc điểm này để lại một ràng buộc trực tiếp: lớp nhúng tiếng Việt khởi tạo mới sẽ đi qua RMSNorm ngay tầng đầu tiên, mà RMSNorm chuẩn hóa *theo độ lớn* của véc-tơ. Nếu phân phối độ lớn của các hàng nhúng lệch xa so với NeoBERT gốc, tín hiệu đưa vào phần thân sẽ bị sai lệch có hệ thống. Vì vậy, Chương 3 có bước hiệu chỉnh chuẩn cho lớp nhúng sau khi chiếu.

2.2.6 Dữ liệu và chiến lược tiền huấn luyện

Bảng 2.1 tóm tắt toàn bộ cấu hình của NeoBERT. Đây là những chi tiết quan trọng vì ViNeoBERT kế thừa trực tiếp kiến trúc này, chỉ thay thế bộ từ vựng và lớp nhúng tiếng Anh bằng tiếng Việt.

Bảng 2.1: Cấu hình NeoBERT [5].

Thông số	Giá trị
Số lớp Transformer	28
Kích thước ẩn d	768
Số đầu chú ý	12 (mỗi đầu $d_h = 64$)
Chiều FFN trung gian d_{ff}	2.048 (SwiGLU, sau điều chỉnh 2/3)
Tổng tham số	≈ 245 triệu (tính đủ lớp nhúng + thân + đầu giải mã)
Mã hóa vị trí	RoPE, $\theta_{base} = 10.000$
Hàm kích hoạt FFN	SwiGLU (không bias)
Chuẩn hóa	Pre-RMSNorm
Từ vựng tiếng Anh	30.522 token (WordPiece, tương thích BERT)
Ngữ cảnh tối đa	4.096 token
Dữ liệu tiền huấn luyện	RefinedWeb (600 tỷ token; duyệt $\approx 2,1$ nghìn tỷ)
Mục tiêu huấn luyện	MLM, không có NSP; tỷ lệ che 20%; luôn dùng [MASK]
Bộ tối ưu	AdamW, $\beta = (0,9; 0,95)$, weight decay 0,1
Gradient clipping	norm tối đa 1,0
Lịch tốc độ học	Cosine decay (peak 6×10^{-4} , khởi động 2.000 bước, làm nguội về 10% trong 90% số bước)
Batch size hiệu dụng	~ 2 triệu token/bước
Số bước huấn luyện	≈ 1 triệu bước (pha chính)
Tối ưu bộ nhớ	FlashAttention-2
Pha mở rộng ngữ cảnh	~ 100 tỷ token ở 4.096 token (50 nghìn bước)

NeoBERT được tiền huấn luyện trên RefinedWeb [19] – kho ngữ liệu web quy mô lớn xây dựng từ CommonCrawl (khoảng 600 tỷ token); qua nhiều lượt lặp, mô hình duyệt tổng cộng $\approx 2,1$ nghìn tỷ token trong tiền huấn luyện. Về mục tiêu huấn luyện, NeoBERT chỉ dùng MLM (bỏ NSP theo RoBERTa [2]), tăng tỷ lệ che lên 20% [20] – cao hơn 15% của BERT, được chứng minh tối ưu hơn cho các Encoder hiện đại – và đơn giản hóa lược đồ che bằng cách *luôn* thay token được chọn bằng [MASK] (bỏ quy tắc 80/10/10 của BERT). Tối ưu bằng AdamW [21] với $\beta = (0,9; 0,95)$, weight decay 0,1, gradient clipping norm tối đa 1,0 và lịch cosine [22] (làm

nguội về 10% tốc độ học đỉnh trong 90% số bước). FlashAttention-2 [23] được dùng để giảm chi phí bộ nhớ khi huấn luyện trên chuỗi dài.

Quá trình tiền huấn luyện được chia *hai pha*: pha chính (≈ 1 triệu bước, mỗi bước ~ 2 triệu token) ở độ dài chuỗi 1.024 token, sau đó một pha ngắn (≈ 100 tỷ token, 50 nghìn bước) ở 4.096 token để kích hoạt khả năng ngữ cảnh dài. Thiết kế hai pha này là hợp lý vì huấn luyện toàn bộ ở độ dài 4.096 token sẽ tốn kém hơn nhiều: chi phí tính toán của tầng tự chú ý tăng theo bình phương độ dài chuỗi – gấp 16 lần khi chuyển từ 1.024 lên 4.096 token – trong khi phần chi phí còn lại của mô hình chỉ tăng tuyến tính theo số token (gấp 4 lần). ViNeoBERT áp dụng chiến lược hoàn toàn tương tự (nhưng ở quy mô nhỏ hơn nhiều) khi mở rộng ngữ cảnh ở Giai đoạn 3 (Chương 3 và Mục 4.5.3).

Điều cần lưu ý quan trọng nhất: NeoBERT đã học một phần thân Encoder *rất mạnh* trên tiếng Anh – sâu 28 lớp với RoPE, SwiGLU và Pre-RMSNorm – nhưng toàn bộ lớp nhúng và đầu ra gắn với từ vựng tiếng Anh. Đây chính xác là lý do đề tài tập trung vào khởi tạo lại lớp nhúng tiếng Việt bằng SALT thay vì huấn luyện lại từ đầu – phần thân đã huấn luyện trên 2,1 nghìn tỷ token tiếng Anh là tài sản quý không cần tái tạo.

2.3 Chuyển giao không gian nhúng và thích nghi ngôn ngữ

Đối với đề tài, thách thức trung tâm không chỉ là thay bộ từ vựng tiếng Anh của NeoBERT bằng bộ từ vựng tiếng Việt. Lớp nhúng mới còn phải nằm đủ gần không gian biểu diễn mà phần thân Transformer của NeoBERT đã học, để quá trình tiếp tục tiền huấn luyện không bắt đầu từ một trạng thái nhiễu loạn. Vì vậy, mục này gộp hai nền tảng có liên quan chặt chẽ: hình học của không gian biểu diễn và các phương pháp khởi tạo nhúng cho thích nghi ngôn ngữ.

2.3.1 Hình học của không gian biểu diễn

Không gian nhúng của mô hình ngôn ngữ thường có hình học phức tạp hơn một không gian Euclid đều đặn. Ba kết quả có liên quan trực tiếp tới đề tài là: biểu diễn theo ngữ cảnh có tính bất đẳng hướng [24]; các không gian đơn ngữ được huấn luyện độc lập không nhất thiết đẳng cấu [25]; và ma trận nhúng đầu vào không luôn có cùng hình học với ma trận đầu ra khi mô hình không buộc trọng số [26].

Các nhận xét này dẫn tới một lựa chọn thiết kế quan trọng: không nên dùng một ánh xạ tuyến tính toàn cục để chuyển toàn bộ từ vựng tiếng Việt sang NeoBERT. Thay vào đó, đề tài ưu tiên hướng chuyển giao cục bộ theo từng token, trong đó mỗi token mới được ánh xạ dựa trên các token neo gần nó về mặt ngữ nghĩa.

2.3.2 Thích nghi ngôn ngữ và rủi ro quên lãng

NeoBERT được tiền huấn luyện trên tiếng Anh, còn huấn luyện lại một mô hình tương đương cho tiếng Việt từ đầu là không thực tế về chi phí. Hướng khả thi hơn là tiếp tục tiền huấn luyện (Continued Pre-training – CPT) trên ngữ liệu tiếng Việt, tương tự tinh thần thích nghi miền của Gururangan và cộng sự [27].

Rủi ro của CPT là mô hình có thể phải học lại từ tín hiệu đầu vào nhiều nếu lớp nhúng mới được khởi tạo ngẫu nhiên. Các nghiên cứu về tái sử dụng mô hình xuyên ngôn ngữ [28] và phân tích CPT gần đây [29], [30] đều gợi ý rằng điểm khởi đầu của lớp nhúng ảnh hưởng lớn tới ổn định huấn luyện. Vì vậy, đề tài xem khởi tạo nhúng là bước quyết định trước khi CPT.

Một khía cạnh thực hành quan trọng khác của CPT là *lịch tốc độ học*. Lịch cosine truyền thống yêu cầu cam kết trước tổng số bước huấn luyện, kém linh hoạt khi tổng ngân sách chưa được biết chắc từ đầu. Lịch Warmup–Stable–Decay (WSD) [31] tách quá trình thành pha khởi động, pha ổn định ở tốc độ học đỉnh và một pha làm nguội ngắn ở cuối, cho phép

kéo dài hoặc dừng huấn luyện linh hoạt; Hägele và cộng sự [32] cho thấy cách tiếp cận này đạt chất lượng tương đương cosine, đồng thời xác định dạng làm nguội căn bậc hai ($1 - \sqrt{f}$) là hiệu quả nhất ở cùng ngân sách. Ngoài ra, khi huấn luyện tiếp từ một điểm kiểm tra đã làm nguội, tốc độ học cần được khởi động lại thay vì giữ ở mức gần 0 [30]. Các kết quả này là cơ sở cho lịch huấn luyện được thiết kế ở Chương 3.

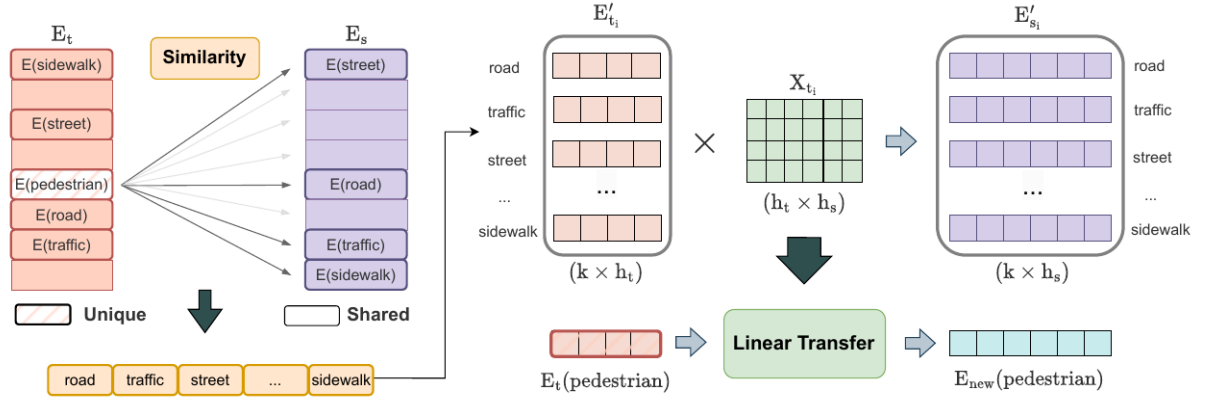
2.3.3 Khởi tạo nhúng có thông tin

Các phương pháp như WECHSEL [33], FOCUS [34] và OFA [35] đều theo cùng một hướng: token mới không được xem như ký hiệu rỗng, mà được khởi tạo bằng thông tin ngữ nghĩa từ token hoặc không gian nhúng đã biết. WECHSEL dựa trên nhúng tĩnh song ngữ, FOCUS tận dụng phần từ vựng chồng lấp và Sparsemax [36], còn OFA mở rộng sang thiết lập đa ngôn ngữ lớn hơn.

SALT kế thừa tinh thần này nhưng phù hợp hơn với đề tài vì nó tận dụng trực tiếp một mô hình đã tiền huấn luyện ở ngôn ngữ đích, thay vì chỉ dựa vào nhúng tĩnh hoặc phép trộn từ từ vựng nguồn.

2.3.4 Semantic Aware Linear Transfer (SALT)

SALT của Lee và cộng sự [8] khởi tạo từ vựng đích bằng cách chuyển biểu diễn từ một mô hình tiền huấn luyện ở ngôn ngữ đích sang không gian của mô hình nguồn. Thay vì học một ánh xạ chung cho toàn bộ từ vựng, SALT dùng các token neo trong phần từ vựng chồng lấp để ước lượng ánh xạ cục bộ cho từng token mới. Hình 2.5 minh họa luồng xử lý này: với mỗi token không trùng (ví dụ *pedestrian*), phương pháp tìm các neo gần nghĩa nhất trong phần chồng lấp, xếp các cặp nhúng tương ứng của chúng ở hai không gian thành hai ma trận, học một ánh xạ tuyến tính cục bộ X giữa hai ma trận đó, rồi áp dụng X lên nhúng gốc của token để thu về nhúng trong không gian mô hình nguồn.



Hình 2.5: Sơ đồ tổng quan SALT (Nguồn: Lee và cộng sự [8]).

Trong nghiên cứu gốc, SALT chủ yếu được kiểm chứng trên các mô hình dạng decoder như Gemma và XGLM. Đề tài kế thừa ý tưởng này nhưng áp dụng vào bối cảnh khác: mô hình nền là NeoBERT Encoder-only, ngôn ngữ đích là tiếng Việt, và lớp đầu ra của NeoBERT cần được xử lý như một ma trận riêng. Các điều chỉnh cụ thể được trình bày trong Chương 3.

2.4 Nhận xét định hướng

Từ các cơ sở trên, chúng tôi xem NeoBERT là một nền tảng phù hợp để xây dựng Encoder tiếng Việt hiện đại: phần thân Transformer đã mạnh, hỗ trợ ngữ cảnh dài, nhưng lớp từ vựng và không gian đầu vào vẫn mang thiên lệch tiếng Anh. Vì vậy, hướng tiếp cận hợp lý không phải huấn luyện lại từ đầu, mà là tái sử dụng phần thân NeoBERT và khởi tạo lại lớp nhúng tiếng Việt một cách có thông tin.

Nhận xét quan trọng nhất của chúng tôi là bước khởi tạo không gian nhúng quyết định chất lượng thích nghi ban đầu. Nếu điểm khởi đầu lệch quá xa không gian NeoBERT, quá trình CPT dễ tiêu tốn tài nguyên để sửa lỗi căn chỉnh thay vì học tiếng Việt. Do đó, Chương 3 sẽ tập trung vào một biến thể SALT phù hợp với NeoBERT, bao gồm lựa chọn tập neo,

chiều cục bộ, xử lý lớp đầu ra tách rời và hiệu chỉnh chuẩn của chúng.

Chương 3

Phương pháp xây dựng Vietnamese NeoBERT

Chương này trình bày chi tiết phương pháp xây dựng mô hình Vietnamese NeoBERT (ViNeoBERT). Trước hết, chúng tôi mô tả tổng quan quy trình ba giai đoạn cùng các mô hình nguồn được sử dụng (Mục 3.1). Tiếp theo, chúng tôi giải thích lý do lựa chọn ViDeBERTa làm mô hình hiến tặng và so sánh bộ tách từ với PhoBERT (Mục 3.2), rồi trình bày bước cắt tỉa và đồng bộ từ vựng (Mục 3.3), quá trình trích xuất tập neo – được mở rộng bằng các cặp dịch, một cải tiến của đề tài so với SALT gốc (Mục 3.4), và phép chiếu SALT áp dụng cho cả lớp nhúng đầu vào lẫn lớp đầu ra tách rời của NeoBERT (Mục 3.5). Cuối cùng, chúng tôi mô tả giai đoạn căn chỉnh đóng băng (Mục 3.6) và giai đoạn tiếp tục tiền huấn luyện thích nghi (Mục 3.7). Các giá trị siêu tham số cụ thể được trình bày trong Chương 4.

3.1 Tổng quan phương pháp

3.1.1 Đặt vấn đề

Như đã phân tích trong Chương 2, việc xây dựng một Encoder hiện đại cho tiếng Việt từ NeoBERT đối mặt đồng thời ba thách thức: (1) NeoBERT

chỉ được tiền huấn luyện trên ngữ liệu tiếng Anh, nên toàn bộ lớp nhúng của nó phản ánh phân phối từ vựng Anh ngữ; (2) tiếp tục tiền huấn luyện với một lớp nhúng *khởi tạo ngẫu nhiên* để dẫn tới hiện tượng quên lãng thảm họa; và (3) huấn luyện lại từ đầu trên quy mô hàng nghìn tỷ token là bất khả thi về tài nguyên. Phương pháp của đề tài giải quyết đồng thời ba thách thức này bằng cách *khởi tạo có thông tin* lớp nhúng tiếng Việt thông qua kỹ thuật SALT, sau đó tinh chỉnh mô hình trên ngữ liệu tiếng Việt với chi phí khiêm tốn.

3.1.2 Mô hình nguồn và mô hình hiến tặng

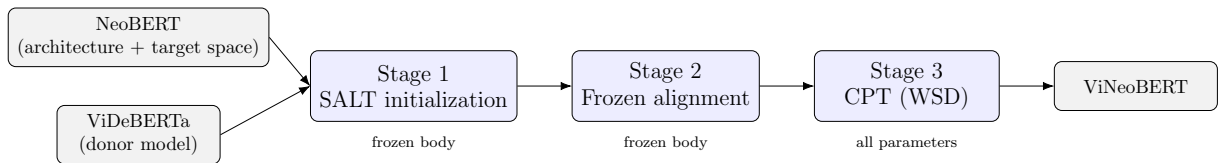
Phương pháp sử dụng hai mô hình tiền huấn luyện:

- **NeoBERT** [5] đóng vai trò *kiến trúc đích*: toàn bộ phần thân Transformer (28 lớp với RoPE, SwiGLU, Pre-RMSNorm) cùng không gian biểu diễn tiếng Anh được giữ nguyên và dùng làm hệ tọa độ đích cho phép chiếu. Một đặc điểm quan trọng là NeoBERT sử dụng *đầu giải mã tách rời* (untied head): lớp đầu ra của tác vụ MLM là một ánh xạ tuyến tính riêng $y = W_{\text{dec}} h + b_{\text{dec}}$ với ma trận trọng số W_{dec} và véc-tơ độ chệch b_{dec} *độc lập* với ma trận nhúng đầu vào E . Do đó, việc thích nghi sang tiếng Việt đòi hỏi khởi tạo *cả hai* ma trận token: ma trận nhúng đầu vào E và ma trận đầu ra W_{dec} .
- **ViDeBERTa** [7] (bản **base**) đóng vai trò *mô hình hiến tặng* (donor): cung cấp không gian nhúng tiếng Việt chất lượng cao làm nguồn để tái sử dụng. ViDeBERTa dùng bộ tách từ SentencePiece theo thuật toán Unigram với kích thước từ vựng lớn (khoảng 128 nghìn token), trong đó mỗi đơn vị từ vựng đi kèm một điểm log-xác suất.

3.1.3 Quy trình ba giai đoạn

Mô hình ViNeoBERT được xây dựng qua ba giai đoạn nối tiếp, minh họa trong Hình 3.1:

1. **Khởi tạo SALT**: xây dựng tập từ vựng tiếng Việt, trích xuất tập neo, rồi chiếu không gian nhúng của ViDeBERTa sang không gian NeoBERT để khởi tạo cả ma trận nhúng đầu vào lẫn ma trận đầu ra; độ chệch của đầu ra được khởi tạo theo tần suất từ đơn (Mục 3.3–3.5). Giai đoạn này *không* cập nhật phần thân Transformer.
2. **Căn chỉnh đóng băng**: đóng băng toàn bộ phần thân, chỉ huấn luyện ma trận nhúng và ma trận đầu ra trên một lượng nhỏ ngữ liệu tiếng Việt, nhằm để hai ma trận vừa khởi tạo căn chỉnh với nhau và với phần thân (Mục 3.6).
3. **Tiếp tục tiền huấn luyện thích nghi**: tinh chỉnh toàn bộ mô hình trên ngữ liệu tiếng Việt quy mô lớn với mục tiêu MLM theo lịch tốc độ học WSD (Mục 3.7).



Hình 3.1: Quy trình ba giai đoạn xây dựng ViNeoBERT.

3.2 Lựa chọn mô hình hiến tặng và bộ tách từ

3.2.1 Vai trò của bộ tách từ trong mô hình ngôn ngữ

Bộ tách từ (tokenizer) là thành phần đầu vào không thể tách rời của mô hình ngôn ngữ: nó quyết định cách văn bản thô được phân đoạn thành các đơn vị (token) trước khi đưa vào mô hình. Chất lượng của bộ tách từ ảnh hưởng trực tiếp tới (1) độ hiệu quả biểu diễn – một bộ tách từ tốt cho tiếng Việt sẽ tạo ra các token mang nghĩa rõ ràng, còn bộ tách từ kém có

thể cắt nhằm ranh giới hình vị; (2) độ bao phủ từ vựng – từ vựng không đủ lớn buộc mô hình phải dùng nhiều token để biểu diễn một khái niệm đơn giản; và (3) tính tương thích khi chuyển giao – bộ tách từ của mô hình hiến tặng phải đủ phong phú để cung cấp neo ngữ nghĩa chất lượng cao.

Trong lĩnh vực xử lý ngôn ngữ tự nhiên, hai thuật toán tách từ phổ biến nhất cho tiếng Việt là BPE (Byte-Pair Encoding) và Unigram SentencePiece. BPE được sử dụng bởi PhoBERT: bắt đầu từ ký tự đơn, lặp lại gộp hai ký hiệu liền kề phổ biến nhất cho đến khi đạt kích thước từ vựng mong muốn. Unigram SentencePiece (được ViDeBERTa sử dụng) tiếp cận ngược: bắt đầu với một từ vựng lớn, loại dần các đơn vị ít thông tin nhất theo tiêu chí xác suất Unigram; kết quả là một phân phối xác suất trên các cách phân đoạn, mỗi token được gán một điểm log-xác suất.

3.2.2 So sánh PhoBERT và ViDeBERTa làm mô hình hiến tặng

Đáng chú ý, nghiên cứu gốc SALT [8] sử dụng chính PhoBERT làm mô hình hiến tặng tiếng Việt. Trong đề tài này, chúng tôi chủ đích chọn ViDeBERTa thay cho PhoBERT vì ba lý do chính:

1. Bộ tách từ tự chứa, không ràng buộc công cụ tách từ ngoài.

PhoBERT dùng thuật toán BPE huấn luyện trên văn bản đã được *tách từ sẵn* bằng VnCoreNLP [37] (“học_sinh đi học_kỳ này”); do đó mọi văn bản đưa vào PhoBERT đều phải qua bước tách từ ngoài này, và một mô hình kế thừa từ vựng PhoBERT sẽ mang theo ràng buộc đó cùng nguy cơ lệch phân phối khi công cụ tách từ hoạt động sai. Ngược lại, ViDeBERTa dùng bộ tách từ SentencePiece Unigram thao tác ở cấp *tiểu từ* (sub-word) trực tiếp trên chuỗi Unicode; ViNeoBERT kế thừa bộ tách từ này nên xử lý được văn bản thô của CulturaX – kho dữ liệu dùng trong CPT của đề tài – mà không phụ thuộc vào một công cụ tách từ riêng.

2. Từ vựng lớn hơn – neo phong phú hơn. ViDeBERTa có từ vựng ≈ 128.000 token (so với 64.000 của PhoBERT), trong đó mỗi đơn vị

kèm điểm log-xác suất Unigram. Từ vựng lớn hơn đồng nghĩa với nhiều ứng viên neo hơn trong bước SALT – đặc biệt quan trọng cho việc bao phủ từ vựng thuần Việt và từ ghép. Bảng 3.1 tóm tắt sự khác biệt chính.

3. Không gian nhúng phong phú về ngữ nghĩa tiếng Việt. ViDeBERTa được tiền huấn luyện theo kiến trúc DeBERTaV3 [38] trên kho ngữ liệu web CC100 tiếng Việt quy mô lớn (khoảng 138GB văn bản) [7] – lớn hơn nhiều lần so với 20GB (Wikipedia + tin tức) của PhoBERT. Không gian nhúng học từ dữ liệu web quy mô lớn giúp phép chiếu SALT tạo ra các véc-tơ ban đầu tốt hơn, vì neo lấy từ ViDeBERTa sẽ mã hóa ngữ nghĩa tiếng Việt chính xác hơn.

Bảng 3.1: So sánh bộ tách từ của PhoBERT và ViDeBERTa.

Đặc điểm	PhoBERT	ViDeBERTa
Thuật toán tách từ	BPE	SentencePiece Unigram
Kích thước từ vựng	64.000	≈ 128.000
Đầu vào yêu cầu	Đã tách từ (VnCoreNLP)	Văn bản thô
Điểm log-xác suất Unigram	Không có	Có (dùng để cắt tỉa)
Dữ liệu tiền huấn luyện	20GB (Wikipedia + tin tức)	$\approx 138\text{GB}$ (CC100)
Tương thích CulturaX (raw)	Không trực tiếp	Có
Tiềm năng neo SALT	Hạn chế	Phong phú

3.3 Cắt tỉa và đồng bộ từ vựng

Phép chiếu SALT yêu cầu lớp nhúng nguồn và đích có cùng kích thước từ vựng. Vì từ vựng của ViDeBERTa lớn hơn nhiều so với 30.522 token của NeoBERT, cần một bước *cắt tỉa từ vựng* (vocabulary pruning) để đồng bộ kích thước. Quá trình này giữ lại toàn bộ các token đặc biệt (như [CLS], [SEP], [MASK], [PAD]), sau đó chọn bổ sung các token thông thường có *điểm log-xác suất Unigram cao nhất* – tức các đơn vị từ vựng phổ biến và đáng tin cậy nhất – cho tới khi đạt đúng 30.522 token.

Gọi $\mathcal{V}_s$ là tập token đặc biệt và $\mathcal{V}_n$ là tập token thông thường của ViDeBERTa, với điểm Unigram score($\cdot$). Tập từ vựng tiếng Việt sau cắt

tĩa $\mathcal{V}_t$ được xác định như Công thức (3.1):

$$\mathcal{V}_t = \mathcal{V}_s \cup \text{TopK}(\mathcal{V}_n, \text{score}, K), \quad K = 30,522 - |\mathcal{V}_s| \quad (3.1)$$

Trong đó $\text{TopK}(\mathcal{V}_n, \text{score}, K)$ trả về K token có điểm Unigram cao nhất. Bộ tách từ được ghi lại với ánh xạ chỉ số mới, đồng thời lưu một ánh xạ $\pi : \text{id mới} \rightarrow \text{id gốc}$ trong ViDeBERTa để truy xuất đúng hàng nhúng hiển tĩng ở các bước sau. Phép so khớp chuỗi piece là chính xác vì các đơn vị Unigram là duy nhất trong từ vựng.

3.4 Trích xuất tập neo (anchor set)

3.4.1 Vai trò của tập neo

Tập neo (anchor set) $\mathcal{A}$ là tập các cặp token $(t_{\text{vi}}, t_{\text{neo}})$ có tương ứng ngữ nghĩa rõ ràng giữa từ vựng tiếng Việt (ViDeBERTa đã cắt tĩa) và từ vựng NeoBERT. Mỗi cặp neo cung cấp một điểm *đã biết* của ánh xạ giữa hai không gian nhúng, và giữ vai trò kép trong phép chiếu SALT:

- **Khởi tạo trực tiếp:** bản thân token neo t_{vi} được khởi tạo bằng cách *sao chép nguyên vẹn* hàng nhúng NeoBERT của t_{neo} , bảo đảm độ trung thực tuyệt đối với không gian đích cho các token đã có tương ứng chắc chắn.
- **Tập giám sát:** các cặp $(e_{\text{vi}}, e_{\text{neo}})$ của tập neo đóng vai trò tập giám sát (support set) để *học các ánh xạ tuyến tính cục bộ*, từ đó suy ra nhúng cho những token *không* thuộc tập neo (Mục 3.5).

Do đó, chất lượng và độ phủ của tập neo quyết định trực tiếp chất lượng của phép chiếu SALT.

Trong nghiên cứu gốc, SALT [8] cũng như FOCUS [34] chỉ khai thác các token *trùng lặp bề mặt* (cùng dạng chuỗi) giữa hai từ vựng làm neo. Đối với cặp ngôn ngữ Anh–Việt vốn khác biệt lớn về hình thái, số token

trùng lặp bề mặt là khá hạn chế và độ phủ ngữ nghĩa hẹp. Vì vậy, chúng tôi xây dựng tập neo từ *ba nguồn bổ sung cho nhau*: (1) neo trùng lặp bề mặt *đã xác minh nghĩa*, (2) neo số, và (3) các *cặp dịch* Anh–Việt khai thác tự động. Trong đó, việc mở rộng tập neo bằng cặp dịch là một trong những đóng góp chính của đề tài.

3.4.2 Neo trùng lặp bề mặt và neo số

Neo trùng lặp bề mặt. Theo ý tưởng gốc của SALT, các token có *cùng dạng chuỗi* ở cả hai từ vựng là ứng viên neo tự nhiên, bởi trùng dạng chuỗi thường hàm ý cùng khái niệm – điển hình là từ vay mượn, tên riêng và thuật ngữ khoa học (*internet, vitamin, radio*). Chúng tôi trích các *từ đầy đủ* (word-initial), độ dài đủ lớn, xuất hiện ở cả hai từ vựng và lấy giao làm tập ứng viên.

Tuy nhiên, vì tiếng Việt cũng dùng hệ chữ Latin như tiếng Anh, *cùng dạng chuỗi chưa chắc cùng nghĩa*: nhiều token trùng chuỗi thực chất là hiện tượng *đồng tự khác nghĩa* (false friends). Chẳng hạn “song” trong tiếng Việt nghĩa là *song song* hoặc liên từ *nhưng*, còn “song” trong tiếng Anh nghĩa là *bài hát*; “sang” tiếng Việt nghĩa là *sang trọng*, còn trong tiếng Anh là dạng quá khứ của *sing*. Một cặp neo đồng tự khác nghĩa sẽ tạo ra một điểm ánh xạ sai lệch, kéo phép chiếu đi chệch. Để loại các neo giả này, chúng tôi *xác minh nghĩa* từng ứng viên bằng dịch máy MarianMT Việt–Anh [39]: chỉ giữ lại làm neo khi bản dịch tiếng Anh *trùng khớp chính chuỗi gốc*. Một khái niệm chung thực sự (vay mượn, tên riêng) sẽ dịch về chính nó và được giữ; ngược lại, một token đồng tự khác nghĩa sẽ dịch sang một từ tiếng Anh khác và bị loại bỏ.

Neo số. Song song, chúng tôi giữ trực tiếp các token *số* xuất hiện ở cả hai từ vựng làm neo mà *không* cần xác minh nghĩa, vì ký hiệu số mang tính phổ quát (universal) giữa các ngôn ngữ – “2024” biểu thị cùng một giá trị bất kể ngôn ngữ. Đây là nguồn neo tin cậy tuyệt đối và không phụ thuộc chất lượng dịch máy.

3.4.3 Mở rộng tập neo bằng cặp dịch ba tầng

Hai nguồn trên cho số neo hạn chế và tập trung ở từ vay mượn, tên riêng và số – không bao phủ được phần lớn kho từ vựng thuần Việt. Để *làm giàu và mở rộng độ phủ* của tập neo, chúng tôi bổ sung nguồn neo thứ ba: các cặp dịch Anh–Việt khai thác tự động, trong đó t_{vi} và t_{neo} khác *dạng chuỗi* nhưng tương ứng về nghĩa (ví dụ “nước” $\leftrightarrow$ “water”). Đây là đóng góp chính của đề tài ở bước khởi tạo.

Trước khi khai thác, chúng tôi lọc để chỉ giữ các *từ tiếng Việt đầy đủ và hợp lệ*: loại token chứa ký tự không bản địa (f, j, w, z), giới hạn độ dài, yêu cầu đúng một cụm nguyên âm liền mạch, và tuân thủ các quy tắc chính tả tiếng Việt (chẳng hạn k, gh, ngh chỉ đứng trước $i, e, ê, y$; q luôn đi kèm u). Bộ lọc chính tả này bảo đảm ứng viên là từ tiếng Việt thực thụ, tránh nhiều từ các chuỗi ngoại lai. Trên tập đã lọc, cặp dịch được khai thác theo *ba tầng* bổ sung cho nhau:

- **Tầng 1 – dịch ngược (back-translation)**: mỗi từ tiếng Việt được dịch sang tiếng Anh rồi *dịch ngược* trở lại tiếng Việt; cặp neo chỉ được chấp nhận khi (a) kết quả dịch ngược trùng với từ gốc, (b) bản dịch tiếng Anh là một *từ đơn* khác với từ gốc, và (c) thuộc từ vựng NeoBERT.
- **Tầng 2 – từ ghép**: các từ ghép tiếng Việt (đa âm tiết) chưa được phủ được dịch trực tiếp sang tiếng Anh, giữ lại khi bản dịch là một từ đơn thuộc từ vựng NeoBERT.
- **Tầng 3 – từ đơn còn lại**: các từ đơn chưa được phủ, sau khi loại bỏ một danh sách từ chức năng và từ dễ nhập nhằng (blacklist), được dịch trực tiếp và giữ theo cùng ràng buộc “một từ đơn thuộc từ vựng NeoBERT”; tầng này được giới hạn số lượng để kiểm soát chi phí và chất lượng.

Cơ sở khoa học của phép kiểm tra *dịch ngược* ở Tầng 1 là: một từ đa nghĩa thường không dịch ngược về chính nó một cách ổn định, do bản

dịch trung gian có thể ứng với một nghĩa khác. Vì vậy, ràng buộc dịch ngược trùng khớp tự nhiên loại bỏ phần lớn các từ đa nghĩa, chỉ giữ lại các cặp tương ứng một-một đáng tin cậy. Ràng buộc “bản dịch là một từ đơn thuộc từ vựng NeoBERT” bảo đảm mỗi cặp neo ứng đúng *một* token NeoBERT.

Tập neo cuối cùng là *hợp* của ba nguồn – neo bề mặt đã xác minh, neo số và cặp dịch ba tầng – sau khi loại các cặp trùng lặp, thu được ánh xạ $t_{vi} \mapsto t_{neo}$ dùng thống nhất cho cả lớp nhúng đầu vào lẫn lớp đầu ra ở Mục 3.5. Thực nghiệm khởi tạo cho thấy việc mở rộng tập neo bằng cặp dịch làm tăng đáng kể số lượng neo và độ phủ ngữ nghĩa, qua đó cải thiện chất lượng khởi tạo so với phương án chỉ dùng neo trùng lặp bề mặt như nghiên cứu gốc.

Để minh họa cụ thể ba nguồn neo, xét bốn token tiếng Việt đại diện. Token *internet* trùng dạng chuỗi ở cả hai từ vựng và dịch máy trả về đúng “internet”, nên được giữ làm **neo bề mặt** và khởi tạo bằng chính hàng nhúng NeoBERT của “internet”. Token số *2024* xuất hiện ở cả hai từ vựng và được nhận trực tiếp làm **neo số** mà không cần qua dịch. Token thuần Việt *nước* khác hẳn dạng chuỗi so với “water”: ở Tầng 1, dịch máy cho “water”, dịch ngược cho lại “nước”, và “water” là một từ đơn thuộc từ vựng NeoBERT – cả ba điều kiện thỏa mãn nên cặp (*nước*, *water*) được thêm làm **cặp dịch**. Ngược lại, token đồng tự khác nghĩa “song” (tiếng Việt: *song song*) dịch sang “parallel” chứ không phải “song”, nên bị loại khỏi tập neo bề mặt. Thuật toán 3.1 tóm tắt toàn bộ quy trình trích xuất tập neo từ ba nguồn.

3.5 Phép chiếu SALT cho lớp nhúng và lớp đầu ra

Mục này trình bày phép chiếu SALT – bước khởi tạo cốt lõi. Với mỗi token tiếng Việt *không thuộc* tập neo, chúng tôi xây dựng nhúng của nó

Thuật toán 3.1 Trích xuất tập neo từ ba nguồn

Require: Từ vựng tiếng Việt $\mathcal{V}_t$, từ vựng NeoBERT $\mathcal{V}_{\text{neo}}$; dịch máy MT; bộ lọc chính tả tiếng Việt $\text{vi}?(.)$

Ensure: Tập neo $\mathcal{A}$ dạng ánh xạ $t_{\text{vi}} \mapsto t_{\text{neo}}$

```
1:  $\mathcal{A} \leftarrow \emptyset$ 
2: for all từ đầy đủ  $t \in \mathcal{V}_t \cap \mathcal{V}_{\text{neo}}$  do ▷ (A) neo bề mặt
3:   if  $\text{MT}(t) = t$  then ▷ loại đồng tự khác nghĩa
4:      $\mathcal{A} \leftarrow \mathcal{A} \cup \{(t, t)\}$ 
5: for all token số  $t \in \mathcal{V}_t \cap \mathcal{V}_{\text{neo}}$  do ▷ (B) neo số
6:    $\mathcal{A} \leftarrow \mathcal{A} \cup \{(t, t)\}$ 
7: for all từ Việt hợp lệ  $t \in \mathcal{V}_t$  thỏa  $\text{vi}(t)$  do ▷ (C) cặp dịch
8:    $e \leftarrow \text{MT}(t)$ 
9:   if  $e$  là từ đơn và  $e \in \mathcal{V}_{\text{neo}}$  và  $\text{MT}^{-1}(e) = t$  then
10:     $\mathcal{A} \leftarrow \mathcal{A} \cup \{(t, e)\}$  ▷ dịch ngược trùng khớp
11: return  $\mathcal{A}$  sau khi khử các cặp trùng lặp
```

bằng một ánh xạ tuyến tính cục bộ học từ các neo lân cận về mặt ngữ nghĩa. Các token neo và token đặc biệt được sao chép trực tiếp hàng tương ứng từ NeoBERT.

3.5.1 Lựa chọn neo bằng Sparsemax

Để xác định các neo liên quan tới một token đích t , chúng tôi đo độ tương đồng ngữ nghĩa trong không gian nhúng từ tĩnh fastText [40], [41]. Gọi $\mathbf{f}_t$ là véc-tơ fastText của t và $\{\mathbf{f}_a\}_{a \in \mathcal{A}}$ là véc-tơ fastText của các neo. Độ tương đồng cosine và trọng số neo được tính như Công thức (3.2):

$$s_{t,a} = \frac{\mathbf{f}_t^\top \mathbf{f}_a}{\|\mathbf{f}_t\| \|\mathbf{f}_a\|}, \quad \mathbf{w}_t = \text{sparsemax}(\mathbf{s}_t), \quad \mathcal{S}_t = \{a : w_{t,a} > 0\} \quad (3.2)$$

Trong đó $\text{sparsemax}(\cdot)$ [36] là phép chiếu lên đơn hình xác suất, cho kết quả *thưa* – chỉ một tập con nhỏ $\mathcal{S}_t$ các neo liên quan nhất nhận trọng số khác 0. Việc dùng Sparsemax thay cho softmax có lý do thực tế cụ thể: softmax luôn gán trọng số khác 0 cho *tất cả* các neo, kể cả những neo ở

rất xa token đích về mặt ngữ nghĩa – khiến ánh xạ tuyến tính cục bộ bị nhiễu bởi các điểm không liên quan. Sparsemax tự động đặt trọng số bằng 0 cho các neo có độ tương đồng thấp; chỉ tập con $\mathcal{S}_t$ thực sự gần nghĩa mới tham gia phép chiếu, giúp ma trận X_t phản ánh đúng quan hệ tuyến tính cục bộ quanh token t .

3.5.2 Ánh xạ tuyến tính cục bộ theo từng token

Như đã lập luận ở Mục 2.3.1, do hai không gian nhúng là bất đẳng hướng và không đẳng cấu, một ánh xạ tuyến tính toàn cục duy nhất là không đủ. Thay vào đó, SALT học một *ánh xạ tuyến tính riêng cho từng token* từ tập neo được chọn. Gọi $A_t^{\text{src}} \in \mathbb{R}^{k \times h}$ là ma trận xếp chồng các nhúng *hiển tặc* (ViDeBERTa) của $k = |\mathcal{S}_t|$ neo trong $\mathcal{S}_t$, và $A_t^{\text{tgt}} \in \mathbb{R}^{k \times h}$ là ma trận các hàng *đích* (NeoBERT) tương ứng. Khi số neo đủ lớn ($k \geq k_0$, chúng tôi chọn $k_0 = 8$ qua thực nghiệm – giá trị nhỏ hơn khiến giả nghịch đảo kém ổn định và chất lượng chiếu giảm đáng kể), ánh xạ cục bộ X_t và nhúng kết quả $\mathbf{e}_t$ được xác định như Công thức (3.3):

$$X_t = (A_t^{\text{src}})^+ A_t^{\text{tgt}}, \quad \mathbf{e}_t = \mathbf{d}_t X_t \quad (3.3)$$

Trong đó $(\cdot)^+$ là *giả nghịch đảo* Moore–Penrose (nghiệm bình phương tối thiểu chuẩn nhỏ nhất), $\mathbf{d}_t \in \mathbb{R}^h$ là nhúng hiển tặc của token t , và h là số chiều ẩn. Ánh xạ X_t học quan hệ tuyến tính giữa hai không gian *cục bộ quanh token* t , dựa trên các neo gần nghĩa nhất với nó.

Khi số neo không đủ ($k < k_0$), giả nghịch đảo trở nên kém ổn định; khi đó chúng tôi thoái lui về *trung bình có trọng số Sparsemax* của các hàng đích, như Công thức (3.4):

$$\mathbf{e}_t = \frac{\sum_{a \in \mathcal{S}_t} w_{t,a} \mathbf{a}^{\text{tgt}}}{\sum_{a \in \mathcal{S}_t} w_{t,a}} \quad (3.4)$$

Trường hợp token không có véc-tơ fastText hợp lệ, nhúng được khởi tạo

ngẫu nhiên theo phân phối chuẩn khớp với thống kê của NeoBERT.

Toàn bộ phép chiếu SALT cho *một* ma trận đích – áp dụng lần lượt cho ma trận nhúng đầu vào và ma trận đầu ra ở Mục 3.5.3 – được tóm tắt trong Thuật toán 3.2.

Thuật toán 3.2 Phép chiếu SALT theo từng token cho một ma trận đích

Require: Từ vựng $\mathcal{V}_t$; tập neo $\mathcal{A}$; nhúng hiện tại $\{\mathbf{d}_t\}$; hàng đích NeoBERT $\{\mathbf{a}^{\text{tgt}}\}$; véc-tơ fastText $\{\mathbf{f}_t\}$; ngưỡng k_0 ; chuẩn trung bình

μ_{neo}

Ensure: Ma trận khởi tạo $E \in \mathbb{R}^{|\mathcal{V}_t| \times h}$

```

1: for all token  $t \in \mathcal{V}_t$  do
2:   if  $t$  là neo hoặc token đặc biệt then
3:      $E[t] \leftarrow \mathbf{a}_t^{\text{tgt}}$  ▷ sao chép nguyên hàng NeoBERT
4:   else if  $t$  không có véc-tơ fastText then
5:      $E[t] \leftarrow$  mẫu ngẫu nhiên khớp thống kê NeoBERT
6:   else
7:      $s_{t,a} \leftarrow \cos(\mathbf{f}_t, \mathbf{f}_a)$  cho mọi  $a \in \mathcal{A}$ 
8:      $\mathbf{w}_t \leftarrow \text{sparsemax}(\mathbf{s}_t)$ ;  $\mathcal{S}_t \leftarrow \{a : w_{t,a} > 0\}$ 
9:     if  $|\mathcal{S}_t| \geq k_0$  then
10:       $X_t \leftarrow (A_t^{\text{src}})^+ A_t^{\text{tgt}}$ ;  $E[t] \leftarrow \mathbf{d}_t X_t$  ▷ ánh xạ cục bộ
11:    else
12:       $E[t] \leftarrow (\sum_a w_{t,a} \mathbf{a}^{\text{tgt}}) / \sum_a w_{t,a}$  ▷ trung bình có trọng số
13:       $E[t] \leftarrow E[t] \cdot \mu_{\text{neo}} / \|E[t]\|$  ▷ hiệu chỉnh chuẩn (ma trận nhúng)
14: return  $E$ 

```

3.5.3 Áp dụng cho cả lớp nhúng đầu vào và lớp đầu ra

Đây là điểm khác biệt then chốt so với SALT gốc. Vì NeoBERT dùng đầu giải mã *tách rời*, ma trận đầu ra W_{dec} là một ma trận độc lập với hình học riêng – và theo kết quả về hiện tượng thoái hóa biểu diễn (Mục 2.3.1), nó *không* thể được khởi tạo bằng cách sao chép ma trận nhúng. Do đó, chúng tôi áp dụng *cùng một quy trình chiếu SALT theo từng token* cho cả hai ma trận, chỉ khác ở tập hàng đích:

- Với **ma trận nhúng đầu vào** E : hàng đích A_t^{tgt} lấy từ các *hàng nhúng* NeoBERT của các neo.
- Với **ma trận đầu ra** W_{dec} : hàng đích A_t^{tgt} lấy từ các *hàng đầu giải mã* NeoBERT của các neo.

Nhờ vậy, mỗi ma trận được khởi tạo trong đúng không gian của nó, phản ánh trung thực quan hệ ngữ nghĩa mà các neo thiết lập trong từng không gian.

3.5.4 Hiệu chỉnh chuẩn của lớp nhúng

Như đã lưu ý ở Mục 2.2.5, NeoBERT dùng Pre-RMSNorm vốn chuẩn hóa theo độ lớn của véc-tơ; do đó độ lớn (norm) của mỗi hàng nhúng ảnh hưởng trực tiếp đến hành vi mô hình. Các nhúng sinh ra từ phép chiếu SALT thường có độ lớn lệch so với phân phối của NeoBERT. Để tránh gây bất ổn định, chúng tôi hiệu chỉnh độ lớn của mỗi hàng *không phải neo* về độ lớn trung bình của NeoBERT trong khi giữ nguyên hướng, như Công thức (3.5):

$$\mathbf{e}_t \leftarrow \mathbf{e}_t \cdot \frac{\mu_{\text{neo}}}{\|\mathbf{e}_t\|}, \quad \mu_{\text{neo}} = \frac{1}{|\mathcal{V}_{\text{neo}}|} \sum_v \|E_v^{\text{neo}}\| \quad (3.5)$$

Trong đó μ_{neo} là độ lớn trung bình của các hàng nhúng NeoBERT. Các hàng neo và token đặc biệt được giữ nguyên để bảo toàn độ trung thực với NeoBERT.

3.5.5 Khởi tạo độ chệch đầu ra theo tần suất

Lớp đầu ra của NeoBERT có một véc-tơ độ chệch b_{dec} mã hóa *tiên nghiệm tần suất* của token. Các phương pháp khởi tạo trước đây thường để độ chệch này bằng 0, khiến mô hình ở bước đầu phải lãng phí công sức học lại phân phối tần suất biên. Đề tài khởi tạo b_{dec} theo *log-tần-suất từ đơn* (unigram log-frequency) của tiếng Việt theo phương pháp của Meister

và cộng sự [42], như Công thức (3.6):

$$b_{\text{dec},v} = \log p_v, \quad p_v = \frac{c_v + 1}{\sum_{u \in \mathcal{V}_t} c_u + |\mathcal{V}_t|} \quad (3.6)$$

Trong đó c_v là số lần xuất hiện của token v trong ngữ liệu tiếng Việt (đếm qua 20.000 văn bản CulturaX-vi với bộ tách từ đã cắt tĩa), và phép cộng 1 là làm trơn Laplace. Cụ thể, phép đếm sử dụng đúng cùng bộ tách từ sẽ dùng trong huấn luyện, bảo đảm chỉ số token trong b_{dec} khớp với các hàng của ma trận nhúng và ma trận đầu ra.

Cơ sở lý thuyết của cách khởi tạo này đến từ một quan sát của Meister và cộng sự [42]: lớp tuyến tính cuối phân rã tự nhiên thành *tích của hai phân phối* (product of experts):

$$p_\theta(y \mid y_{<t}) \propto \underbrace{e^{W\phi(y_{<t})}}_{p_{W\phi}(\cdot \mid y_{<t}) \text{ (ngữ cảnh)}} \cdot \underbrace{e^b}_{p_b(\cdot) \text{ (tiên nghiệm)}} \quad (3.7)$$

Thành phần p_b là phân phối vô điều kiện hoàn toàn độc lập với ngữ cảnh. Khi $b_{\text{dec}} = \log p_v$, thành phần này mã hóa phân phối từ đơn tiếng Việt, nhờ đó *giải phóng* phần còn lại $p_{W\phi}$ để chuyên biệt hóa vào thông tin ngữ nghĩa thay vì phải mô hình hóa lại tần suất bề mặt. Điều này có thể diễn đạt chặt chẽ qua gradient hàm mất mát theo độ chệch:

$$\frac{\partial \mathcal{L}}{\partial b_v} = \mathbb{E}[\text{softmax}(y)_v] - \hat{p}_v \quad (3.8)$$

tức hiệu giữa phân phối biên mô hình đang dự đoán và phân phối biên thực nghiệm của dữ liệu. Khi $b = \log p_v$ và γ đủ nhỏ, $\mathbb{E}[\text{softmax}(y)_v] \approx p_v \approx \hat{p}_v$, nên gradient của b_v xấp xỉ bằng không ngay từ bước đầu: toàn bộ tín hiệu gradient được dồn vào W_{dec} và phần thân Transformer để học ngữ cảnh và ngữ nghĩa, thay vì lãng phí vào việc hiệu chỉnh phân phối biên. Meister và cộng sự xác nhận thực nghiệm rằng: (i) ngay từ bước đầu huấn luyện, mô hình ngôn ngữ đã “thu hẹp” đầu ra về phân phối từ đơn bất kể ngữ cảnh,

và quá trình đó tốn hàng trăm đến hàng nghìn bước gradient nếu khởi tạo $b = \mathbf{0}$; (ii) khởi tạo theo log-unigram cải thiện hiệu quả huấn luyện (diện tích dưới đường cong BLEU trên tập phát triển) trên 5/6 cặp ngôn ngữ thử nghiệm, đồng thời cải thiện BLEU cuối trên cùng số cặp.

Nhờ cách khởi tạo này, hàm mất mát khởi điểm xấp xỉ *entropy từ đơn* của tiếng Việt ($\approx 7,3$ nat), thấp hơn đáng kể so với mức ngẫu nhiên $\log |\mathcal{V}_t|$ ($\approx 10,3$ nat với $|\mathcal{V}_t| = 30.522$). Ý tưởng khởi tạo độ chệch theo xác suất tiên nghiệm để ổn định giai đoạn đầu huấn luyện cũng được Lin và cộng sự [43] áp dụng trong ngữ cảnh phát hiện đối tượng với mất cân bằng lớp nặng, mặc dù bài toán đó dùng một hằng số vô hướng tiên nghiệm thay vì toàn bộ phân phối biên trên từ vựng.

3.5.6 Co trọng số đầu ra để tiên nghiệm dẫn dắt giai đoạn đầu

Tiên nghiệm tần suất ở trên chỉ phát huy tác dụng nếu ở bước khởi đầu, thành phần logit do trọng số W_{dec} sinh ra *không lấn át* độ chệch b_{dec} . Ngay sau phép chiếu, các hàng của W_{dec} tuy đã mang cấu trúc ngữ nghĩa từ SALT nhưng chưa từng được huấn luyện cùng phần thân trên tiếng Việt, nên biên độ logit của chúng chưa đáng tin cậy: nếu giữ nguyên, các logit “nhiều” này sẽ nhấn chìm tín hiệu tiên nghiệm. Đây là điểm khác biệt then chốt so với huấn luyện từ đầu: trong ngữ cảnh đó, W_{dec} được khởi tạo ngẫu nhiên với giá trị gần 0, nên điều kiện “tiên nghiệm dẫn dắt” thỏa mãn tự nhiên mà không cần thêm can thiệp [42]. Trong chuyển giao ngôn ngữ chéo, W_{dec} kế thừa biên độ lớn từ không gian NeoBERT tiếng Anh, khiến điều kiện đó không còn tự nhiên thỏa mãn và phải được khôi phục thủ công – đây là nhu cầu đặc thù của bài toán chuyển giao lớp nhúng mà phương pháp đề xuất cần giải quyết. Vì vậy, chúng tôi *co* (scale) trọng số đầu ra bằng một hệ số vô hướng γ như Công thức (3.9):

$$y = \gamma W_{\text{dec}} h + b_{\text{dec}}, \quad \gamma = 0,1 \quad (3.9)$$

Trong đó h là biểu diễn ẩn ở đầu ra của phần thân và γ là hệ số co áp dụng một lần lên W_{dec} tại thời điểm khởi tạo. Cơ sở khoa học của lựa chọn này gồm hai vế:

- **Gần 0 – để tiên nghiệm dẫn dắt:** với γ nhỏ, $\|\gamma W_{\text{dec}} h\|$ trở nên nhỏ so với biên độ của b_{dec} , nên phân phối dự đoán ban đầu do tiên nghiệm tần suất chi phối: mất mát khởi điểm xấp xỉ entropy từ đơn của tiếng Việt, mô hình *bỏ qua* giai đoạn phải học lại phân phối biên và dồn toàn bộ tín hiệu gradient vào việc học ngữ nghĩa.
- **Khác 0 – để giữ ngữ nghĩa:** phép nhân vô hướng bảo toàn *hướng* của các hàng W_{dec} , tức bảo toàn cấu trúc lân cận ngữ nghĩa thu được từ phép chiếu SALT. Trong quá trình huấn luyện, mô hình chỉ cần *khuếch đại dần* biên độ của tín hiệu ngữ nghĩa sẵn có thay vì học lại từ đầu, nhờ đó tốc độ học về sau nhanh hơn so với một lớp đầu ra ngẫu nhiên.

Giả thuyết này được kiểm chứng bằng thực nghiệm trong Chương 4: khi giữ nguyên biên độ ($\gamma = 1$) hoặc chỉ co một nửa ($\gamma = 0,5$), mất mát khởi điểm cao hơn hẳn (với $\gamma = 1$ gần chạm mức ngẫu nhiên $\log |\mathcal{V}_t|$, với $\gamma = 0,5$ vẫn cao hơn nhiều mức entropy từ đơn) và – quan trọng hơn – quá trình huấn luyện bị *mắc kẹt* ở mức mất mát cao, không hội tụ về vùng của các phương án tốt. Hiện tượng này không chỉ là hệ quả của điểm khởi đầu cao: một mô hình khởi tạo ngẫu nhiên chuẩn cũng bắt đầu gần $\log |\mathcal{V}_t|$ nhưng vẫn hội tụ bình thường. Dữ liệu thực nghiệm (Chương 4) xác nhận một phần cơ chế này: tại bước 10, chuẩn gradient (`grad_norm`) của $\gamma = 1,0$ đạt ≈ 68 – cao hơn 33 lần so với ≈ 2 của $\gamma = 0,1$ – trong khi ở bước đó logits chủ yếu vẫn là nhiễu vì W_{dec} chưa hề được căn chỉnh trên tiếng Việt. Điều này xác nhận các gradient ngược xuất hiện sớm ở $\gamma = 1$ có biên độ rất lớn nhưng mang ít thông tin ngữ nghĩa. Một cách lý giải là các gradient đó làm biến dạng cấu trúc lân cận ngữ nghĩa mà SALT đã xây dựng trong W_{dec} , phá hủy tài sản chuyển giao trước khi nó kịp phát huy hiệu quả; phần này vẫn cần kiểm chứng trực tiếp, ví dụ bằng cách

đo độ dịch chuyển tương đồng cosine của các hàng W_{dec} qua từng bước huấn luyện đầu. Trong khi đó, $\gamma = 0,1$ vừa cho khởi điểm thấp vừa hội tụ nhanh. Do đó $\gamma = 0,1$ được chọn cho toàn bộ quy trình.

3.6 Giai đoạn căn chỉnh đóng băng

Sau bước khởi tạo, ma trận nhúng và ma trận đầu ra của tiếng Việt là *mới* đối với phần thân Transformer đã được huấn luyện trên tiếng Anh. Nếu lập tức tinh chỉnh toàn bộ mô hình, phần thân có thể bị kéo theo để thích ứng với hai ma trận mới còn chưa ổn định, làm gia tăng nguy cơ quên lãng thảm họa (Mục 2.3.2). Vì vậy, chúng tôi chèn một *giai đoạn căn chỉnh đóng băng*: đóng băng toàn bộ phần thân Transformer và chỉ huấn luyện ma trận nhúng đầu vào, ma trận đầu ra và độ chệch của nó trên một lượng nhỏ ngữ liệu tiếng Việt với mục tiêu MLM.

Trực giác của bước này là để hai ma trận vừa khởi tạo *căn chỉnh với nhau và với phần thân cố định* trong không gian biểu diễn trước khi cho phép phần thân thay đổi; nhờ đó điểm khởi đầu của giai đoạn tiếp tục tiền huấn luyện ổn định hơn. Cách làm này nằm trong tinh thần chiến lược *đóng băng rồi thích nghi* của de Vries và Nissim [28], vốn cho thấy việc tách giai đoạn học lớp nhúng khỏi giai đoạn tinh chỉnh toàn bộ giúp tái sử dụng mô hình hiệu quả hơn. Trong thực nghiệm (Chương 4), chúng tôi quan sát thấy giai đoạn này *luôn* cải thiện chất lượng lớp nhúng khởi tạo, do đó được giữ lại trong quy trình.

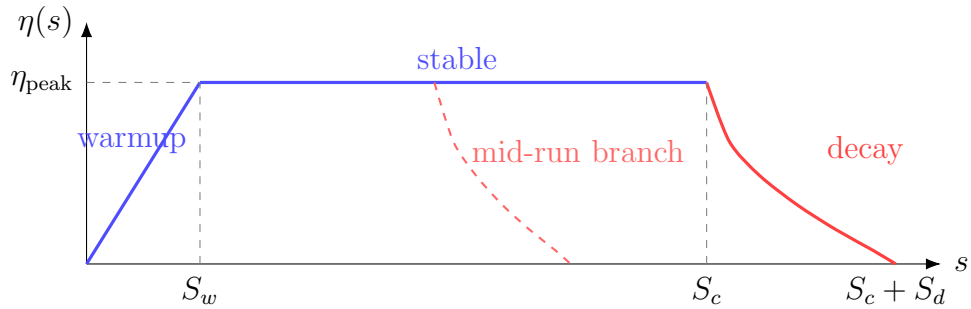
3.7 Tiếp tục tiền huấn luyện thích nghi

Ở giai đoạn cuối, toàn bộ mô hình được tinh chỉnh trên ngữ liệu tiếng Việt quy mô lớn CulturaX với mục tiêu MLM. Theo Gururangan và cộng sự [27], việc tiếp tục tiền huấn luyện trên dữ liệu miền đích mang lại cải thiện ổn định; ở đây miền đích là toàn bộ ngôn ngữ tiếng Việt.

Lịch tốc độ học WSD. Chúng tôi sử dụng lịch *Warmup–Stable–Decay* (WSD) [31] thay cho lịch cosine cố định, vì WSD tách pha ổn định khỏi pha làm nguội nên cho phép kéo dài huấn luyện tùy ý mà không phải cam kết trước tổng số bước. Tốc độ học η theo bước s được định nghĩa như Công thức (3.10):

$$\eta(s) = \begin{cases} \eta_{\text{peak}} \cdot \frac{s}{S_w} & \text{nếu } s < S_w \quad (\text{khởi động}) \\ \eta_{\text{peak}} & \text{nếu } S_w \leq s < S_c \quad (\text{ổn định}) \\ \eta_{\text{peak}} \cdot \left(1 - \sqrt{\frac{s-S_c}{S_d}}\right) & \text{nếu } s \geq S_c \quad (\text{làm nguội}) \end{cases} \quad (3.10)$$

Trong đó S_w là mốc kết thúc khởi động, S_c là mốc bắt đầu làm nguội, và S_d là độ dài giai đoạn làm nguội. Pha *ổn định* giữ tốc độ học ở đỉnh trong phần lớn quá trình, còn pha *làm nguội* chỉ chiếm một phần nhỏ cuối ngân sách và dùng dạng căn bậc hai $1 - \sqrt{f}$ (với $f \in [0, 1]$ là phần bước đã trôi qua trong pha làm nguội) – dạng mà Hägele và cộng sự [32] chứng minh cho chất lượng hạ nguồn tốt nhất ở một ngân sách cho trước. Hình 3.2 minh họa hình dạng của lịch và ưu điểm cốt lõi của nó: tại bất kỳ điểm nào trong pha ổn định đều có thể “rẽ nhánh” một pha làm nguội ngắn để thu về mô hình hoàn chỉnh, trong khi nhánh chính tiếp tục huấn luyện – điều mà lịch cosine (phải cam kết trước tổng số bước) không làm được.



Hình 3.2: Lịch tốc độ học Warmup–Stable–Decay (WSD).

Khởi động lại tốc độ học. Một điểm kiểm tra đã được làm nguội về tốc độ học gần 0 gần như không thể cập nhật trọng số nếu huấn luyện tiếp

mà không xử lý. Do đó, mỗi khi tiếp tục huấn luyện trên dữ liệu mới (ví dụ một pha làm nguội trên dữ liệu mới), tốc độ học được *khởi động lại* rồi làm nguội lại theo đúng khuyến nghị của Ibrahim và cộng sự [30]. Ngoài ra, khi huấn luyện tiếp, trạng thái bộ tối ưu AdamW (các mô-men bậc nhất và bậc hai, với $\beta = (0,9; 0,95)$ theo NeoBERT) được nạp lại *nguyên vẹn*, để bộ tiền điều kiện của bộ tối ưu khớp với chế độ mà phần thân đã được huấn luyện.

3.8 Tổng kết phương pháp

Tóm lại, phương pháp xây dựng ViNeoBERT gồm ba giai đoạn: (1) khởi tạo lớp nhúng và lớp đầu ra bằng phép chiếu SALT theo từng token từ ViDeBERTa sang không gian NeoBERT, với tập neo được làm giàu bằng cặp dịch và độ chệch đầu ra khởi tạo theo tần suất; (2) căn chỉnh đóng băng để hai ma trận mới ổn định cùng phần thân; và (3) tiếp tục tiền huấn luyện thích nghi theo lịch WSD. So với SALT gốc, đề tài có ba khác biệt chính: *mở rộng tập neo bằng cặp dịch* (Mục 3.4), *áp dụng phép chiếu cho cả lớp đầu ra tách rời – kèm khởi tạo độ chệch theo tần suất và co trọng số đầu ra để tiên nghiệm dẫn dắt giai đoạn đầu* (Mục 3.5), và *bổ sung giai đoạn căn chỉnh đóng băng* (Mục 3.6). Hiệu quả của các thành phần này được đánh giá trong Chương 4.

Chương 4

Thực nghiệm và Đánh giá

Chương này trình bày quá trình thực nghiệm nhằm kiểm chứng phương pháp đã đề xuất ở Chương 3. Trước hết, chúng tôi mô tả môi trường, dữ liệu và cấu hình huấn luyện (Mục 4.1–4.2), cùng các độ đo được sử dụng (Mục 4.3). Phần kết quả được tổ chức theo hai câu hỏi nghiên cứu: (1) *phương án khởi tạo nào là tốt nhất?* – trả lời bằng thí nghiệm so sánh các phương án khởi tạo ở cùng ngân sách huấn luyện (Mục 4.4); và (2) *mô hình cuối cùng đạt chất lượng ra sao so với các mô hình hiện có?* – trả lời bằng đường cong chất lượng theo mốc ngân sách, bảng đánh giá đa nhiệm trên mười hai tác vụ hiểu tiếng Việt, và kiểm chứng ngữ cảnh dài sau pha mở rộng lên 4.096 token (Mục 4.5). Cuối chương là phần thảo luận chung (Mục 4.6).

4.1 Môi trường và dữ liệu thực nghiệm

4.1.1 Môi trường thực nghiệm

Toàn bộ thực nghiệm được thực hiện trên GPU NVIDIA A100 (bộ nhớ 80GB, nền tảng Google Colab), sử dụng PyTorch và hệ sinh thái Hugging Face (`transformers`, `datasets`, `safetensors`).

4.1.2 Dữ liệu tiền huấn luyện

Giai đoạn căn chỉnh đóng băng và tiếp tục tiền huấn luyện sử dụng phần tiếng Việt của CulturaX [44] – kho ngữ liệu web đa ngữ quy mô lớn đã được làm sạch và khử trùng lặp qua nhiều tầng lọc. Dữ liệu được truyền trực tuyến (streaming); tổng ngân sách sử dụng xấp xỉ 5 tỷ token, tương đương khoảng 20GB văn bản. Bảng 4.1 tóm tắt thống kê dữ liệu sử dụng trong đề tài.

Bảng 4.1: Thống kê dữ liệu thực nghiệm.

Dữ liệu	Vai trò	Quy mô sử dụng
CulturaX (tiếng Việt)	Căn chỉnh đóng băng	~20 triệu token
CulturaX (tiếng Việt)	Tiếp tục tiền huấn luyện	~5 tỷ token (~20GB văn bản)
UIT-ViQuAD 2.0 (câu hỏi trả lời được)	Tinh chỉnh hạ nguồn	~18 nghìn câu hỏi huấn luyện
Bộ đánh giá đa nhiệm (Mục 4.1.3)	Tinh chỉnh hạ nguồn	12 tác vụ: phân loại, suy luận, gán nhãn, tương đồng ngữ nghĩa, Retrieval, Reranking

4.1.3 Dữ liệu đánh giá hạ nguồn

Mô hình được đánh giá trên một bộ tác vụ hiểu ngôn ngữ tiếng Việt đa dạng, phủ các nhóm năng lực khác nhau của một Encoder:

- **Hỏi đáp đọc hiểu:** UIT-ViQuAD 2.0 [45] – bộ dữ liệu hỏi đáp trích xuất xây dựng theo phương pháp luận của SQuAD; mô hình phải xác định đoạn văn bản trả lời trong ngữ cảnh cho trước. Chúng tôi chỉ dùng các câu hỏi *trả lời được* (answerable); tập validation công khai được chia đôi ngẫu nhiên thành tập phát triển và tập kiểm tra. Đây là tác vụ chính vì đủ khó để phân hóa chất lượng biểu diễn giữa các mô hình tốt.
- **Suy luận ngôn ngữ tự nhiên:** XNLI (phần tiếng Việt) [46],

ViANLI (bộ NLI đối kháng, khó hơn đáng kể) [47] và QNLI bản tiếng Việt (suy luận câu hỏi–câu, phỏng theo GLUE [48]).

- **Phân loại văn bản:** UIT-VSFC (phản hồi sinh viên) [49], UIT-VSMEC (cảm xúc đa lớp) [50], UIT-ViCTSD (bình luận độc hại) [51].
- **Tính chấp nhận được về ngữ pháp:** bộ dữ liệu kiểu CoLA [52] cho tiếng Việt (ViCoLA-syn), trong đó câu sai ngữ pháp được sinh tự động từ câu đúng; mô hình phải phân biệt hai loại câu.
- **Gán nhãn chuỗi:** gán nhãn từ loại trên treebank Universal Dependencies Vietnamese-VTB [53].
- **Tương đồng ngữ nghĩa:** bộ STS bản tiếng Việt [54], đo tương quan giữa điểm dự đoán và điểm do con người gán; và tác vụ phát hiện diễn đạt lại (paraphrase) – phân loại nhị phân cặp câu tương đương ngữ nghĩa xây dựng từ dữ liệu STS.
- **Biểu diễn câu (Retrieval và Reranking):** Retrieval và Reranking xây dựng từ các cặp (câu hỏi, đoạn văn) của UIT-ViQuAD. Hai tác vụ này được đánh giá theo giao thức biểu diễn câu của NeoBERT cho MTEB [5]: lấy biểu diễn câu bằng gộp trung bình (mean pooling) rồi tính chỉnh tương phản nhẹ, áp dụng *đồng nhất* cho mọi mô hình so sánh.

Bảng 4.2 tổng hợp toàn bộ bộ đánh giá hạ nguồn cùng nhóm năng lực và độ đo tương ứng của từng tác vụ.

Bảng 4.3 đặt ViNeoBERT và các mô hình đối sánh trong bối cảnh tổng thể, tương tự cách NeoBERT [5] so sánh với ModernBERT và NomicBERT.

4.2 Cấu hình huấn luyện

Bảng 4.4 liệt kê siêu tham số của từng giai đoạn trong quy trình. Các giá trị được giữ cố định cho *mọi* phương án khởi tạo trong thí nghiệm so

Bảng 4.2: Các tác vụ đánh giá hạ nguồn.

Tác vụ (bộ dữ liệu)	Nhóm năng lực	Độ đo
UIT-ViQuAD 2.0	Hỏi đáp đọc hiểu	EM, F1 ($\uparrow$)
XNLI (vi)	Suy luận ngôn ngữ	Accuracy ($\uparrow$)
ViANLI	Suy luận đối kháng	F1-macro ($\uparrow$)
QNLI (vi)	Suy luận câu hỏi-câu	Accuracy ($\uparrow$)
UIT-VSFC	Phân loại phản hồi	F1-weighted ($\uparrow$)
UIT-VSMEC	Phân loại cảm xúc	F1-macro ($\uparrow$)
UIT-ViCTSD	Phân loại độc hại	F1-macro ($\uparrow$)
ViCoLA-syn	Chấp nhận ngữ pháp	MCC ($\uparrow$)
UD-VTB	Gán nhãn từ loại	F1-macro ($\uparrow$)
STS (vi)	Tương đồng ngữ nghĩa	Spearman ($\uparrow$)
Paraphrase (STS)	Phát hiện diễn đạt lại	F1-macro ($\uparrow$)
Retrieval (ViQuAD)	Biểu diễn câu	nDCG@10 ($\uparrow$)
Reranking (ViQuAD)	Biểu diễn câu	MRR ($\uparrow$)

Bảng 4.3: Các mô hình Encoder được đánh giá.

Mô hình	Tham số	Kiến trúc	Dữ liệu tiếng Việt	Ngữ cảnh
PhoBERT-base	135M	RoBERTa	20GB $\times$ 40 epoch $\approx$ 140B token	512
PhoBERT-base-v2	135M	RoBERTa	+120GB OSCAR-2301	512
PhoBERT-large	370M	RoBERTa	Cùng base-v2	512
XLM-R-base	278M	RoBERTa	$\approx$ 137GB CC-100 vi ($\approx$ 25B token vi)	512
XLM-R-large	560M	RoBERTa	Cùng base	512
ViNeoBERT	$\approx$ 245M	NeoBERT	20GB, 1 lượt ($\approx$ 5B token)	4.096

sánh, bảo đảm khác biệt duy nhất giữa các phương án là cách khởi tạo lớp nhúng và lớp đầu ra.

Đối với bước khởi tạo SALT, các siêu tham số chính gồm: số neo kích hoạt tối đa qua sparsemax là 64, ngưỡng số neo tối thiểu cho ánh xạ cục bộ $k_0 = 8$, hệ số co trọng số đầu ra $\gamma = 0,1$, và độ chệch đầu ra được ước lượng từ một mẫu CulturaX khoảng 20 triệu token (Mục 3.5). Ở bước tinh chỉnh hạ nguồn, tác vụ chính (hỏi đáp đọc hiểu) dùng AdamW với tốc độ học 2×10^{-5} , chuỗi tối đa 384, batch 8, tối đa 5 epoch (dừng sớm); các tác vụ còn lại trong bộ đánh giá đa nhiệm được tinh chỉnh theo quy trình tương tự với siêu tham số phù hợp từng tác vụ, giữ đồng nhất giữa các mô hình so sánh.

Bảng 4.4: Siêu tham số huấn luyện.

Siêu tham số	Căn chỉnh đóng băng	CPT (WSD)	Mở rộng ngữ cảnh
Tốc độ học đỉnh	10^{-4}	10^{-4}	10^{-5}
Khởi động (warmup)	10% số bước	2% (chặn [20, 500] bước)	50 bước
Độ dài chuỗi tối đa	1.024	1.024	4.096
Batch size hiệu dụng	$32 \times 4 = 128$	$32 \times 16 = 512$	$4 \times 32 = 128$
Tỷ lệ che MLM	20%	20%	20%
Bộ tối ưu	AdamW, $\beta = (0,9; 0,95)$, weight decay 0,01		
Lịch tốc độ học	tuyến tính	WSD, làm nguội theo CT 3.10	hằng số sau khởi động
Số epoch	1 lượt ($\sim 20M$ token)	1 lượt (~ 5 tỷ token)	1 lượt ($\sim 200M$ token)

4.3 Độ đo đánh giá

Chúng tôi sử dụng các nhóm độ đo sau; trừ khi ghi chú khác, mọi kết quả tính chỉnh được tính trung bình trên 5 lần chạy với hạt giống khác nhau và báo cáo kèm độ lệch chuẩn:

- **Mất mát MLM và perplexity (PPL)(↓)**: mất mát cross-entropy của tác vụ mô hình hóa ngôn ngữ có mặt nạ trên tập giữ lại (held-out) của CulturaX, và perplexity tương ứng $PPL = \exp(\mathcal{L}_{MLM})$ – dùng để theo dõi quá trình huấn luyện.
- **Exact Match (EM)(↑) và F1(↑)** cho hỏi đáp đọc hiểu: EM là tỷ lệ câu trả lời trùng khớp hoàn toàn với đáp án chuẩn sau chuẩn hóa; F1 đo mức chồng lấp ở cấp token giữa câu trả lời dự đoán và đáp án chuẩn.
- **Accuracy, F1-macro, F1-weighted (↑)** cho các tác vụ phân loại và suy luận: F1-macro lấy trung bình đều giữa các lớp (phù hợp dữ liệu lệch lớp), F1-weighted lấy trung bình theo tỷ trọng lớp.
- **Hệ số tương quan Matthews (MCC)(↑)** cho tác vụ chấp nhận được về ngữ pháp: độ đo cân bằng cho phân loại nhị phân lệch lớp, nhận giá trị trong $[-1; 1]$ – quy ước chuẩn của bộ CoLA gốc.
- **Hệ số tương quan Spearman (↑)** cho tác vụ tương đồng ngữ nghĩa: đo tương quan hạng giữa điểm dự đoán và điểm con người

gán.

- **nDCG@10** ($\uparrow$) và **MRR** ($\uparrow$) cho truy hồi và xếp hạng lại: nDCG@10 đo chất lượng danh sách 10 kết quả đầu với chiết khấu theo vị trí; MRR là trung bình nghịch đảo thứ hạng của ngữ cảnh đúng. Hai tác vụ này được chạy một lần theo giao thức biểu diễn câu chung nên không kèm độ lệch chuẩn.

Định nghĩa hình thức (công thức tường minh) của F1 ở cấp token, MCC, nDCG@ k và MRR được trình bày trong Phụ lục B.

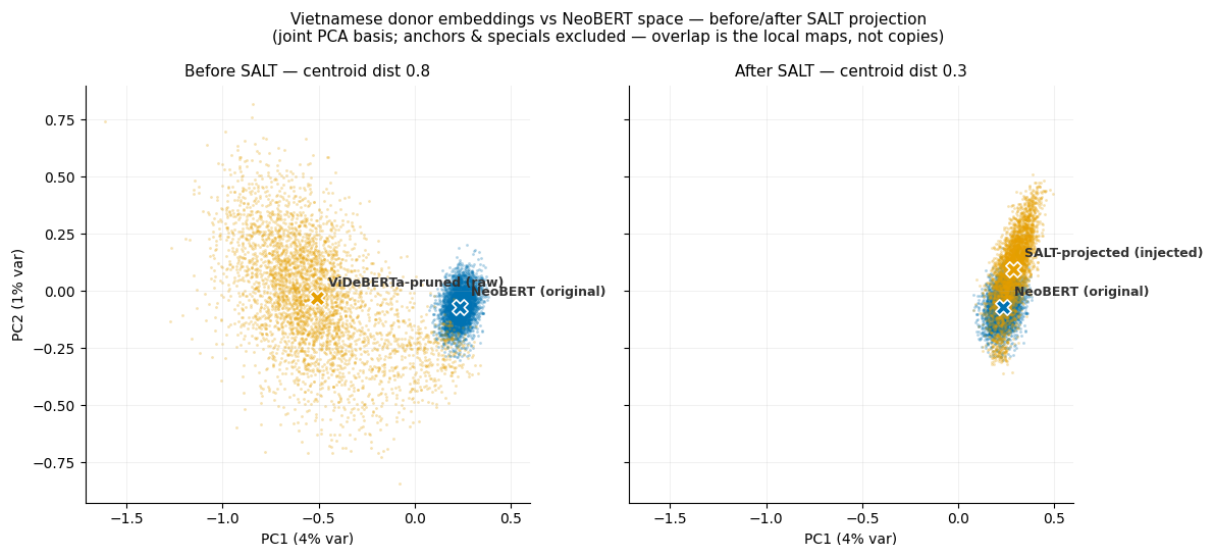
4.4 So sánh các phương án khởi tạo

Mục này kiểm chứng các lựa chọn thiết kế của phương pháp theo ba bước. Trước hết, chúng tôi kiểm tra trực quan xem phép chiếu SALT có thực sự đưa từ vựng tiếng Việt vào đúng không gian của NeoBERT hay không (Mục 4.4.1). Sau đó, hai quyết định thiết kế được kiểm chứng bằng thí nghiệm có kiểm soát ở *cùng ngân sách dữ liệu và cùng cấu hình*, mỗi thí nghiệm chỉ thay đổi đúng thành phần đang xét: chọn hệ số co trọng số đầu ra γ (Mục 4.4.2), và chọn chiến lược khởi tạo hai ma trận từ vựng (Mục 4.4.3).

4.4.1 Kiểm chứng hình học của phép chiếu SALT

Trước khi so sánh các phương án bằng huấn luyện, chúng tôi kiểm chứng trực tiếp câu hỏi căn bản nhất của phương pháp: *phép chiếu SALT có thực sự đưa các vectơ tiếng Việt vào đúng vùng không gian mà phần thân NeoBERT mong đợi hay không?* Hình 4.1 trực quan hóa lớp nhúng đầu vào tại thời điểm khởi tạo, trước khi bất kỳ bước huấn luyện nào diễn ra.

Về cách dựng hình: mỗi điểm là vectơ nhúng của một token, giảm chiều bằng PCA với *hệ trục chung* (joint PCA basis) – các thành phần chính



Hình 4.1: Lớp nhúng tiếng Việt trước và sau phép chiếu SALT.

được tính trên hợp của cả hai tập vectơ, nên vị trí tương đối giữa hai đám mây có thể so sánh trực tiếp. Hai trục là hai thành phần chính đầu tiên, giải thích lần lượt 4% và 1% phương sai (“PC1 (4% var)”, “PC2 (1% var)” trong hình); tỷ lệ nhỏ này là điều bình thường với không gian 768 chiều, nhưng hai hướng “lớn nhất” đó đủ để bộc lộ độ lệch mang tính hệ thống giữa hai không gian. Quan trọng nhất, *các token neo và token đặc biệt đã bị loại khỏi hình* – chúng được sao chép nguyên vẹn từ NeoBERT nên nếu giữ lại sẽ tạo ra sự chồng lấp giả tạo; phần chồng lấp quan sát được vì vậy hoàn toàn là sản phẩm của các ánh xạ cục bộ theo token (Mục 3.5), không phải của các dòng sao chép.

Ở hình bên trái (“Before SALT”), đám mây nhúng thô của từ vựng ViDeBERTa sau cắt tỉa (“ViDeBERTa-pruned (raw)”, màu vàng) nằm lệch hẳn khỏi đám mây NeoBERT gốc (“NeoBERT (original)”, màu xanh): khoảng cách giữa hai tâm (centroid) trên mặt phẳng PCA là 0,8 đơn vị. Hai mô hình được huấn luyện hoàn toàn độc lập nên không gian nhúng của chúng khác nhau cả về *vị trí lẫn thang độ lớn*: đám mây ViDeBERTa trải rộng hơn hẳn so với đám mây NeoBERT. Nếu tiêm thẳng các vectơ thô này vào mô hình, phần thân sẽ nhận đầu vào lệch phân phối ở cả hai

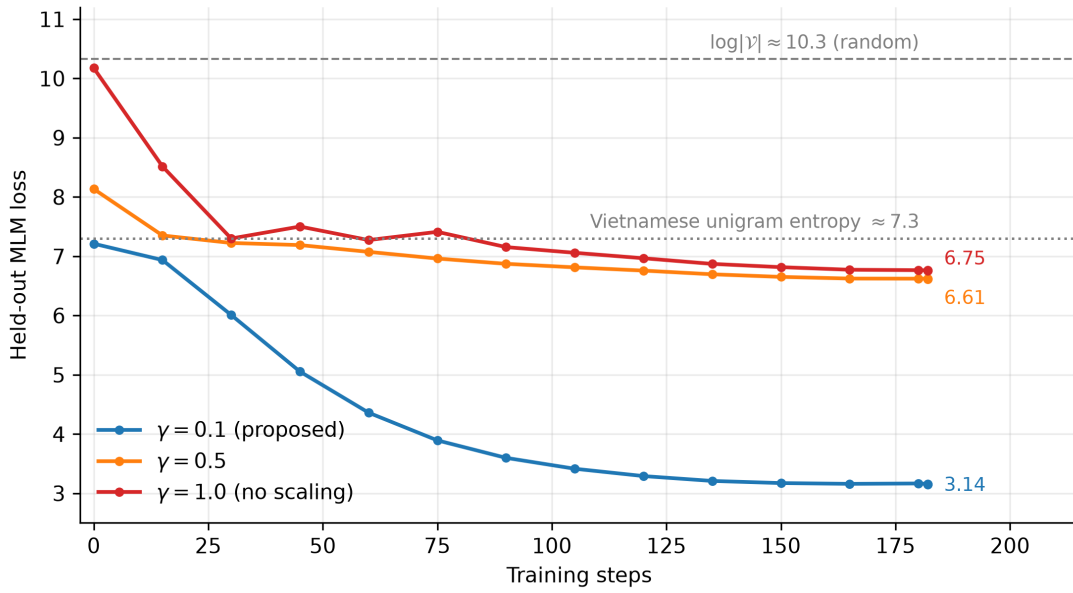
phương diện đó, và các khối Pre-RMSNorm vốn nhạy với độ lớn sẽ khuếch đại sai lệch qua 28 tầng.

Ở hình bên phải (“After SALT”), các vectơ sau phép chiếu (“SALT-projected (injected)” – tức các dòng thực sự được tiêm vào mô hình) đã phủ chồng lên đám mây NeoBERT: khoảng cách hai tâm giảm còn 0,3 – chỉ còn khoảng một phần ba so với trước – và độ trải thu về khớp với thang độ lớn của NeoBERT. Nói cách khác, phép chiếu theo token đưa từ vựng tiếng Việt về đúng *vị trí* trong không gian mà phần thân NeoBERT được huấn luyện để xử lý, còn bước hiệu chỉnh chuẩn (Mục 3.5) đưa chúng về đúng *thang độ lớn*, trong khi cấu trúc ngữ nghĩa tương đối giữa các token vẫn được bảo toàn. Có thể thấy phần đuôi màu vàng còn vươn nhẹ lên phía trên theo PC2 – chủ yếu là các token hiếm rơi vào nhánh trung bình có trọng số hoặc khởi tạo ngẫu nhiên (Mục 3.5); chính giai đoạn căn chỉnh đóng băng ở bước kế tiếp sẽ kéo nốt phần này về khớp với phần thân. Đây cũng là lời giải thích hình học cho quan sát mở đầu mục sau: ngay sau khởi tạo SALT (trước căn chỉnh đóng băng), mô hình đã có mất mát chỉ 7,21 thay vì mức đoán ngẫu nhiên trên 10.

4.4.2 Lựa chọn hệ số co trọng số đầu ra

Thí nghiệm đầu tiên trả lời câu hỏi: *cần co trọng số đầu ra sau phép chiếu SALT tới mức nào?* Ba phương án giống hệt nhau về khởi tạo – SALT theo token cho cả hai ma trận, độ chệch đầu ra theo tần suất từ đơn – và chỉ khác đúng hệ số co $\gamma \in \{0,1; 0,5; 1,0\}$ được tiếp tục tiền huấn luyện ở cùng cấu hình. Hình 4.2 trình bày mất mát đánh giá, kèm hai mức tham chiếu: entropy từ đơn tiếng Việt ($\approx 7,3$) – mức mất mát khi mô hình dự đoán thuần theo tần suất – và mức đoán ngẫu nhiên $\log |\mathcal{V}_t| \approx 10,3$.

Kết quả khớp từng điểm với phân tích ở Mục 3.5.6. Phương án không co ($\gamma = 1,0$) khởi đầu ở 10,18 – sát mức đoán ngẫu nhiên: các logit nhiễu từ ma trận vừa chiếu đã nhấn chìm hoàn toàn tiên nghiệm tần suất cài trong độ chệch. Phương án co một nửa ($\gamma = 0,5$) khởi đầu ở 8,13, lơ lửng



Hình 4.2: Mất mát MLM theo hệ số co γ .

giữa hai mức tham chiếu. Chuẩn gradient ở những bước đầu phản ánh đúng mức sai lệch: 68,4 với $\gamma = 1,0$ và 15,8 với $\gamma = 0,5$, so với chỉ 2,1 của $\gamma = 0,1$ – các cập nhật đầu tiên của hai phương án γ lớn đủ mạnh để phá vỡ cấu trúc ngữ nghĩa mà SALT vừa dựng, và hệ quả là cả hai *mắc kẹt* quanh mức 6,61–6,75 cho tới hết ngân sách. Ngược lại, phương án $\gamma = 0,1$ khởi đầu ngay tại vùng entropy từ đơn (7,21) – tiên nghiệm tần suất được bảo toàn làm điểm xuất phát – rồi hội tụ về 3,14, bỏ xa hai phương án còn lại trong suốt quá trình. Do đó, $\gamma = 0,1$ được chọn cho mọi thí nghiệm về sau.

4.4.3 Lựa chọn chiến lược khởi tạo hai ma trận từ vựng

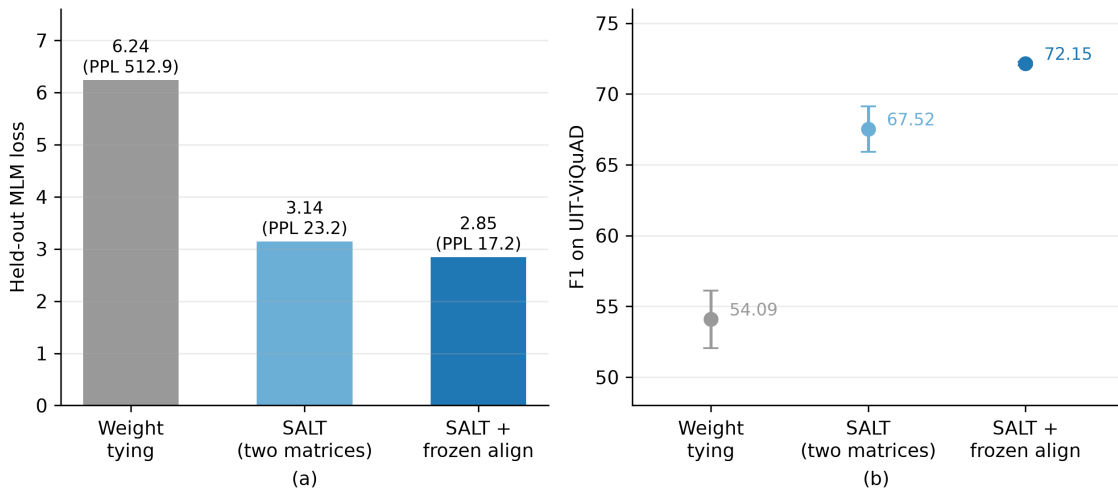
Thí nghiệm thứ hai so sánh ba chiến lược khởi tạo hai ma trận từ vựng, tương ứng ba mức đầu tư tăng dần:

- **Buộc trọng số:** lớp đầu ra sao chép nguyên ma trận nhúng tại khởi tạo – thói quen kế thừa từ các kiến trúc buộc trọng số, bỏ qua việc

NeoBERT dùng đầu giải mã tách rời;

- **SALT hai ma trận:** chiếu theo token *riêng* cho lớp nhúng và lớp đầu ra với $\gamma = 0,1$ (Mục 3.5.3);
- **SALT + căn chỉnh đóng băng:** như trên, thêm giai đoạn huấn luyện phụ trợ ~ 20 triệu token chỉ cho hai ma trận trên phần thân đóng băng trước khi CPT (Mục 3.6).

Cả ba được tiếp tục tiền huấn luyện tới cùng mốc ~ 100 triệu token, đo mất mát trên tập đánh giá, rồi tinh chỉnh trên UIT-ViQuAD với cấu hình đồng nhất qua ba hạt giống. Hình 4.3 tổng hợp cả hai mặt so sánh.



Hình 4.3: So sánh ba chiến lược khởi tạo.

Chiến lược buộc trọng số thất bại rõ rệt ở cả hai mặt: mất mát 6,24 (perplexity ≈ 513 , Hình 4.3a) và chỉ $54,09 \pm 2,04$ điểm F1 (Hình 4.3b). Nguyên nhân đã được phân tích ở Mục 2.3.1: lớp nhúng và lớp đầu ra đảm nhận hai vai trò hình học khác nhau, nên ép chúng dùng chung một ma trận tại khởi tạo khiến cả hai nhiệm vụ cùng bị tổn hại – phương án này khởi đầu ở mức mất mát 21,67, tệ nhất trong mọi thí nghiệm của đề tài. Khởi tạo SALT riêng cho từng ma trận khắc phục triệt để: mất mát giảm còn 3,14 (perplexity 23,2) và F1 tăng vọt lên $67,52 \pm 1,61$ – hơn chiến lược buộc trọng số 13,4 điểm.

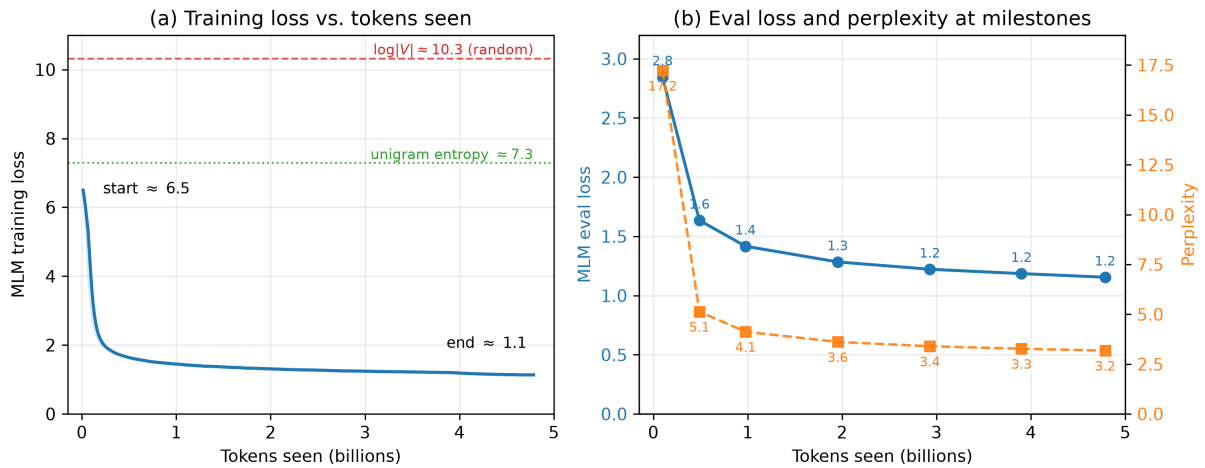
Bổ sung giai đoạn căn chỉnh đóng băng cải thiện thêm một nấc rõ rệt ở cả hai mặt: mất mát còn 2,85 (perplexity 17,2) – tương đương giảm khoảng 26% perplexity so với không căn chỉnh – nhờ hai ma trận đã kịp “khớp” với nhau và với phần thân trước khi bước vào CPT; và ở hạ nguồn, phương án này đạt $72,15 \pm 0,12$ điểm F1 tại cùng mốc ngân sách – cao hơn 4,6 điểm so với không căn chỉnh, đồng thời độ lệch chuẩn giữa các hạt giống giảm hơn một bậc (0,12 so với 1,61), tức mô hình vừa tốt hơn vừa ổn định hơn khi tinh chỉnh. Chi phí đổi lại chỉ là ~ 20 triệu token huấn luyện hai ma trận (Bảng 4.4). Do đó, *SALT kết hợp căn chỉnh đóng băng* được chọn làm mô hình chính để huấn luyện với toàn bộ ngân sách.

4.5 Kết quả của mô hình cuối

Mô hình chính được tiếp tục tiền huấn luyện tới khoảng 5 tỷ token CulturaX theo lịch WSD. Chúng tôi phân tích kết quả theo ba góc nhìn: diễn tiến chất lượng theo ngân sách huấn luyện (Mục 4.5.1), bức tranh đa nhiệm tại mốc cuối (Mục 4.5.2), và khả năng xử lý ngữ cảnh dài sau pha mở rộng lên 4.096 token (Mục 4.5.3). Đối sánh là hai họ mô hình phổ biến nhất với tiếng Việt – PhoBERT [6], đơn ngữ, huấn luyện trên dữ liệu đã tách từ, và XLM-R [55], đa ngữ – ở cả hai cỡ **base** và **large**.

4.5.1 Diễn tiến theo ngân sách huấn luyện

Trước khi xét chất lượng hạ nguồn, Hình 4.4 cho thấy diễn tiến của chính quá trình tiếp tục tiền huấn luyện: hình (a) vẽ mất mát huấn luyện theo số token đã duyệt, kèm hai mức tham chiếu $\log |V|$ (dự đoán ngẫu nhiên) và entropy tự đơn tiếng Việt; hình (b) vẽ mất mát và perplexity đánh giá tại các mốc. Nhờ khởi tạo SALT, mất mát MLM *khởi đầu* chỉ khoảng 6,5 – đã nằm sâu dưới mức dự đoán ngẫu nhiên đều $\log |V| \approx 10,3$ và gần với entropy tự đơn tiếng Việt ($\approx 7,3$) – thay vì bắt đầu từ trạng thái nhiễu loạn như khởi tạo ngẫu nhiên. Phần lớn mức giảm diễn ra rất

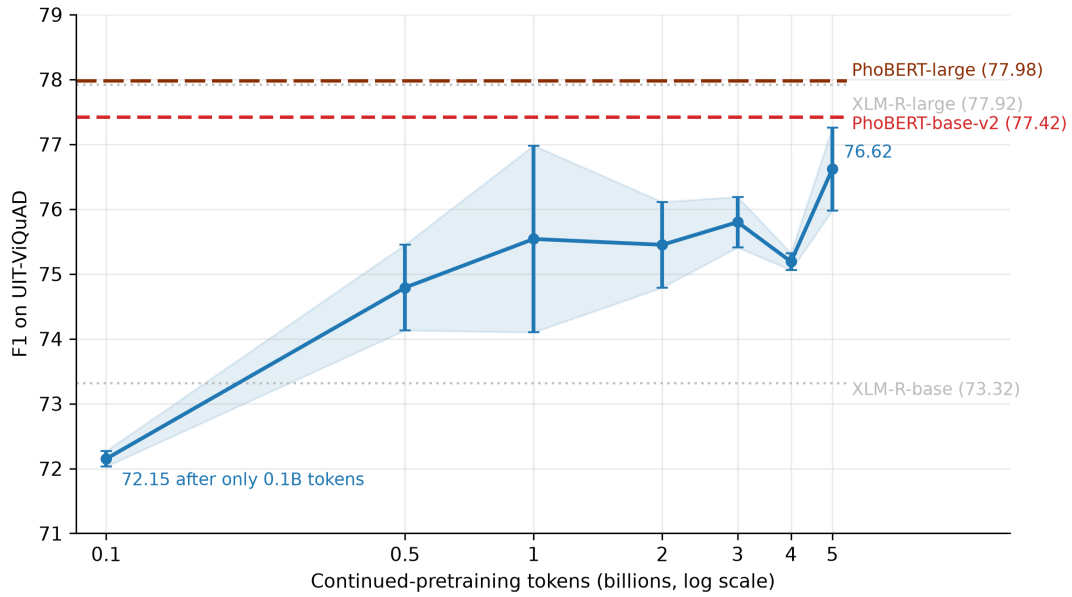


Hình 4.4: Đường học giai đoạn CPT (WSD).

sớm: chỉ trong khoảng 0,2 tỷ token đầu, mất mát đã rơi xuống dưới 2,0. Sau đó tốc độ học được giữ ở đỉnh trong phần lớn ngân sách (pha ổn định của lịch WSD, khoảng 4 tỷ token đầu) và mất mát tiếp tục hạ chậm; pha làm nguội trên khoảng 1 tỷ token cuối – xấp xỉ 20% ngân sách (Mục 3.7) – kéo mất mát về $\approx 1,1$. Mất mát đánh giá trên tập giữ lại biến thiên đồng pha: perplexity giảm từ 17,2 ở khoảng 0,1 tỷ token xuống 3,3 ở khoảng 3,9 tỷ token và 3,2 ở điểm kiểm tra cuối. Diễn tiến này xác nhận ở cấp độ tiền huấn luyện điều mà đường cong hạ nguồn dưới đây thể hiện ở cấp độ tác vụ: phần thân kế thừa hội tụ nhanh trên tiếng Việt thay vì phải học lại từ đầu.

Tại mỗi mốc ngân sách, chúng tôi rẽ một nhánh làm nguội ngắn để lấy điểm kiểm tra rồi tinh chỉnh trên UIT-ViQuAD; Hình 4.5 thể hiện điểm F1 theo lượng token đã tiếp tục tiền huấn luyện (thang log). Đối sánh chính là họ PhoBERT (hai đường đứt nét đậm) – cùng là mô hình đơn ngữ chuyên cho tiếng Việt nên là mốc tham chiếu trực tiếp của đề tài; hai đường chấm xám của XLM-R chỉ mang tính tham khảo, vì đây là mô hình đa ngữ với tiếng Việt chỉ là một trong một trăm ngôn ngữ.

Điểm nổi bật nhất của đường cong là *tốc độ*: ngay tại mốc đầu tiên – 0,1 tỷ token, tức chỉ 2% ngân sách – ViNeoBERT đã đạt 72,15 điểm F1,



Hình 4.5: Điểm F1 trên UIT-ViQuAD theo số token CPT.

cao hơn mọi chiến lược khởi tạo đối chứng ở Mục 4.4.3 và chỉ còn cách kết quả cuối 4,5 điểm; sang mốc 0,5 tỷ token, mô hình đã vượt kết quả *cuối cùng* của XLM-R-base (74,79 so với 73,32). Đây chính là giá trị thực tiễn của khởi tạo SALT: phần thân kế thừa không phải “học lại tiếng Việt từ đầu”, nên phần lớn chất lượng đến ngay từ những trăm triệu token đầu tiên. Sau đó đường cong chững quanh vùng 1–4 tỷ token trước khi bật lên 76,62 ở mốc 5 tỷ – mốc được áp dụng pha làm nguội trên dữ liệu mới (Mục 3.7) – thu hẹp khoảng cách với PhoBERT-base-v2 còn 0,8 điểm và với PhoBERT-large còn 1,4 điểm, với xu hướng vẫn đang đi lên.

Khoảng cách còn lại đó cần được đặt cạnh lượng dữ liệu mà mỗi mô hình đã tiêu thụ, tóm tắt trong Bảng 4.5. PhoBERT huấn luyện 40 epoch trên kho 20GB (~3,5 tỷ token mỗi lượt): bản base chạy 540 nghìn bước với batch 1.024 chuỗi 256 token, bản large 1,08 triệu bước với batch 512 – cả hai đều duyệt xấp xỉ *140 tỷ token* [6]; bản base-v2 còn được huấn luyện thêm trên 120GB văn bản OSCAR-2301 (kho ~25 tỷ token mỗi lượt, số bước không được công bố). XLM-R huấn luyện 1,5 triệu bước với batch 8.192 chuỗi 512 token – tổng cỡ *6,3 nghìn tỷ token* đa ngữ; riêng phần

tiếng Việt trong CC-100 đã là 24,76 tỷ token (137,3 GiB), và với tỷ lệ lấy mẫu $\alpha = 0,3$ lượng tiếng Việt mô hình này đã duyệt ước tính ở mức hàng trăm tỷ token [55]. Trong khi đó, ViNeoBERT duyệt đúng *một lượt* qua 5 tỷ token – tức chỉ khoảng **3,5%** lượng token của họ PhoBERT và cỡ vài phần trăm lượng tiếng Việt của XLM-R – nhưng đạt 99,0% điểm F1 của PhoBERT-base-v2 và 98,3% của PhoBERT-large trên tác vụ này.

Bảng 4.5: Lượng dữ liệu tiếng Việt đã duyệt.

Mô hình	Kho ngữ liệu	Token đã duyệt
PhoBERT (base/large)	20GB ($\sim 3,5$ tỷ token)	~ 140 tỷ (40 epoch)
PhoBERT-base-v2	140GB (~ 25 tỷ token)	không công bố ($\geq$ bản gốc)
XLM-R (base/large)	CC-100 2,5TB; phần vi 24,76 tỷ token	~ 6.300 tỷ (đa ngữ); vi ước hàng trăm tỷ
ViNeoBERT	20GB (~ 5 tỷ token)	5 tỷ (một lượt)

4.5.2 Đánh giá đa nhiệm tại mốc 5 tỷ token

Để có bức tranh toàn diện hơn một tác vụ đơn lẻ, mô hình cuối được đánh giá trên mười hai tác vụ thuộc năm nhóm năng lực (Mục 4.1.3), so với bốn mô hình đối sánh: hai bản **base** cùng hạng tham số và hai bản **large** lớn hơn đáng kể (PhoBERT-large khoảng 370 triệu, XLM-R-large khoảng 560 triệu tham số, so với ≈ 245 triệu của ViNeoBERT). Kết quả trình bày trong Bảng 4.6: mỗi ô là giá trị trung bình qua 5 lần chạy với hạt giống khác nhau, riêng hai tác vụ Retrieval và Reranking chạy một lần (Mục 4.3).

Bảng 4.6: Kết quả đánh giá đa nhiệm tại mốc 5 tỷ token (giá trị tốt nhất in đậm).

Tác vụ	Độ đo	PhoBERT-base-v2	PhoBERT-large	XLM-R-base	XLM-R-large	ViNeoBERT
UIT-VSFC	F1-weighted ($\uparrow$)	0,9345	0,9356	0,9281	0,9312	0,9284
UIT-VSMEC	F1-macro ($\uparrow$)	0,6415	0,6132	0,6145	0,3125	0,6163
UIT-ViCTSD	F1-macro ($\uparrow$)	0,7455	0,7526	0,7110	0,5759	0,7315
ViCoLA-syn	MCC ($\uparrow$)	0,8103	0,7915	0,7827	0,4845	0,7789
XNLI (vi)	Accuracy ($\uparrow$)	0,7720	0,7769	0,7541	0,8000	0,7714
ViANLI	F1-macro ($\uparrow$)	0,4529	0,3882	0,3625	0,1666	0,4273
QNLI (vi)	Accuracy ($\uparrow$)	0,8872	0,8861	0,8592	0,8940	0,8791
UD-VTB (POS)	F1-macro ($\uparrow$)	0,7799	0,7879	0,7827	0,8062	0,8089
STS (vi)	Spearman ($\uparrow$)	0,8557	0,8509	0,8226	0,8644	0,8392
Paraphrase (STS)	F1-macro ($\uparrow$)	0,9351	0,9400	0,9016	0,9437	0,9276
Retrieval (ViQuAD)	nDCG@10 ($\uparrow$)	0,4719	0,4659	0,1386	0,2565	0,4061
Reranking (ViQuAD)	MRR ($\uparrow$)	0,8922	0,8794	0,5282	0,6741	0,8448

Từ Bảng 4.6, chúng tôi rút ra bốn nhận xét:

- **So sánh với PhoBERT:** PhoBERT là đối sánh chính – mô hình chuyên biệt tiếng Việt được tiền huấn luyện quy mô lớn và là chuẩn hiện tại của lĩnh vực. So với PhoBERT-base-v2 cùng hạng tham số, ViNeoBERT thắng ở gán nhãn từ loại (0,8089 so với 0,7799), Retrieval (0,4061 so với 0,4719 – thua nhưng bỏ xa XLM-R-base 0,1386), và Reranking (0,8448 so với 0,8922), trong khi ngang ngửa ở VSFC (+0,0061 cho PhoBERT), XNLI (+0,0006), QNLI (+0,0081), và thua rõ hơn ở VSMEC, ViCoLA-syn, ViANLI. PhoBERT-large – lớn gấp 1,5 lần – dẫn đầu ở VSFC, ViCTSD và các tác vụ paraphrase/STS, song ViNeoBERT đã vượt qua ở gán nhãn từ loại.
- **So sánh với XLM-R:** XLM-R là đối sánh thứ hai – mô hình đa ngữ không chuyên biệt tiếng Việt. So với XLM-R-base cùng hạng tham số, ViNeoBERT thắng ở 11/12 tác vụ bảng này, và 12/13 nếu tính đọc hiểu ở Mục 4.5.1. XLM-R-large tuy dẫn một số tác vụ suy luận và tương đồng ngữ nghĩa nhưng thể hiện *bất ổn tinh chỉnh nghiêm trọng* trên các bộ dữ liệu nhỏ (phương sai rất cao ở VSMEC, ViCoLA-syn, ViCTSD) và sụp đổ hoàn toàn ở ViANLI – hiện tượng đã biết của các mô hình lớn trên dữ liệu ít. ViNeoBERT ổn định trên toàn bộ 12 tác vụ.
- **Hiệu quả dữ liệu – điểm mấu chốt:** lượng tiếng Việt mà ViNeoBERT tiếp xúc chỉ khoảng 5 tỷ token ($\sim 20\text{GB}$) duyệt đúng một lượt. PhoBERT được huấn luyện 40 epoch trên 20GB (Wikipedia + tin tức), tức lượng token đã duyệt lớn hơn ViNeoBERT khoảng 30 lần [6]; bản base-v2 huấn luyện bổ sung trên 120GB văn bản OSCAR-2301. XLM-R sử dụng 2,5TB dữ liệu CC-100, riêng phần tiếng Việt $\approx 137\text{GB}$ [55] – gấp nhiều lần toàn bộ ngân sách của đề tài. Kết quả cạnh tranh sòng phẳng với các bản **large** khi chỉ “đọc” ít hơn một đến hai bậc xác nhận trực tiếp luận điểm của đề tài: khởi

tạo có thông tin cho phép kế thừa năng lực phần thân đã huấn luyện thay vì học lại từ đầu.

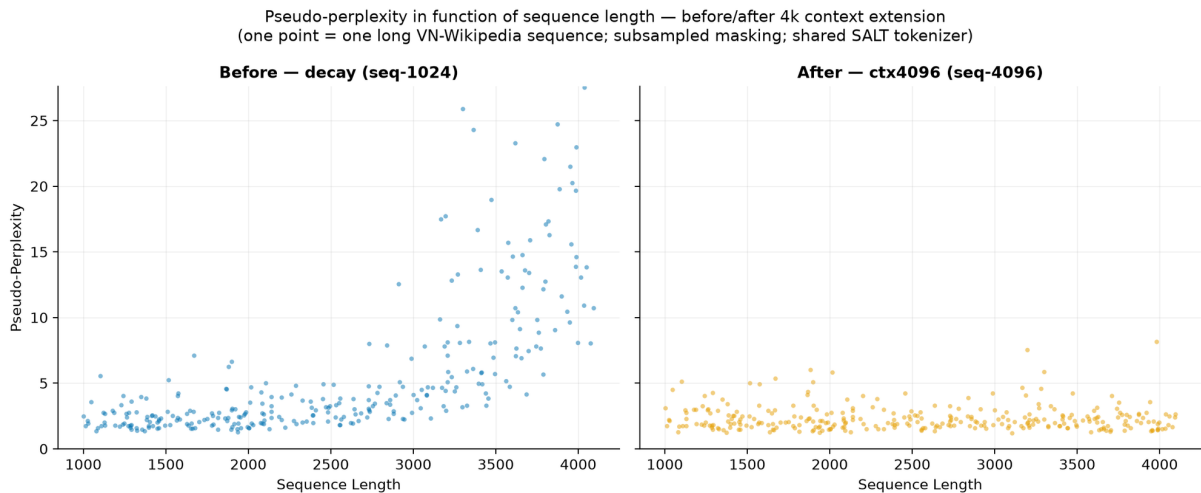
- **Phân bố điểm mạnh:** các tác vụ ViNeoBERT mạnh nhất (đọc hiểu span dài, gán nhãn từ loại, Retrieval/Reranking) gắn với chất lượng *biểu diễn theo ngữ cảnh ở cấp token* – đúng năng lực được kế thừa từ phần thân NeoBERT sâu 28 lớp. PhoBERT giữ ưu thế ở nhóm phân loại câu ngắn và tương đồng ngữ nghĩa – vốn hưởng lợi từ bộ tách từ tiếng Việt và dữ liệu trong miền phong phú hơn.

4.5.3 Mở rộng ngữ cảnh lên 4.096 token

NeoBERT được tiền huấn luyện theo hai pha: pha chính ở độ dài 1.024 token, sau đó một pha ngắn ở độ dài 4.096 token để kích hoạt toàn bộ trần ngữ cảnh [5]. ViNeoBERT kế thừa kiến trúc hỗ trợ 4.096 token (tần số RoPE được tính sẵn tới độ dài này), nhưng toàn bộ quá trình CPT dùng chuỗi 1.024 token – vì vậy chúng tôi tái hiện pha thứ hai tương tự: từ điểm kiểm tra đã làm nguội ở mốc 5 tỷ token, mô hình được huấn luyện tiếp khoảng 200 triệu token ($\sim 4\%$ ngân sách CPT) ở độ dài 4.096, trên phần dữ liệu CulturaX *chưa từng thấy* (bốn đoạn 1.024 liên tiếp được ghép thành một chuỗi 4.096), với tốc độ học hằng số 10^{-5} – một phần mười đỉnh CPT (cột “Mở rộng ngữ cảnh” trong Bảng 4.4).

Khả năng ngữ cảnh dài được kiểm chứng theo đúng giao thức của NeoBERT: *pseudo-perplexity* (PPPL) [56] theo độ dài chuỗi. Với mô hình MLM, PPPL của một chuỗi được ước lượng bằng cách che lần lượt từng vị trí và lấy tích xác suất khôi phục; PPPL càng thấp và càng *ổn định theo độ dài* thì mô hình khai thác ngữ cảnh xa càng tốt. Trong Hình 4.6, mỗi điểm là một văn bản dài (1.000–4.096 token) lấy từ Wikipedia tiếng Việt; PPPL mỗi điểm được xấp xỉ bằng lấy mẫu 32 vị trí che (subsampling masking), và *cùng* tập cặp (văn bản, độ dài) được dùng cho cả hai điểm kiểm tra nên hai hình so sánh được trực tiếp.

Kết quả tái hiện đúng hiện tượng mà NeoBERT báo cáo. Trước pha



Hình 4.6: Pseudo-perplexity theo độ dài chuỗi, trước và sau pha mở rộng 4.096 token.

mở rộng (hình trái, “Before – decay (seq-1024)”): nhờ tính chất tương đối của RoPE, mô hình ngoại suy được một phần ra ngoài độ dài huấn luyện – PPPL giữ ổn định quanh 2–3 tới khoảng 2.500–3.000 token – nhưng vượt ngưỡng đó chất lượng sụp đổ nhanh, nhiều điểm vượt 10 và có điểm lên tới trên 25: ngữ cảnh xa không những không giúp mà còn gây nhiễu đoán. Sau pha mở rộng (hình phải, “After – ctx4096 (seq-4096)”): đám mây điểm phẳng quanh 2–3 trên *toàn dải* tới 4.096 token, không còn xu hướng tăng theo độ dài. Nói cách khác, chỉ với ~ 200 triệu token huấn luyện bổ sung, trần ngữ cảnh 4.096 token kế thừa từ kiến trúc đã trở thành năng lực sử dụng được trong thực tế.

4.6 Thảo luận

Kết quả thực nghiệm khép vòng với các mục tiêu đặt ra ở Chương 1. Thí nghiệm so sánh khởi tạo (Mục 4.4) chứng minh từng lựa chọn thiết kế của phương pháp đều đóng góp dương: phép chiếu SALT đưa từ vựng tiếng Việt về đúng vị trí và thang độ lớn của không gian NeoBERT (khoảng cách tâm giảm từ 0,8 còn 0,3); hệ số co $\gamma = 0,1$ là cần thiết – không co hoặc

co không đủ khiến huấn luyện mắc kẹt; khởi tạo SALT riêng cho hai ma trận vượt xa cách buộc trọng số mặc định (67,52 so với 54,09 điểm F1); và căn chỉnh đóng băng giảm khoảng 26% perplexity, nâng F1 lên 72,15 với chi phí không đáng kể. Đánh giá theo ngân sách (Mục 4.5) cho thấy phương pháp đạt hiệu quả dữ liệu cao: vượt XLM-R-base chỉ với khoảng 0,5 tỷ token; ở mốc 5 tỷ token, mô hình cao hơn XLM-R-base ở 11/12 tác vụ đa nhiệm, dẫn đầu cả bảng ở gán nhãn từ loại và giữ độ ổn định tinh chỉnh tốt hơn hẳn các bản **large**, dù lượng tiếng Việt tiếp xúc ít hơn các mô hình đối sánh một đến hai bậc.

Thực nghiệm cũng bộc lộ một số giới hạn – ngân sách 5 tỷ token còn nhỏ so với các mô hình đối sánh, một vài tác vụ phân loại đã bão hòa, và hai tác vụ biểu diễn câu mới chạy một lần nên chưa ước lượng được phương sai – được thảo luận đầy đủ cùng các hướng khắc phục ở Chương 5.

Chương 5

Kết luận và Hướng phát triển

5.1 Kết luận

Khóa luận đã nghiên cứu và xây dựng thành công **ViNeoBERT** – một mô hình Encoder hiện đại cho tiếng Việt – bằng cách thích nghi NeoBERT thông qua kỹ thuật SALT được cải tiến, thay vì huấn luyện lại từ đầu. Đối chiếu với các mục tiêu đặt ra ở Chương 1, đề tài đã đạt được các kết quả sau:

- **Về nghiên cứu kiến trúc:** Chúng tôi đã phân tích các cải tiến kiến trúc của NeoBERT (RoPE, SwiGLU, Pre-RMSNorm, tỷ lệ sâu–hẹp tối ưu) và xác định đúng hai điểm mấu chốt ảnh hưởng tới việc chuyển giao: chuẩn hóa Pre-RMSNorm nhảy với độ lớn của nhúng – dẫn tới bước hiệu chỉnh chuẩn – và đầu giải mã tách rời đòi hỏi khởi tạo riêng cho lớp đầu ra (Chương 2, 3).
- **Về phương pháp chuyển giao:** Chúng tôi đã triển khai thuật toán SALT và cải tiến nó cho cặp ngôn ngữ Anh–Việt với ba đóng góp: (1) mở rộng tập neo bằng cặp dịch khai thác tự động, kèm cơ chế lọc từ đồng tự khác nghĩa đặc thù cho ngôn ngữ hệ chữ Latin; (2) áp dụng phép chiếu theo từng token cho cả lớp đầu ra tách rời, kết hợp khởi tạo độ chệch theo tần suất từ đơn và hệ số co trọng số $\gamma = 0,1$;

(3) bổ sung giai đoạn căn chỉnh đóng băng trước khi tiếp tục tiền huấn luyện (Chương 3).

- **Về kiểm chứng thực nghiệm:** Thí nghiệm so sánh có kiểm soát các phương án khởi tạo ở cùng ngân sách (Mục 4.4) xác nhận từng lựa chọn thiết kế: hệ số co $\gamma = 0,1$ là cần thiết – không co hoặc co không đủ khiến huấn luyện mắc kẹt ở mức mất mát cao; khởi tạo SALT riêng cho hai ma trận vượt xa cách buộc trọng số mặc định (67,52 so với 54,09 điểm F1 trên UIT-ViQuAD sau 100 triệu token); và căn chỉnh đóng băng giảm khoảng 26% perplexity, nâng F1 lên 72,15 với chi phí phụ trợ không đáng kể.
- **Về hiệu quả của mô hình:** Sau khoảng 5 tỷ token CulturaX (~20GB văn bản), ViNeoBERT đạt 76,62 điểm F1 trên đọc hiểu UIT-ViQuAD (hơn XLM-R-base 3,3 điểm), cao hơn XLM-R-base ở 11/12 tác vụ của bảng đánh giá đa nhiệm, dẫn đầu cả năm mô hình so sánh ở gán nhãn từ loại và cạnh tranh sòng phẳng với các bản **large** có gấp 1,5–2,3 lần tham số (Mục 4.5). Pha mở rộng ngữ cảnh ~200 triệu token ở độ dài 4.096 giữ pseudo-perplexity ổn định trên toàn dải ngữ cảnh, biến trần 4.096 token kế thừa từ kiến trúc thành năng lực sử dụng được. Đáng nói nhất, lượng tiếng Việt mô hình tiếp xúc chỉ khoảng 5 tỷ token – ít hơn một đến hai bậc so với PhoBERT (40 epoch trên 20GB, bản v2 thêm 120GB OSCAR) hay XLM-R (~137GB tiếng Việt trong CC-100). Kết quả này xác nhận giả thuyết trung tâm của đề tài: *tái sử dụng không gian biểu diễn sẵn có là con đường tiết kiệm để đưa các kiến trúc Encoder thế hệ mới đến với tiếng Việt.*

Bên cạnh mô hình, đề tài đóng góp một quy trình ba giai đoạn có thể tái lập – khởi tạo SALT cải tiến, căn chỉnh đóng băng, tiếp tục tiền huấn luyện theo lịch WSD – cùng bộ mã nguồn công khai đi kèm.

5.2 Hạn chế của đề tài

Trong khuôn khổ thời gian và tài nguyên của một khóa luận, đề tài còn các hạn chế sau:

- **Ngân sách huấn luyện:** khoảng 5 tỷ token ($\sim 20\text{GB}$ văn bản) vẫn nhỏ so với dữ liệu tiền huấn luyện của các mô hình đối sánh; ViNeoBERT chưa vượt PhoBERT-base-v2 ở nhóm tác vụ phân loại câu ngắn, dù xu hướng đường cong học cho thấy khoảng cách có thể tiếp tục thu hẹp khi tăng ngân sách.
- **Phạm vi đánh giá:** bộ đánh giá tập trung vào các tác vụ hiểu và biểu diễn câu; một số năng lực như nhận dạng thực thể có tên hay ứng dụng ngữ cảnh dài chưa được khảo sát, và hai tác vụ truy hỏi–xếp hạng lại mới chạy một lần nên chưa ước lượng được phương sai.
- **Ngữ cảnh dài:** pha mở rộng ngữ cảnh lên 4.096 token mới được kiểm chứng bằng độ đo nội tại (pseudo-perplexity theo độ dài chuỗi, Mục 4.5.3); chưa có đánh giá hạ nguồn trên các tác vụ đòi hỏi ngữ cảnh dài như hỏi đáp đa đoạn hay truy hỏi tài liệu.
- **Bộ tách từ:** từ vựng kế thừa từ ViDeBERTa hoạt động ở mức âm tiết; ảnh hưởng của việc không tách từ (so với PhoBERT dùng dữ liệu đã tách từ) chưa được phân tích riêng.

5.3 Hướng phát triển

Từ các kết quả và hạn chế trên, chúng tôi đề xuất năm hướng phát triển:

1. **Mở rộng ngân sách tiếp tục tiền huấn luyện:** tăng lượng token CulturaX vượt mốc 5 tỷ và khai thác thêm các pha làm nguội trên

dữ liệu mới – vốn đã cho thấy hiệu quả bật điểm ở mốc cuối – nhằm vượt các mô hình họ PhoBERT trên các tác vụ hạ nguồn.

2. **Trộn lại một phần dữ liệu tiếng Anh khi tiếp tục tiền huấn luyện:** giữ lại một tỷ lệ nhỏ (khoảng 5–10%) dữ liệu tiếng Anh – ngôn ngữ nguồn của NeoBERT – trong hỗn hợp huấn luyện ở Giai đoạn 3 như một cơ chế *phát lại* (replay), nhằm bảo toàn biểu diễn ở các tầng trong và giảm quên lãng thảm họa. Ibrahim và cộng sự [30] cho thấy chỉ cần phát lại một phần nhỏ dữ liệu cũ là đủ để chống suy sụp khi phân phối dữ liệu dịch chuyển; đáng chú ý hơn, Elhady và cộng sự [29] chỉ ra rằng trong thích nghi ngôn ngữ, việc giữ lại dữ liệu tiếng Anh tuy *không* cải thiện perplexity nhưng lại quyết định sự xuất hiện của năng lực hạ nguồn – một tín hiệu mà mất mát MLM đơn thuần không phản ánh. Đề tài dùng khởi tạo SALT và giai đoạn căn chỉnh đóng băng làm lớp bảo vệ chống quên lãng, còn phát lại tiếng Anh là một cơ chế hỗ trợ tự nhiên đáng để khảo sát tiếp.
3. **Khai thác ngữ cảnh dài ở hạ nguồn:** xây dựng và đánh giá trên các tác vụ đòi hỏi ngữ cảnh dài (hỏi đáp đa đoạn, truy hồi tài liệu) để bổ sung cho kiểm chứng nội tại bằng pseudo-perplexity ở Mục 4.5.3; khi cần vượt trần 4.096 token, các kỹ thuật nội suy RoPE như YaRN [14] là ứng viên tự nhiên.
4. **Mở rộng bộ đánh giá:** bổ sung nhận dạng thực thể có tên, truy hồi trên kho ngữ liệu quy mô lớn hơn và lặp lại nhóm tác vụ biểu diễn câu qua nhiều lần chạy, nhằm đánh giá toàn diện và tin cậy hơn chất lượng biểu diễn của mô hình ở vai trò một Encoder đa dụng.
5. **Tổng quát hóa quy trình:** áp dụng quy trình ba giai đoạn của đề tài cho các ngôn ngữ ít tài nguyên khác hoặc các kiến trúc nền khác, qua đó kiểm chứng tính tổng quát của các cải tiến (tập neo theo cặp dịch, khởi tạo lớp đầu ra tách rời, căn chỉnh đóng băng) ngoài phạm vi cặp Anh–Việt và NeoBERT.

Tài liệu tham khảo

- [1] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proceedings of NAACL-HLT*, 2019, pp. 4171–4186.
- [2] Y. Liu et al., “RoBERTa: A robustly optimized BERT pretraining approach,” *arXiv preprint arXiv:1907.11692*, 2019.
- [3] A. Dubey et al., “The Llama 3 herd of models,” *arXiv preprint arXiv:2407.21783*, 2024.
- [4] DeepSeek-AI, “DeepSeek-V3 technical report,” *arXiv preprint arXiv:2412.19437*, 2024.
- [5] L. Le Breton, Q. Fournier, M. El Mezouar, J. X. Morris, and S. Chandar, “NeoBERT: A next-generation BERT,” *arXiv preprint arXiv:2502.19587*, 2025.
- [6] D. Q. Nguyen and A. T. Nguyen, “PhoBERT: Pre-trained language models for Vietnamese,” in *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2020, pp. 1037–1042.
- [7] C. D. Tran, N. H. Pham, A. Nguyen, T. S. Hy, and T. Vu, “ViDeBERTa: A powerful pre-trained language model for Vietnamese,” in *Findings of the Association for Computational Linguistics: EACL 2023*, 2023, pp. 1071–1078.
- [8] S. Lee, S. Hong, H. Moon, and H. Lim, “Semantic aware linear transfer by recycling pre-trained language models for cross-lingual trans-

- fer,” *Findings of the Association for Computational Linguistics: ACL 2025*, 2025, arXiv:2505.10945.
- [9] A. Vaswani et al., “Attention is all you need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
 - [10] Z. Nussbaum, J. X. Morris, B. Duderstadt, and A. Mulyar, “Nomic embed: Training a reproducible long context text embedder,” *arXiv preprint arXiv:2402.01613*, 2024.
 - [11] B. Warner et al., “Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference,” *arXiv preprint arXiv:2412.13663*, 2024.
 - [12] Y. Levine, N. Wies, O. Sharir, H. Bata, and A. Shashua, “The depth-to-width interplay in self-attention,” in *Advances in Neural Information Processing Systems (NeurIPS)*, arXiv:2006.12467, 2020.
 - [13] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, “RoFormer: Enhanced transformer with rotary position embedding,” *Neurocomputing*, vol. 568, 2024, arXiv:2104.09864.
 - [14] B. Peng, J. Quesnelle, H. Fan, and E. Shippole, “YaRN: Efficient context window extension of large language models,” *arXiv preprint arXiv:2309.00071*, 2023.
 - [15] D. Hendrycks and K. Gimpel, “Gaussian error linear units (GELUs),” *arXiv preprint arXiv:1606.08415*, 2016.
 - [16] N. Shazeer, “GLU variants improve transformer,” *arXiv preprint arXiv:2002.05202*, 2020.
 - [17] B. Zhang and R. Sennrich, “Root mean square layer normalization,” in *Advances in Neural Information Processing Systems (NeurIPS)*, arXiv:1910.07467, 2019.

- [18] R. Xiong et al., “On layer normalization in the transformer architecture,” in *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020.
- [19] G. Penedo et al., “The RefinedWeb dataset for Falcon LLM: Outperforming curated corpora with web data, and web data only,” *arXiv preprint arXiv:2306.01116*, 2023.
- [20] A. Wettig, T. Gao, Z. Zhong, and D. Chen, “Should you mask 15% in masked language modeling?” In *Proceedings of the 17th Conference of the European Chapter of the ACL (EACL)*, 2023.
- [21] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *International Conference on Learning Representations (ICLR)*, arXiv:1711.05101, 2019.
- [22] I. Loshchilov and F. Hutter, “SGDR: Stochastic gradient descent with warm restarts,” in *International Conference on Learning Representations (ICLR)*, arXiv:1608.03983, 2017.
- [23] T. Dao, “FlashAttention-2: Faster attention with better parallelism and work partitioning,” in *International Conference on Learning Representations (ICLR)*, arXiv:2307.08691, 2024.
- [24] K. Ethayarajh, “How contextual are contextualized word representations? comparing the geometry of BERT, ELMo, and GPT-2 embeddings,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, 2019.
- [25] A. Ormazabal, M. Artetxe, G. Labaka, A. Soroa, and E. Agirre, “Analyzing the limitations of cross-lingual word embedding mappings,” in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2019, pp. 4990–4995.
- [26] J. Gao, D. He, X. Tan, T. Qin, L. Wang, and T.-Y. Liu, “Representation degeneration problem in training natural language generation

- models,” in *International Conference on Learning Representations (ICLR)*, 2019.
- [27] S. Gururangan et al., “Don’t stop pretraining: Adapt language models to domains and tasks,” in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020.
 - [28] W. de Vries and M. Nissim, “As good as new. how to successfully recycle English GPT-2 to make models for other languages,” in *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, 2021.
 - [29] A. Elhady, E. Agirre, and M. Artetxe, “Emergent abilities of large language models under continued pretraining for language adaptation,” in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, arXiv:2506.00288, 2025.
 - [30] A. Ibrahim et al., “Simple and scalable strategies to continually pre-train large language models,” *Transactions on Machine Learning Research (TMLR)*, 2024, arXiv:2403.08763.
 - [31] S. Hu, Y. Tu, X. Han, C. He, G. Cui, X. Long, et al., “MiniCPM: Unveiling the potential of small language models with scalable training strategies,” *arXiv preprint arXiv:2404.06395*, 2024.
 - [32] A. Hägele, E. Bakouch, A. Kosson, L. Ben Allal, L. Von Werra, and M. Jaggi, “Scaling laws and compute-optimal training beyond fixed training durations,” in *Advances in Neural Information Processing Systems (NeurIPS)*, arXiv:2405.18392, 2024.
 - [33] B. Minixhofer, F. Paischer, and N. Rekabsaz, “WECHSEL: Effective initialization of subword embeddings for cross-lingual transfer of monolingual language models,” in *Proceedings of NAACL-HLT*, 2022, pp. 3992–4006.

- [34] K. Dobler and G. de Melo, “FOCUS: Effective embedding initialization for monolingual specialization of multilingual models,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.
- [35] Y. Liu, P. Lin, M. Wang, and H. Schütze, “OFA: A framework of initializing unseen subword embeddings for efficient large-scale multilingual continued pretraining,” in *Findings of the Association for Computational Linguistics: NAACL 2024*, 2024.
- [36] A. F. T. Martins and R. F. Astudillo, “From softmax to sparsemax: A sparse model of attention and multi-label classification,” in *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, 2016.
- [37] T. Vu, D. Q. Nguyen, D. Q. Nguyen, M. Dras, and M. Johnson, “VnCoreNLP: A Vietnamese natural language processing toolkit,” in *Proceedings of NAACL-HLT 2018: Demonstrations*, 2018, pp. 56–60.
- [38] P. He, J. Gao, and W. Chen, “DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training with gradient-disentangled embedding sharing,” in *International Conference on Learning Representations (ICLR)*, arXiv:2111.09543, 2023.
- [39] M. Junczys-Dowmunt et al., “Marian: Fast neural machine translation in C++,” in *Proceedings of ACL 2018, System Demonstrations*, 2018.
- [40] P. Bojanowski, E. Grave, A. Joulin, and T. Mikolov, “Enriching word vectors with subword information,” *Transactions of the Association for Computational Linguistics (TACL)*, vol. 5, pp. 135–146, 2017.
- [41] E. Grave, P. Bojanowski, P. Gupta, A. Joulin, and T. Mikolov, “Learning word vectors for 157 languages,” in *Proceedings of the 11th International Conference on Language Resources and Evaluation (LREC)*, 2018.

- [42] C. Meister, W. Stokowiec, T. Pimentel, L. Yu, L. Rimell, and A. Kuncoro, “A natural bias for language generation models,” in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, arXiv:2212.09686, 2023.
- [43] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, “Focal loss for dense object detection,” in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 2980–2988.
- [44] T. Nguyen et al., “CulturaX: A cleaned, enormous, and multilingual dataset for large language models in 167 languages,” in *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING)*, arXiv:2309.09400, 2024, pp. 4226–4237.
- [45] K. V. Nguyen, S. Q. Tran, L. T. Nguyen, T. V. Huynh, S. T. Luu, and N. L.-T. Nguyen, “VLSP 2021 – ViMRC challenge: Vietnamese machine reading comprehension,” *VNU Journal of Science: Computer Science and Communication Engineering*, 2022, arXiv:2203.11400; introduces UIT-ViQuAD 2.0.
- [46] A. Conneau et al., “XNLI: Evaluating cross-lingual sentence representations,” in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2018.
- [47] T. V. Huynh, K. V. Nguyen, and N. L.-T. Nguyen, “A new benchmark dataset and mixture-of-experts language models for adversarial natural language inference in Vietnamese,” *Expert Systems with Applications*, 2024, arXiv:2406.17716; ViANLI dataset.
- [48] A. Wang, A. Singh, J. Michael, F. Hill, O. Levy, and S. R. Bowman, “GLUE: A multi-task benchmark and analysis platform for natural language understanding,” in *Proceedings of the 2018 EMNLP Workshop BlackboxNLP*, 2018, pp. 353–355.

- [49] K. V. Nguyen, V. D. Nguyen, P. X. V. Nguyen, T. T. H. Truong, and N. L.-T. Nguyen, “UIT-VSFC: Vietnamese students’ feedback corpus for sentiment analysis,” in *Proceedings of the 10th International Conference on Knowledge and Systems Engineering (KSE)*, 2018.
- [50] V. A. Ho et al., “Emotion recognition for Vietnamese social media text,” in *Proceedings of the 16th International Conference of the Pacific Association for Computational Linguistics (PACLING 2019)*, Springer CCIS; arXiv:1911.09339; UIT-VSMEC dataset, 2020.
- [51] L. T. Nguyen, K. V. Nguyen, and N. L.-T. Nguyen, “Constructive and toxic speech detection for open-domain social media comments in Vietnamese,” in *Proceedings of the 34th International Conference on Industrial, Engineering and Other Applications of Applied Intelligent Systems (IEA/AIE)*, arXiv:2103.10069; UIT-ViCTSD dataset, 2021.
- [52] A. Warstadt, A. Singh, and S. R. Bowman, “Neural network acceptability judgments,” *Transactions of the Association for Computational Linguistics (TACL)*, vol. 7, pp. 625–641, 2019, CoLA dataset.
- [53] P.-T. Nguyen, X.-L. Vu, T.-M.-H. Nguyen, V.-H. Nguyen, and H.-P. Le, “Building a large syntactically-annotated corpus of Vietnamese,” in *Proceedings of the Third Linguistic Annotation Workshop (LAW III)*, Vietnamese Treebank; source of UD_Vietnamese-VTB, 2009.
- [54] D. Cer, M. Diab, E. Agirre, I. Lopez-Gazpio, and L. Specia, “SemEval-2017 task 1: Semantic textual similarity multilingual and cross-lingual focused evaluation,” in *Proceedings of the 11th International Workshop on Semantic Evaluation (SemEval-2017)*, 2017, pp. 1–14.
- [55] A. Conneau et al., “Unsupervised cross-lingual representation learning at scale,” in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020, pp. 8440–8451.

- [56] J. Salazar, D. Liang, T. Q. Nguyen, and K. Kirchhoff, “Masked language model scoring,” in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020, pp. 2699–2712.

Phụ lục A

Bộ lọc chính tả tiếng Việt cho khai thác neo

Phụ lục này mô tả chi tiết bộ lọc chính tả được dùng ở Mục 3.4.3 để bảo đảm các ứng viên khai thác cặp dịch là từ tiếng Việt hợp lệ. Một token chỉ được giữ lại khi vượt qua *toàn bộ* các quy tắc sau:

1. **Ký tự bản địa:** loại mọi token chứa f, j, w, z – các ký tự không thuộc bảng chữ cái tiếng Việt.
2. **Độ dài:** tối đa 7 ký tự cho một âm tiết.
3. **Một cụm nguyên âm:** token phải chứa *đúng một* cụm nguyên âm liền mạch (kể cả nguyên âm mang dấu thanh).
4. **Quy tắc phụ âm đầu:**
 - k, gh, ngh bắt buộc đứng trước các nguyên âm hàng trước $i, e, ê, y$;
 - ngược lại, c, g, ng không được đứng trước các nguyên âm này;
 - q luôn đi kèm u ;
 - p không đứng một mình làm phụ âm đầu (loại các dạng vay mượn như $pô, pin$).

5. **Tổ hợp nguyên âm ngoại lai:** loại các chuỗi không xuất hiện trong âm tiết tiếng Việt như *ea, ee, oo, ii, uu, ion, eon*.
6. **Nguyên âm chưa trọn vẹn:** các nguyên âm như *ă, â* (và các tổ hợp *iê, uô, ươ*) không được đứng cuối âm tiết mà phải có phụ âm cuối đi kèm (loại các mảnh từ như *nhấ, tầ, nghiê*).
7. **Phụ âm đầu/cuối hợp lệ:** phụ âm đầu phải thuộc tập $\{\emptyset, b, c, ch, d, đ, g, gh, gi, h, k, kh, l, m, n, ng, ngh, nh, ph, q, qu, r, s, t, th, tr, v, x\}$; phụ âm cuối phải thuộc tập $\{\emptyset, c, ch, m, n, ng, nh, p, t\}$.
8. **Quy tắc thanh-phụ âm cuối:** âm tiết kết thúc bằng *c, ch, p, t* bắt buộc mang thanh sắc hoặc thanh nặng (loại các dạng sai như *sec, nhac, mít*).
9. **Danh sách loại trừ:** một danh sách nhỏ các chuỗi thỏa cấu trúc âm tiết nhưng là nhiễu trong thực tế (ví dụ mảnh phiên âm, từ cổ ít dùng) được loại thủ công.

Ngoài ra, ở Tầng 3 của quá trình khai thác cặp dịch (Mục 3.4.3), một danh sách chặn (blacklist) gồm các từ chức năng và từ dễ nhập nhằng của tiếng Việt (*các, những, được, để, thế, cái, ...*) được áp dụng trước khi dịch, nhằm tránh tạo ra các cặp neo kém tin cậy từ những từ có phân bố ngữ nghĩa rộng.

Phụ lục B

Định nghĩa hình thức các độ đo đánh giá

Phụ lục này trình bày định nghĩa hình thức của các độ đo được dùng ở Mục 4.3. Các định nghĩa dạng văn xuôi đã được nêu trong phần thân; phần dưới đây bổ sung công thức tường minh để tiện tra cứu.

Với hỏi đáp đọc hiểu, F1 ở cấp token là trung bình điều hòa của độ chính xác P và độ bao phủ R giữa câu trả lời dự đoán và đáp án chuẩn, như Công thức (B.1):

$$F1 = \frac{2PR}{P + R}. \quad (B.1)$$

Hệ số tương quan Matthews cho phân loại nhị phân, tính từ các đại lượng dương thật (TP), âm thật (TN), dương giả (FP) và âm giả (FN), như Công thức (B.2):

$$MCC = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}. \quad (B.2)$$

Với truy hồi và xếp hạng lại, gọi $rel_i \in \{0, 1\}$ là mức liên quan của kết quả ở hạng i và $rank_q$ là thứ hạng của ngữ cảnh đúng đầu tiên cho truy

vấn q , hai độ đo được xác định như Công thức (B.3):

$$\text{nDCG}@k = \frac{1}{Z} \sum_{i=1}^k \frac{2^{\text{rel}_i} - 1}{\log_2(i + 1)}, \quad \text{MRR} = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{\text{rank}_q}, \quad (\text{B.3})$$

trong đó Z là hệ số chuẩn hóa (DCG của thứ hạng lý tưởng) đưa nDCG về đoạn $[0, 1]$, và Q là tập truy vấn đánh giá.